



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΕΣΣΑΛΙΑΣ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

Συγκριτική Ανάλυση Αλγορίθμων Ταξινόμησης Ακεραίων στη Γλώσσα
Προγραμματισμού Python

Κωνσταντίνος Ρομπόκας

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

ΥΠΕΥΘΥΝΟΣ

Δαδαλιάρης Αντώνιος
Επίκουρος Καθηγητής

Λαμία Ιούλιος έτος 2026



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΕΣΣΑΛΙΑΣ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

Συγκριτική Ανάλυση Αλγορίθμων Ταξινόμησης Ακεραίων στη Γλώσσα
Προγραμματισμού Python

Κωνσταντίνος Ρομπόκας

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

ΥΠΕΥΘΥΝΟΣ

Δαδαλιάρης Αντώνιος
Επίκουρος Καθηγητής

Λαμία Ιούλιος έτος 2026



UNIVERSITY OF
THESSALY

SCHOOL OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE & TELECOMMUNICATIONS

Comparative Analysis of Integer Sorting Algorithms in the Python Programming Language

Konstantinos Rompokas

FINAL THESIS

ADVISOR

Dadaliaris Antonios
Assistant Professor

Lamia July year 2026

«Με ατομική μου ευθύνη και γνωρίζοντας τις κυρώσεις ⁽¹⁾, που προβλέπονται από της διατάξεις της παρ. 6 του άρθρου 22 του Ν. 1599/1986, δηλώνω ότι:

1. Δεν παραθέτω κομμάτια βιβλίων ή άρθρων ή εργασιών άλλων αυτολεξεί **χωρίς να τα περικλείω σε εισαγωγικά** και χωρίς να αναφέρω το συγγραφέα, τη χρονολογία, τη σελίδα. Η αυτολεξεί παράθεση χωρίς εισαγωγικά χωρίς αναφορά στην πηγή, είναι λογοκλοπή. Πέραν της αυτολεξεί παράθεσης, λογοκλοπή θεωρείται και η παράφραση εδαφίων από έργα άλλων, συμπεριλαμβανομένων και έργων συμφοιτητών μου, καθώς και η παράθεση στοιχείων που άλλοι συνέλεξαν ή επεξεργάσθηκαν, χωρίς αναφορά στην πηγή. Αναφέρω πάντοτε με πληρότητα την πηγή κάτω από τον πίνακα ή σχέδιο, όπως στα παραθέματα.

2. Δέχομαι ότι η αυτολεξεί **παράθεση χωρίς εισαγωγικά**, ακόμα κι αν συνοδεύεται από αναφορά στην πηγή σε κάποιο άλλο σημείο του κειμένου ή στο τέλος του, είναι αντιγραφή. Η αναφορά στην πηγή στο τέλος π.χ. μιας παραγράφου ή μιας σελίδας, δεν δικαιολογεί συρραφή εδαφίων έργου άλλου συγγραφέα, έστω και παραφρασμένων, και παρουσίασή τους ως δική μου εργασία.

3. Δέχομαι ότι υπάρχει επίσης περιορισμός στο μέγεθος και στη συχνότητα των παραθεμάτων που μπορώ να εντάξω στην εργασία μου εντός εισαγωγικών. Κάθε μεγάλο παράθεμα (π.χ. σε πίνακα ή πλαίσιο, κλπ), προϋποθέτει ειδικές ρυθμίσεις, και όταν δημοσιεύεται προϋποθέτει την άδεια του συγγραφέα ή του εκδότη. Το ίδιο και οι πίνακες και τα σχέδια

4. Δέχομαι όλες τις συνέπειες σε περίπτωση λογοκλοπής ή αντιγραφής.

Ημερομηνία: 05/07/2026

Ο - Η Δηλ.
Κωνσταντίνος Ρομπόκας



(1) «Όποιος εν γνώσει του δηλώνει ψευδή γεγονότα ή αρνείται ή αποκρύπτει τα αληθινά με έγγραφη υπεύθυνη δήλωση του άρθρου 8 παρ. 4 Ν. 1599/1986 τιμωρείται με φυλάκιση τουλάχιστον τριών μηνών. Εάν ο υπαίτιος αυτών των πράξεων σκόπευε να προσπορίσει στον εαυτόν του ή σε άλλον περιουσιακό όφελος βλάπτοντας τρίτον ή σκόπευε να βλάψει άλλον, τιμωρείται με κάθειρξη μέχρι 10 ετών.»

Η παρούσα πτυχιακή εργασία εξετάζει συγκριτικά τη συμπεριφορά τριάντα βασικών αλγορίθμων ταξινόμησης ακεραίων, comparison-based και non comparison-based, καθώς και επτά βελτιστοποιημένων παραλλαγών τους, μέσω υλοποιήσεων σε Python 3.13. Στόχος είναι να διαπιστωθεί κατά πόσο η θεωρητική πολυπλοκότητα κάθε αλγορίθμου, σε χρόνο και σε μνήμη, επιβεβαιώνεται στην πράξη, όταν ο κώδικας τρέχει σε πραγματικό περιβάλλον (WSL2, Python interpreter, χωρίς βελτιστοποιήσεις σε επίπεδο C). Οι αλγόριθμοι δοκιμάστηκαν σε δεδομένα μεγέθους 1.000 έως 1.000.000 στοιχείων, σε τρία σενάρια (best case, random case, worst case), με μέτρηση του χρόνου εκτέλεσης μέσω `time.perf_counter` και της κατανάλωσης μνήμης μέσω `tracemalloc`. Τα αποτελέσματα δείχνουν πως ο Heap Sort, παρά την πολυπλοκότητα $n \log n$, ξεχωρίζει ανάμεσα στους αλγορίθμους αντίστοιχης πολυπλοκότητας, συνδυάζοντας χαμηλή και σταθερή κατανάλωση μνήμης (0.96 KB στο μεγαλύτερο μέγεθος εισόδου) με συνεπή χρόνο εκτέλεσης και καμία αποτυχία σε κανένα σενάριο. Αντίθετα, αλγόριθμοι όπως ο Block Sort και ο Merge-Insertion Sort υποαποδίδουν σημαντικά λόγω του overhead που εισάγει η καθαρή υλοποίησή τους σε Python. Ο πρώτος αποτυγχάνει να ολοκληρώσει εντός ορίου χρόνου στο μεγαλύτερο σύνολο δεδομένων, ενώ ο δεύτερος απαιτεί δεκάδες φορές περισσότερη μνήμη. Παράλληλα, ο Simplified Tim Sort, υλοποιημένος εξ ολοκλήρου σε Python, παρουσιάζει έντονη υποβάθμιση στο worst case σε σχέση με το best case, ακριβώς λόγω του interpreter overhead. Αυτή η διαφορά αναδεικνύεται ξεκάθαρα μέσα από τη σύγκριση με τον βελτιστοποιημένο σε C built-in Tim Sort, καθιστώντας εμφανή την απόσταση ανάμεσα σε θεωρία και πράξη κατά την υλοποίηση αλγορίθμων σε μια γλώσσα υψηλού επιπέδου.

ABSTRACT

This thesis presents a comparative study of thirty core integer sorting algorithms, both comparison-based and non-comparison-based, along with seven optimized variants of them, implemented in Python 3.13. The goal is to check whether the theoretical time and space complexity of each algorithm holds up in practice, when the code runs in a real environment (WSL2, Python interpreter, without low level C optimizations). The algorithms were tested on datasets ranging from 1.000 to 1.000.000 elements, under three scenarios (best case, random case, and worst case), with execution time measured via `time.perf_counter` and memory usage via `tracemalloc`. The results show that Heap Sort, despite its $n \log n$ complexity, stands out among algorithms of the same complexity class by combining low and consistent memory usage (0.96 KB at the largest input size) with stable execution time and zero failures across all scenarios. In contrast, algorithms such as Block Sort and Merge-Insertion Sort underperform significantly due to the overhead introduced by their pure Python implementation. The former fails to complete within the time limit on the largest dataset, while the latter requires tens of times more memory. At the same time, the Simplified Tim Sort, implemented entirely in Python, shows a sharp degradation in the worst case compared to the best case, precisely because of interpreter overhead. This gap becomes clear through comparison with the C optimized built-in Tim Sort, ultimately highlighting the distance between theory and practice when implementing sorting algorithms in a high-level language.

ΠΕΡΙΛΗΨΗ	1
ABSTRACT	2
ΚΕΦΑΛΑΙΟ 1 Εισαγωγή	0
1.1 Αντικείμενο και Υπόβαθρο της Εργασίας	0
1.2 Κίνητρο και Σημασία της Έρευνας	0
1.3 Σκοπός και Στόχοι της Εργασίας	0
1.4 Δομή της Εργασίας	1
ΚΕΦΑΛΑΙΟ 2 Βιβλιογραφική Επισκόπηση	2
2.1 Εισαγωγή στους Αλγορίθμους Ταξινόμησης	2
2.1.1 Κατηγοριοποίηση των Αλγορίθμων	2
2.2 Απλοί Αλγόριθμοι Σύγκρισης	2
2.2.1 Bubble Sort	3
2.2.2 Selection Sort	3
2.2.3 Insertion Sort	3
2.2.4 Gnome Sort	4
2.2.5 Cocktail Shaker Sort	4
2.2.6 Cycle Sort	4
2.2.7 Pancake Sort	4
2.2.8 Stooge Sort	4
2.3 Προηγμένοι Αλγόριθμοι Σύγκρισης	5
2.3.1 Merge Sort	5
2.3.2 Quick Sort	6
2.3.3 Heap Sort	6
2.3.4 Shell Sort	6
2.3.5 Comb Sort	7
2.3.6 Smooth Sort	7
2.3.7 Strand Sort	8
2.3.8 Tree Sort	8
2.3.9 Cartesian Tree Sort	8
2.3.10 Tournament Sort	9
2.3.11 Patience Sort	9
2.4 Δίκτυα Ταξινόμησης	10
2.4.1 Bitonic Sort	10
2.4.2 Pairwise Sorting Network	10
2.5 Υβριδικοί & Σύνθετοι	11
2.5.1 Tim Sort	11
2.5.2 Intro Sort	12
2.5.3 Block Sort	13
2.5.4 Merge-Insertion Sort	13

2.6 Αλγόριθμοι Χωρίς Σύγκριση	14
2.6.1 Counting Sort	14
2.6.2 Radix Sort	15
2.6.3 Bucket Sort	15
2.6.4 Pigeonhole Sort	16
2.6.5 Flash Sort	16
ΚΕΦΑΛΑΙΟ 3 Μεθοδολογία	18
3.1 Γενική Περιγραφή	18
3.2 Περιβάλλον Εκτέλεσης	18
3.3 Υλοποίηση Αλγορίθμων	18
3.3.1 Αμειγείς Υλοποιήσεις σε Python	18
3.3.2 Βελτιστοποιημένες Υλοποιήσεις (Alt)	19
3.4 Σχεδιασμός Δεδομένων Εισόδου	19
3.4.1 Μεγέθη Πίνακα	19
3.4.2 Κατηγορίες Εισόδου	20
3.5 Εργαλεία Μέτρησης	20
3.5.1 Μέτρηση Χρόνου Εκτέλεσης	20
3.5.2 Μέτρηση Κατανάλωσης Μνήμης	21
3.6 Μέτρα Αξιοπιστίας Μετρήσεων	21
3.6.1 Διαχείριση Timeout	21
3.6.2 Διαχείριση Ορίου Αναδρομής	21
3.6.3 Αντιμετώπιση Θερμικής Επιβράδυνσης	22
3.6.4 Garbage Collector	22
3.7 Περιορισμοί της Μελέτης	22
3.7.1 Όριο Αναδρομής και Σταθερότητα Συστήματος	22
3.7.2 Μονοεπαναληπτικό Πειραματικό Σχέδιο	23
3.7.3 Στατική Παύση και Απουσία Δυναμικής Θερμικής Διαχείρισης	23
ΚΕΦΑΛΑΙΟ 4 Αποτελέσματα και Συγκριτική Ανάλυση	25
4.1 Επισκόπηση Αποτελεσμάτων	25
4.1.1 Αποτυχίες Εκτέλεσης - Time Limit & Recursion Limit	25
4.1.2 Ερμηνεία των αποτυχιών Time Limit	26
4.1.3 Ερμηνεία των αποτυχιών Recursion Limit	27
4.1.4 Γενική Εικόνα του Συνόλου Δεδομένων	27
4.2 Αλγόριθμοι Τετραγωνικής Πολυπλοκότητας $O(n^2)$	27
4.2.1 Βασικές Υλοποιήσεις - Bubble Sort, Selection Sort, Insertion Sort	27
4.2.2 Παραλλαγές & εξειδικευμένοι - Gnome, Cocktail Shaker, Cycle, Pancake	29
4.3 Αλγόριθμοι $O(n \log n)$	30
4.3.1 Βασικοί - Merge Sort, Quick Sort, Heap Sort	30

4.3.2 Προσαρμοστικοί και Gap-Based Αλγόριθμοι - Shell, Comb, Smooth, Strand Sort	32
4.3.3 Αλγόριθμοι βασισμένοι σε δέντρα - Tree, Cartesian Tree, Tournament, Patience Sort	33
4.4 Υβριδικοί Αλγόριθμοι	35
4.4.1 Σύγχρονες προσεγγίσεις - Simplified Tim Sort, Intro Sort	35
4.4.2 Λοιποί - Block Sort, Merge-Insertion Sort	36
4.5 Δίκτυα Ταξινόμησης	38
4.5.1 Bitonic Sort, Pairwise Sorting Network	38
4.6 Non-Comparison Αλγόριθμοι $O(n+k)$	39
4.6.1 Αλγόριθμοι κατανομής - Counting, Radix, Pigeonhole	39
4.6.2 Αλγόριθμοι κάδων - Bucket, Flash Sort	41
4.7 Συγκριτική Ανάλυση Αμιγούς Κώδικα Python και Βελτιστοποιημένων Υλοποιήσεων (Alt)	42
4.7.1 Οριακή Επιτάχυνση και Εξάλειψη Υπερβάσεων Χρόνου	42
4.7.2 Σημαντική Επιτάχυνση μέσω Δομών Heap και Δυναδικής Αναζήτησης	44
4.7.3 Tim Sort (Built-in) έναντι Simplified Tim Sort	46
4.7.4 Επίδραση της Βελτιστοποίησης στην Κατανάλωση Μνήμης	47
4.8 Συνολική Συγκριτική Αξιολόγηση	48
4.8.1 Ανάλυση Μνήμης σε Όλες τις Κατηγορίες Αλγορίθμων	48
4.8.2 Βέλτιστες Αλγοριθμικές Επιλογές ανά Σενάριο Εκτέλεσης	51
ΚΕΦΑΛΑΙΟ 5 Συμπεράσματα	53
5.1 Σύνοψη Ερευνητικής Προσπάθειας	53
5.2 Κύρια Ευρήματα	53
5.3 Πρακτικές Συστάσεις	55
5.4 Περιορισμοί και Προτάσεις Μελλοντικής Έρευνας	56
ΒΙΒΛΙΟΓΡΑΦΙΑ	59

ΚΕΦΑΛΑΙΟ 1 Εισαγωγή

1.1 Αντικείμενο και Υπόβαθρο της Εργασίας

Η ταξινόμηση δεδομένων αποτελεί μια θεμελιώδη διαδικασία στην επιστήμη της Πληροφορικής. Η αναγκαιότητά της εντοπίζεται σε ένα ευρύ φάσμα σύγχρονων εφαρμογών. Τέτοιες εφαρμογές είναι η διαχείριση βάσεων δεδομένων, η επεξεργασία μεγάλου όγκου δεδομένων και η βελτιστοποίηση των αλγορίθμων αναζήτησης. Παραδοσιακά, η θεωρητική μελέτη των αλγορίθμων ταξινόμησης βασίζεται στην ανάλυση πολυπλοκότητας χρόνου και χώρου. Η ανάλυση αυτή χρησιμοποιεί τον καθιερωμένο συμβολισμό του ασυμπτωτικού άνω φράγματος (συμβολισμός Big-O). Ο συμβολισμός αυτός περιγράφει τη συμπεριφορά των αλγορίθμων καθώς το μέγεθος της εισόδου αυξάνεται. Στην πράξη, ωστόσο, η εμπειρική απόδοση ενός αλγορίθμου εξαρτάται από επιπλέον παράγοντες του συστήματος. Τέτοιοι παράγοντες περιλαμβάνουν την αρχιτεκτονική του επεξεργαστή, τη διαχείριση της μνήμης και τα χαρακτηριστικά της γλώσσας προγραμματισμού.

1.2 Κίνητρο και Σημασία της Έρευνας

Η γλώσσα προγραμματισμού Python είναι ιδιαίτερα δημοφιλής στον ακαδημαϊκό και επαγγελματικό χώρο. Η δημοτικότητά της οφείλεται στην απλότητα του συντακτικού και στην ταχύτητα ανάπτυξης λογισμικού. Ωστόσο, η Python αποτελεί μια διερμηνευόμενη γλώσσα (interpreted language). Αυτό το χαρακτηριστικό εισάγει συγκεκριμένες ιδιαιτερότητες κατά την εκτέλεση του κώδικα. Ο διερμηνέας προκαλεί σημαντική υπολογιστική επιβάρυνση (overhead). Αυτή η επιβάρυνση μειώνει την ταχύτητα εκτέλεσης των αμιγών υλοποιήσεων. Για την αντιμετώπιση αυτού του περιορισμού, η Python ενσωματώνει έτοιμες βιβλιοθήκες και συναρτήσεις. Οι δομές αυτές είναι υλοποιημένες σε γλώσσα C και εκτελούνται ως μεταγλωττισμένος κώδικας, παρακάμπτοντας τον βρόχο διερμηνείας (interpreter loop) της CPython και το σχετικό υπολογιστικό κόστος ανά εντολή, γεγονός που τις καθιστά σημαντικά ταχύτερες από αμιγείς υλοποιήσεις σε Python. Αυτό δημιουργεί μια ενδιαφέρουσα ερευνητική πρόκληση. Προκύπτει η σαφής ανάγκη για την ποσοτική καταγραφή της διαφοράς ανάμεσα στον αμιγή κώδικα Python και στις βελτιστοποιημένες εκδοχές. Η κατανόηση αυτών των εμπειρικών αποκλίσεων είναι κρίσιμη για την ανάπτυξη αποδοτικών εφαρμογών.

1.3 Σκοπός και Στόχοι της Εργασίας

Ο κύριος σκοπός της παρούσας πτυχιακής εργασίας είναι η συγκριτική αξιολόγηση αλγορίθμων ταξινόμησης ακέραιων αριθμών στην Python. Η έρευνα δεν περιορίζεται στις κλασικές μεθόδους. Επεκτείνεται σε ένα ευρύ φάσμα αλγοριθμικών προσεγγίσεων. Συγκεκριμένα, η μελέτη εξετάζει αλγορίθμους τετραγωνικής και λογαριθμικής πολυπλοκότητας, δίκτυα ταξινόμησης, καθώς και μεθόδους χωρίς σύγκριση.

Για την επίτευξη του κεντρικού σκοπού, η έρευνα θέτει τους εξής επιμέρους στόχους:

- Την ανάπτυξη ενός μεγάλου πλήθους αλγορίθμων σε αμιγή κώδικα Python.
- Την υλοποίηση βελτιστοποιημένων παραλλαγών με τη χρήση έτοιμων δομών της γλώσσας.
- Τον σχεδιασμό ενός αυστηρού πειραματικού πλαισίου για την ταυτόχρονη μέτρηση χρόνου εκτέλεσης και βοηθητικής μνήμης.
- Την αξιολόγηση των αλγορίθμων σε κλιμακούμενα μεγέθη εισόδου (έως ένα εκατομμύριο στοιχεία).
- Την εξέταση διαφορετικών κατανομών δεδομένων (καλύτερη, τυχαία και χειρότερη περίπτωση εισόδου).
- Τη διατύπωση πρακτικών συστάσεων για την επιλογή του κατάλληλου αλγορίθμου βάσει των διαθέσιμων πόρων.

1.4 Δομή της Εργασίας

Το περιεχόμενο της πτυχιακής εργασίας οργανώνεται σε πέντε διακριτά κεφάλαια.

- Το Κεφάλαιο 1 αποτελεί την εισαγωγή της μελέτης. Εκεί περιγράφεται το γενικό αντικείμενο, το κίνητρο, ο σκοπός και οι στόχοι της έρευνας.
- Το Κεφάλαιο 2 περιλαμβάνει τη βιβλιογραφική ανασκόπηση και τη θεωρητική ανάλυση των αλγορίθμων. Οι μέθοδοι ταξινομούνται και παρουσιάζονται ανάλογα με τη θεωρητική τους πολυπλοκότητα και τον τρόπο λειτουργίας τους.
- Το Κεφάλαιο 3 παραθέτει τη μεθοδολογία της έρευνας και την αρχιτεκτονική του πειράματος. Στο τμήμα αυτό αναλύονται οι τεχνικές προδιαγραφές, οι τεχνικές μέτρησης και οι περιορισμοί της διαδικασίας.
- Το Κεφάλαιο 4 αναλύει διεξοδικά τα εμπειρικά αποτελέσματα. Η ενότητα αυτή εστιάζει στη συγκριτική ανάλυση του χρόνου και της μνήμης, αναδεικνύοντας τις βέλτιστες επιλογές ανά σενάριο.
- Το Κεφάλαιο 5 συνοψίζει τα τελικά συμπεράσματα και τα κύρια ευρήματα. Παράλληλα, προσφέρει πρακτικές οδηγίες προς τους δημιουργούς λογισμικού και διατυπώνει προτάσεις για μελλοντική έρευνα.

ΚΕΦΑΛΑΙΟ 2 Βιβλιογραφική Επισκόπηση

2.1 Εισαγωγή στους Αλγορίθμους Ταξινόμησης

Η ταξινόμηση αποτελεί ένα από τα θεμελιώδη προβλήματα στην επιστήμη των υπολογιστών. Ορίζεται ως η αναδιάταξη μιας ακολουθίας n στοιχείων κατά αύξουσα ή φθίνουσα σειρά σύμφωνα με κάποια σχέση διάταξης. Πέρα από την αυτόνομη χρησιμότητά της, η ταξινόμηση αποτελεί προαπαιτούμενο βήμα για πλήθος άλλων αλγορίθμων, γεγονός που της προσδίδει κεντρική θέση στη θεωρητική και πρακτική πληροφορική (Cormen, 2009).

Η μελέτη των αλγορίθμων ταξινόμησης ξεκινά ουσιαστικά με το έργο του Donald Knuth, ο οποίος στον τρίτο τόμο της σειράς *The Art of Computer Programming* κατέγραψε και ανέλυσε συστηματικά τις οικογένειες εσωτερικής και εξωτερικής ταξινόμησης, θέτοντας τις βάσεις για κάθε μεταγενέστερη βιβλιογραφική αναφορά στο πεδίο (Knuth, 1997).

2.1.1 Κατηγοριοποίηση των Αλγορίθμων

Η πιο ευρέως αποδεκτή κατηγοριοποίηση στη βιβλιογραφία διακρίνει δύο μεγάλες οικογένειες, τους αλγορίθμους βάσει σύγκρισης (comparison-based) και τους αλγορίθμους χωρίς σύγκριση (non-comparison-based). Οι πρώτοι προσδιορίζουν τη σχετική τάξη των στοιχείων αποκλειστικά μέσω συγκρίσεων ζευγών, ενώ οι δεύτεροι εκμεταλλεύονται ιδιότητες της αριθμητικής αναπαράστασης των δεδομένων για να επιτύχουν γραμμική πολυπλοκότητα.

Το κρίσιμο θεωρητικό αποτέλεσμα για τους comparison-based αλγορίθμους είναι το κατώτατο όριο $\Omega(n \log n)$ το οποίο μας δείχνει ότι κανένας αλγόριθμος που βασίζεται αποκλειστικά σε συγκρίσεις δεν μπορεί να ταξινομήσει n στοιχεία σε χρόνο μικρότερο από $\Omega(n \log n)$ στη χειρότερη περίπτωση (Cormen, 2009). Η απόδειξη βασίζεται στο μοντέλο του decision tree, κάθε αλγόριθμος σύγκρισης αντιστοιχεί σε ένα δυαδικό δέντρο αποφάσεων με τουλάχιστον $n!$ φύλλα (οι δυνατές διατάξεις εισόδου), οπότε το ύψος του δέντρου είναι τουλάχιστον $\log_2(n!) \approx n \log n$ (Cormen, 2009). Αυτό ακριβώς δικαιολογεί γιατί αλγόριθμοι όπως Merge Sort και Heap Sort θεωρούνται ασυμπτωτικά βέλτιστοι για τη γενική περίπτωση, ενώ απλούστεροι όπως Bubble Sort και Selection Sort υπολείπονται σε απόδοση για μεγάλα n .

2.2 Απλοί Αλγόριθμοι Σύγκρισης

Οι αλγόριθμοι αυτής της κατηγορίας αποτελούν ιστορικά τις πρώτες συστηματικές προσεγγίσεις στο πρόβλημα της ταξινόμησης. Χαρακτηρίζονται από απλή λογική υλοποίησης και τετραγωνική χρονική πολυπλοκότητα $O(n^2)$ στη μέση και χειρότερη περίπτωση. Αυτό τους κάνει ακατάλληλους για μεγάλα σύνολα δεδομένων, αλλά αρκετά αποδοτικούς για μικρά n ή σχεδόν ταξινομημένες ακολουθίες (Knuth, 1998).

Αλγόριθμος	Best	Random	Worst	Μνήμη	Σταθερότητα
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ναι
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Όχι
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ναι
Gnome Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ναι
Cocktail Shaker	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ναι
Cycle Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Όχι
Pancake Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Όχι
Stooge Sort	$O(n^{2.709})$	$O(n^{2.709})$	$O(n^{2.709})$	$O(n)$	Όχι

2.2.1 Bubble Sort

Ο Bubble Sort είναι ο απλούστερος αλγόριθμος ταξινόμησης, σε κάθε πέρασμα από τον πίνακα συγκρίνει διαδοχικά ζεύγη στοιχείων και τα ανταλλάσσει αν δεν βρίσκονται στη σωστή σειρά, σπρώχνοντας σταδιακά το μεγαλύτερο στοιχείο στο τέλος. Η ονομασία προέρχεται ακριβώς από αυτή την εικόνα, τα μεγαλύτερα στοιχεία ανεβαίνουν σαν φυσαλίδες προς την επιφάνεια (Knuth, 1998).

Σε βελτιστοποιημένη εκδοχή του, ένας απλός έλεγχος αν έγινε κάποια ανταλλαγή στο τελευταίο πέρασμα, κάνει τον αλγόριθμο προσαρμοστικό (adaptive), επιτρέποντάς του να τερματίζει σε $O(n)$ για ήδη ταξινομημένη είσοδο (Knuth, 1998).

2.2.2 Selection Sort

Ο Selection Sort επιλέγει σε κάθε βήμα το ελάχιστο στοιχείο από το αταξινόμητο τμήμα και το τοποθετεί στη σωστή θέση μέσω μιας μόνο ανταλλαγής. Παρά την απλότητά του, δεν είναι adaptive και εκτελεί πάντα $O(n^2)$ συγκρίσεις ανεξαρτήτως της αρχικής διάταξης. Το πλεονέκτημά του έναντι του Bubble Sort είναι το μικρότερο πλήθος ανταλλαγών $O(n)$ στη χειρότερη περίπτωση (Cormen, 2009), γεγονός που τον κάνει χρήσιμο σε συστήματα όπου κάθε εγγραφή σε μνήμη έχει κόστος, όπως σε SSD ή flash storage.

2.2.3 Insertion Sort

Ο Insertion Sort επεξεργάζεται τον πίνακα από αριστερά προς τα δεξιά. Διατηρεί ένα ταξινομημένο τμήμα στα αριστερά και σε κάθε βήμα εισάγει το επόμενο στοιχείο (key) στη σωστή θέση μέσα σε αυτό, μετατοπίζοντας δεξιά όλα τα ήδη ταξινομημένα στοιχεία που είναι μεγαλύτερα του. Είναι ο πιο αποδοτικός από τους απλούς αλγορίθμους για σχεδόν ταξινομημένα δεδομένα, με χρόνο $O(n + d)$ όπου d ο αριθμός των αντιστροφών (inversions - ζεύγη στοιχείων εκτός σειράς) (Cormen, 2009). Αυτή η ιδιότητα τον καθιστά βασικό δομικό στοιχείο υβριδικών αλγορίθμων όπως ο Tim Sort και ο Intro Sort.

2.2.4 Gnome Sort

Ο Gnome Sort (ή Stupid Sort) λειτουργεί με μια μοναδική μεταβλητή θέσης που κινείται μπρος-πίσω στον πίνακα προχωρώντας όταν τα γειτονικά στοιχεία είναι στη σωστή σειρά και υποχωρεί κάνοντας ανταλλαγή όταν δεν είναι. Η λογική του αλγορίθμου είναι ισοδύναμη με του Insertion Sort, με τη θεμελιώδη διαφορά ότι δεν χρησιμοποιεί εσωτερικό βρόχο, αποτελώντας ένα εναλλακτικό παράδειγμα υλοποίησης της ίδιας αλγοριθμικής λογικής (Sarbazi-Azad, 2000).

2.2.5 Cocktail Shaker Sort

Ο Cocktail Shaker Sort (γνωστός και ως Bidirectional Bubble Sort) αποτελεί βελτίωση του Bubble Sort, αντί να σαρώνει τον πίνακα μόνο από αριστερά προς τα δεξιά, εναλλάσσει κατεύθυνση σε κάθε πέρασμα. Αυτό αντιμετωπίζει αποτελεσματικότερα το πρόβλημα των μικρών στοιχείων στο τέλος του πίνακα, που ο κλασικός Bubble Sort μετακινεί πολύ αργά. Η υλοποίησή μας χρησιμοποιεί επιπλέον δύο δείκτες (start, end) που στενεύουν μετά από κάθε πέρασμα, αποφεύγοντας περιττές συγκρίσεις με στοιχεία που έχουν ήδη βρει την τελική τους θέση (Knuth, 1998).

2.2.6 Cycle Sort

Ο Cycle Sort βασίζεται στην παρατήρηση ότι κάθε πίνακας μπορεί να αναλυθεί σε κύκλους μεταθέσεων (permutation cycles), κάθε στοιχείο ανήκει σε έναν κύκλο και πρέπει να μετακινηθεί ακριβώς μία φορά στη θέση του. Το αποτέλεσμα είναι ο αλγόριθμος με το θεωρητικά ελάχιστο πλήθος εγγραφών σε σύγκριση με οποιονδήποτε άλλο αλγόριθμο in-place ($O(n)$ συνολικές εγγραφές), γεγονός που τον κάνει κατάλληλο για συστήματα με flash memory, όπου κάθε εγγραφή μειώνει τη διάρκεια ζωής της συσκευής (Haddon, 1990).

2.2.7 Pancake Sort

Ο Pancake Sort επιτρέπει μόνο μία λειτουργία, την αναστροφή (flip) ενός προθέματος (prefix) του πίνακα, δηλαδή την αντιστροφή της σειράς των πρώτων k στοιχείων. Το πρόβλημα ταξινόμησης με αναστροφές έχει ιδιαίτερη θεωρητική σημασία, καθώς ο Bill Gates δημοσίευσε το 1979 το μοναδικό επιστημονικό του άρθρο ακριβώς για αυτό το πρόβλημα, αποδεικνύοντας άνω φράγμα $(5n+5)/3$ αναστροφών (Gates & Papadimitriou, 1979). Ο αλγόριθμος δεν χρησιμοποιείται στην πράξη λόγω $O(n^2)$ πολυπλοκότητας, αλλά έχει εφαρμογές σε δίκτυα διασύνδεσης (interconnection networks) και γενετική βιολογία για τη μέτρηση χρωμοσωμικών αναδιατάξεων.

2.2.8 Stooge Sort

Ο Stooge Sort είναι ένας εξαιρετικά αργός αναδρομικός αλγόριθμος, ο οποίος αναλύεται κυρίως για εκπαιδευτικούς σκοπούς. Λειτουργεί ανταλλάσσοντας το πρώτο με το τελευταίο στοιχείο (αν χρειάζεται) και στη συνέχεια καλείται αναδρομικά για τα δύο πρώτα τρίτα, τα δύο τελευταία τρίτα, και ξανά τα δύο πρώτα τρίτα του πίνακα.

Αυτή η λογική διαίρεσης οδηγεί σε ασυμπτωτική χρονική πολυπλοκότητα $O(n^{2.709})$, καθιστώντας τον στην πράξη υποδεέστερο ακόμα και από απλούς αλγορίθμους όπως ο Bubble Sort (Cormen, 2009, Problem 7-3).

2.3 Προηγμένοι Αλγόριθμοι Σύγκρισης

Οι αλγόριθμοι αυτής της κατηγορίας επιτυγχάνουν χρονική πολυπλοκότητα $O(n \log n)$, η οποία είναι το θεωρητικά βέλτιστο κατώτατο όριο για οποιονδήποτε αλγόριθμο βάσει σύγκρισης. Πολλοί από αυτούς βασίζονται στη στρατηγική διαίρει και βασίλευε (divide-and-conquer), ενώ άλλοι αξιοποιούν προηγμένες δομές δεδομένων, όπως οι σωροί (heaps) και τα δέντρα αναζήτησης, για να βελτιστοποιήσουν τις συγκρίσεις (Cormen, 2009).

Αλγόριθμος	Best	Random	Worst	Μνήμη	Σταθερότητα
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Ναι
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Όχι
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Όχι
Shell Sort	$O(n \log n)$	Εξαρτάται από το χάσμα	$O(n^2) *$	$O(1)$	Όχι
Comb Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(1)$	Όχι
Smooth Sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Όχι
Strand Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(n)$	Ναι
Tree Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$	Ναι
Cartesian Tree	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Όχι
Tournament Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Όχι
Patience Sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Όχι

2.3.1 Merge Sort

Ο Merge Sort αναπτύχθηκε από τον John von Neumann το 1945, καθιστώντας τον έναν από τους ιστορικά πρώτους αλγορίθμους ταξινόμησης που σχεδιάστηκαν για ηλεκτρονικούς υπολογιστές. Μια λεπτομερής ανάλυσή του εμφανίστηκε σε τεχνική έκθεση των Goldstine και von Neumann ήδη από το 1948 (Knuth, 1998).

Ο αλγόριθμος ακολουθεί τυπική στρατηγική divide-and-conquer, διαιρεί αναδρομικά τον πίνακα εισόδου σε δύο ίσα τμήματα, ταξινομεί καθένα ξεχωριστά, και στη συνέχεια τα συγχωνεύει (merge) σε μία ταξινομημένη ακολουθία. Το βασικό βήμα είναι η συγχώνευση δύο ήδη ταξινομημένων υποπινάκων σε γραμμικό χρόνο $O(n)$, που διασφαλίζει τη συνολική πολυπλοκότητα $O(n \log n)$ τόσο στη μέση όσο και στη χειρότερη περίπτωση (Cormen, 2009).

Αξιοσημείωτη ιδιότητα του Merge Sort είναι η σταθερότητα (stability), ίσα στοιχεία διατηρούν τη σχετική τους σειρά από την είσοδο στην έξοδο. Ωστόσο, η τυπική εκτός-θέσης (out-of-place) υλοποίηση απαιτεί επιπλέον $O(n)$ χώρο για τους βοηθητικούς πίνακες, γεγονός που αποτελεί τον κύριο περιορισμό του σε memory-constrained περιβάλλοντα.

2.3.2 Quick Sort

Ο Quick Sort αναπτύχθηκε από τον Tony Hoare το 1959 και δημοσιεύτηκε το 1962 στο The Computer Journal (Hoare, 1962). Βασική ιδέα είναι η επιλογή ενός στοιχείου-άξονα (pivot) και η αναδιάταξη του πίνακα έτσι ώστε όλα τα στοιχεία μικρότερα του pivot να βρίσκονται αριστερά του και όλα τα μεγαλύτερα δεξιά, η λεγόμενη διαμέριση (partition). Η διαδικασία επαναλαμβάνεται αναδρομικά στα δύο τμήματα.

Η μέση πολυπλοκότητα είναι $O(n \log n)$ και ο αλγόριθμος λειτουργεί in-place, κάτι που τον καθιστά σημαντικά αποδοτικότερο σε μνήμη από τον Merge Sort. Η χειρότερη περίπτωση παραμένει $O(n^2)$ και εκδηλώνεται όταν ο pivot επιλέγεται συστηματικά ως το ελάχιστο ή μέγιστο στοιχείο (Cormen, 2009). Για τον λόγο αυτό, σύγχρονες υλοποιήσεις χρησιμοποιούν τεχνική median-of-three ή τυχαιοποίηση του pivot για να αντιμετωπίσουν πρακτικά τη χειρότερη περίπτωση. Στην υλοποίηση της παρούσας εργασίας χρησιμοποιήθηκε η στρατηγική middle element, το στοιχείο της μέσης θέσης μετακινείται στο τέλος του υποπίνακα και χρησιμοποιείται ως pivot. Η προσέγγιση αυτή είναι απλούστερη από το πλήρες median-of-three, αλλά αρκεί για να αποτρέψει τη χειρότερη περίπτωση σε ταξινομημένες εισόδους.

2.3.3 Heap Sort

Ο Heap Sort εφευρέθηκε από τον J. W. J. Williams (1964), στο ίδιο άρθρο που εισήγαγε τον δυαδικό σωρό (binary heap) ως αυτόνομη δομή δεδομένων. Στη συνέχεια, ο Robert W. Floyd (1964) δημοσίευσε βελτιωμένη έκδοση που οικοδομεί τον σωρό σε $O(n)$ χρόνο αντί $O(n \log n)$, η οποία αποτελεί τη βάση όλων των σύγχρονων υλοποιήσεων.

Ο αλγόριθμος λειτουργεί σε δύο φάσεις. Αρχικά, κατασκευάζει ένα max-heap από τον πίνακα (heapify), και στη συνέχεια εξάγει επαναληπτικά το μέγιστο στοιχείο από τη ρίζα και το τοποθετεί στο τέλος του πίνακα, επαναφέροντας την ιδιότητα σωρού (sift-down) κάθε φορά. Αυτή η δομή εγγυάται $O(n \log n)$ πολυπλοκότητα και στη χειρότερη περίπτωση, κάτι που ο Quick Sort δεν μπορεί να εγγυηθεί.

Παρά τις άριστες ασυμπτωτικές ιδιότητες, ο Heap Sort παρουσιάζει στην πράξη χειρότερη cache συμπεριφορά από τον Quick Sort. Ειδικότερα, η εκτέλεση sift-down απαιτεί προσπέλαση στοιχείων σε μη γραμμικές θέσεις μνήμης, οδηγώντας σε συχνά cache misses.

2.3.4 Shell Sort

Ο Shell Sort αναπτύχθηκε από τον Donald Shell (1959) και ήταν ο πρώτος αλγόριθμος ταξινόμησης που κατάφερε να σπάσει το φράγμα των $O(n^2)$ συγκρίσεων χωρίς να χρησιμοποιεί divide-and-conquer. Η βασική ιδέα είναι μια γενίκευση του Insertion Sort όπου αντί να συγκρίνει μόνο γειτονικά στοιχεία, συγκρίνει στοιχεία που απέχουν μεταξύ τους κατά ένα μεταβλητό κενό (gap), το οποίο μειώνεται σταδιακά σε κάθε πέρασμα μέχρι να φτάσει στο 1, οπότε ο αλγόριθμος εκτελεί ένα τελικό, κλασικό Insertion Sort.

Το κρίσιμο χαρακτηριστικό του Shell Sort είναι ότι η χρονική πολυπλοκότητά του εξαρτάται άμεσα από την επιλεγμένη ακολουθία κενών. Ενώ η αρχική ακολουθία του Shell ($N/2, N/4, \dots, 1$) δίνει πολυπλοκότητα $O(n^2)$ στη χειρότερη περίπτωση, με την εισαγωγή της ακολουθίας κενών του Hibbard (1963) της μορφής $2^k - 1$, η χρονική πολυπλοκότητα του αλγορίθμου βελτιώνεται σε $O(n^{3/2})$. Στην υλοποίηση της παρούσας εργασίας χρησιμοποιείται η αρχική ακολουθία του Shell, η οποία επιλέχθηκε για την απλότητά της, αποδίδοντας $O(n^2)$ στη χειρότερη περίπτωση.

2.3.5 Comb Sort

Ο Comb Sort προτάθηκε από τον Włodzimierz Dobosiewicz (1980) και αναδημοσιεύτηκε με πιο ευρεία διάδοση από τους Lacey & Box (1991). Ουσιαστικά αποτελεί επέκταση του Bubble Sort που υιοθετεί την ίδια λογική κενών με τον Shell Sort. Πιο συγκεκριμένα, αντί να συγκρίνει μόνο γειτονικά στοιχεία, χρησιμοποιεί αρχικά ένα μεγάλο κενό το οποίο μειώνεται σε κάθε πέρασμα με σταθερό συντελεστή συρρίκνωσης (shrink factor ≈ 1.3), έως ότου το κενό γίνει ίσο με 1. Αυτή η τακτική επιλύει το πρόβλημα των στοιχείων με μικρή τιμή που βρίσκονται στο τέλος του πίνακα γνωστά ως χελώνες (turtles), τα οποία στον κλασικό Bubble Sort απαιτούν μεγάλο αριθμό επαναλήψεων για να μετακινηθούν στις σωστές τους θέσεις (Knuth, 1998).

Παρόλο που η χειρότερη περίπτωση παραμένει θεωρητικά $O(n^2)$, στην πράξη ο Comb Sort συμπεριφέρεται κοντά στο $O(n \log n)$ για τυχαίες εισόδους, καθιστώντας τον αξιοσημείωτα ταχύτερο από τον Bubble Sort με μηδαμινή επιπλέον πολυπλοκότητα υλοποίησης.

2.3.6 Smooth Sort

Ο Smooth Sort σχεδιάστηκε από τον Edsger W. Dijkstra και δημοσιεύτηκε το 1982 στο περιοδικό Science of Computer Programming (Dijkstra, 1982). Αποτελεί προσαρμοστική παραλλαγή του Heap Sort και έχει σχεδιαστεί ειδικά για να εκμεταλλεύεται την υπάρχουσα τάξη της εισόδου. Ενώ ο κλασικός Heap Sort αγνοεί πλήρως αν ο πίνακας είναι ήδη ταξινομημένος, ο Smooth Sort εκτελείται σε $O(n)$ για ήδη ταξινομημένη είσοδο και σε $O(n \log n)$ για τυχαία ή αντίστροφη.

Η κεντρική καινοτομία του αλγορίθμου είναι η χρήση Leonardo heap αντί του κλασικού δυαδικού σωρού. Ο Leonardo heap δομείται από μια ακολουθία δέντρων

των οποίων τα μεγέθη ακολουθούν τους αριθμούς Leonardo ($L(0)=1, L(1)=1, L(k)=L(k-1)+L(k-2)+1$), μια ακολουθία παρόμοια με τους αριθμούς Fibonacci. Αυτή η δομή επιτρέπει in-place λειτουργία με $O(1)$ επιπλέον χώρο, διατηρώντας παράλληλα την προσαρμοστικότητα που λείπει από τον Heap Sort (Dijkstra, 1982). Στην πράξη ωστόσο, το υψηλό σταθερό κόστος υλοποίησης κάνει τον Smooth Sort σπάνια να υπερτερεί εμπειρικά έναντι απλούστερων adaptive αλγορίθμων όπως ο Tim Sort.

2.3.7 Strand Sort

Ο Strand Sort είναι ένας αναδρομικός αλγόριθμος που εκμεταλλεύεται φυσικά ταξινομημένα υποτμήματα (strands) που υπάρχουν ήδη μέσα στην άταχτη είσοδο. Σε κάθε πέρασμα, ο αλγόριθμος εξάγει από τον πίνακα εισόδου τη μεγαλύτερη αύξουσα υποακολουθία που μπορεί να βρει με γραμμική σάρωση (strand), και στη συνέχεια τη συγχωνεύει με τη λίστα αποτελεσμάτων. Η διαδικασία επαναλαμβάνεται μέχρι ο πίνακας εισόδου να αδειάσει.

Η πολυπλοκότητά του εξαρτάται άμεσα από τον αριθμό των φυσικών runs στην είσοδο. Στην καλύτερη περίπτωση (ήδη ταξινομημένος πίνακας) εκτελείται σε $O(n)$, ενώ στη χειρότερη (αντίστροφα ταξινομημένος) κάθε strand αποτελείται από ένα μόνο στοιχείο και ο αλγόριθμος εκπίπτει σε $O(n^2)$. Επειδή η υλοποίηση βασίζεται σε συνδεδεμένες λίστες (linked lists) για αποδοτικές εισαγωγές και αφαιρέσεις, ο Strand Sort δεν είναι in-place και απαιτεί $O(n)$ επιπλέον χώρο.

2.3.8 Tree Sort

Ο Tree Sort υλοποιεί την ταξινόμηση μέσω Δυαδικού Δέντρου Αναζήτησης (Binary Search Tree - BST), εισάγοντας ένα-ένα τα στοιχεία του πίνακα σε ένα BST και στη συνέχεια εκτελεί ενδοδιατακτική διάσχιση (in-order traversal), η οποία από τη φύση της παράγει τα στοιχεία σε αύξουσα σειρά (Cormen, 2009). Η μέση πολυπλοκότητα είναι $O(n \log n)$, αντίστοιχη με τις άλλες δομές αυτής της κατηγορίας.

Ωστόσο, ο Tree Sort παρουσιάζει κρίσιμη αδυναμία για ταξινομημένη ή αντίστροφα ταξινομημένη είσοδο, το BST εκφυλίζεται σε γραμμική αλυσίδα (degenerate tree) με βάθος $O(n)$, οδηγώντας σε χρόνο $O(n^2)$ και σε υπέρβαση ορίου αναδρομής (Recursion Limit).

2.3.9 Cartesian Tree Sort

Ο Cartesian Tree Sort βασίζεται στη δομή δεδομένων Cartesian Tree, η οποία εισήχθη αρχικά από τον Vuillemin (1980) και αναλύθηκε ως προς τις ιδιότητες προσαρμοστικότητάς της από τους Levcopoulos και Petersson (1989).

Πρόκειται για ένα δυαδικό δέντρο που συνδυάζει ταυτόχρονα δύο ιδιότητες: την ιδιότητα του Δυαδικού Δέντρου Αναζήτησης (BST) ως προς τις θέσεις των στοιχείων, και την ιδιότητα σωρού (heap) ως προς τις τιμές τους. Το δέντρο οικοδομείται σε $O(n)$ χρόνο μέσω αλγορίθμου βασισμένου σε στοίβα, και η ταξινόμηση επιτυγχάνεται στη συνέχεια μέσω min-heap (priority queue). Η ρίζα του δέντρου εισάγεται στον σωρό

και σε κάθε βήμα εξάγεται το ελάχιστο στοιχείο, ενώ τα παιδιά του εισάγονται αμέσως στη θέση του. Η διαδικασία αυτή παράγει τα στοιχεία σε αύξουσα σειρά.

Το αξιοσημείωτο χαρακτηριστικό αυτού του αλγορίθμου είναι η προσαρμοστικότητα του. Για ήδη ταξινομημένη είσοδο, το Cartesian Tree εκφυλίζεται σε γραμμικό μονοπάτι και η κατασκευή του ολοκληρώνεται σε $O(n)$, άρα και η συνολική ταξινόμηση γίνεται γραμμική. Στη χειρότερη και μέση περίπτωση η πολυπλοκότητα παραμένει $O(n \log n)$, χωρίς να υπάρχει ο κίνδυνος εκφυλισμού $O(n^2)$ που εμφανίζεται στον Tree Sort με απλό BST. Ωστόσο, απαιτεί $O(n)$ επιπλέον χώρο για την αποθήκευση του δέντρου, και η υλοποίησή του είναι αισθητά πολυπλοκότερη από τους περισσότερους αλγορίθμους της κατηγορίας.

2.3.10 Tournament Sort

Ο Tournament Sort αποτελεί βελτίωση του Selection Sort. Αντί να αναζητά το ελάχιστο στοιχείο με γραμμική σάρωση $O(n)$ σε κάθε βήμα, οργανώνει τα στοιχεία σε μια δομή δέντρου τουρνουά (tournament tree), ουσιαστικά ένα min-heap, από το οποίο η εξαγωγή του μικρότερου κοστίζει μόνο $O(\log n)$. Στη δομή αυτή, τα γειτονικά στοιχεία συγκρίνονται ανά ζεύγη. Το μικρότερο στοιχείο κάθε ζεύγους προωθείται στο επόμενο επίπεδο του δέντρου, μέχρι το ελάχιστο στοιχείο του συνόλου να καταλάβει τη ρίζα. Μετά την εξαγωγή του, η θέση του πληρώνεται από το επόμενο στοιχείο και το δέντρο επαναρυθμίζεται σε $O(\log n)$.

Η πιο σημαντική πρακτική εφαρμογή του Tournament Sort δεν βρίσκεται στην εσωτερική ταξινόμηση, αλλά στην εξωτερική ταξινόμηση (external sorting). Πιο συγκεκριμένα, η παραλλαγή του ως Replacement Selection χρησιμοποιείται για τη δημιουργία αρχικών ταξινομημένων τμημάτων (runs) σε αλγορίθμους που επεξεργάζονται δεδομένα μεγαλύτερα από τη διαθέσιμη RAM, ένα κλασικό πρόβλημα που περιγράφει αναλυτικά ο Knuth (1998). Για εσωτερική ταξινόμηση, ο Tournament Sort είναι ουσιαστικά ισοδύναμος με τον Heap Sort ως προς ασυμπτωτική πολυπλοκότητα, αλλά με υψηλότερο σταθερό κόστος λόγω της ρητής δενδρικής δομής.

2.3.11 Patience Sort

Ο Patience Sort εμπνέεται από το ομώνυμο παιχνίδι με τράπουλα (solitaire) όπου τα στοιχεία τοποθετούνται σε στοίβες (piles) ακολουθώντας έναν απλό κανόνα, κάθε νέα κάρτα τοποθετείται στην αριστερότερη στοίβα της οποίας η κορυφή είναι μεγαλύτερη ή ίση με την τρέχουσα τιμή (αν δεν υπάρχει τέτοια στοίβα, δημιουργείται νέα). Η εύρεση της κατάλληλης στοίβας γίνεται με δυαδική αναζήτηση (binary search) σε ξεχωριστή λίστα κορυφών (tops), που συντηρείται παράλληλα με τις στοίβες. Στην υλοποίηση της παρούσας εργασίας χρησιμοποιείται και έκδοση χωρίς την χρήση της `bisect_left`, αποφεύγοντας εξάρτηση από βιβλιοθήκες C. Ο αλγόριθμος αποδίδεται στον A.S.C. Ross και αναλύθηκε από τον C.L. Mallows (1962).

Το θεωρητικό αποτέλεσμα είναι ότι ο αριθμός των στοιβών που δημιουργούνται ισούται ακριβώς με το μήκος της Μεγαλύτερης Αύξουσας Υποακολουθίας (Longest

Increasing Subsequence - LIS) της εισόδου, συνδέοντας τον αλγόριθμο με ένα κλασικό πρόβλημα συνδυαστικής (Aldous & Diaconis, 1999). Στη φάση συγχώνευσης, οι k στοίβες συγχωνεύονται με k -way merge σε $O(n \log k)$ χρόνο. Η συνολική πολυπλοκότητα είναι $O(n \log n)$ στη μέση και χειρότερη περίπτωση, ενώ για σχεδόν ταξινομημένη είσοδο (λίγες στοίβες, μικρό k) προσεγγίζει το $O(n)$, καθιστώντας τον Patience Sort έναν από τους πιο προσαρμοστικούς (adaptive) αλγορίθμους της κατηγορίας.

2.4 Δίκτυα Ταξινόμησης

Τα δίκτυα ταξινόμησης (sorting networks) αποτελούν μια θεμελιωδώς διαφορετική κατηγορία από τους αλγορίθμους που εξετάστηκαν στις προηγούμενες ενότητες. Αντί για ένα ευέλικτο πρόγραμμα που αποφασίζει δυναμικά ποια στοιχεία θα συγκρίνει βάσει των δεδομένων εισόδου, ένα δίκτυο ταξινόμησης αποτελείται από ένα σταθερό, προκαθορισμένο σύνολο συγκριτών (comparators), καθένας από τους οποίους συγκρίνει και ανταλλάσσει ένα συγκεκριμένο ζεύγος θέσεων ανεξαρτήτως της εισόδου (Knuth, 1998).

Αυτή η ντετερμινιστική φύση τα καθιστά άμεσα υλοποιήσιμα σε υλικό (hardware) και ιδανικά για παραλληλοποίηση, καθώς πολλαπλοί συγκριτές που δεν μοιράζονται θέσεις μπορούν να εκτελεστούν ταυτόχρονα. Το βάθος (depth) του δικτύου, δηλαδή ο μέγιστος αριθμός σειριακών βημάτων, ορίζει τον απαιτούμενο χρόνο σε πλήρως παράλληλη εκτέλεση. Η ανάπτυξη βέλτιστων δικτύων ταξινόμησης αποτελεί ανοιχτό ερευνητικό πρόβλημα στη θεωρητική πληροφορική (Cormen, 2001).

Αλγόριθμος	Best	Random	Worst	Μνήμη	Σταθερότητα
Bitonic Sort	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	Όχι
Pairwise Network	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	Όχι

2.4.1 Bitonic Sort

Ο Bitonic Sort, σχεδιασμένος από τον Batcher (1968), ανήκει στην κατηγορία των δικτύων ταξινόμησης (sorting networks). Η λογική του βασίζεται στη δημιουργία μπιτονικών ακολουθιών (ακολουθίες που αυξάνονται και έπειτα μειώνονται), οι οποίες στη συνέχεια συγχωνεύονται αναδρομικά. Απαιτεί το μέγεθος του πίνακα να είναι δύναμη του 2, γι' αυτό και συχνά συμπληρώνεται με τεχνητές τιμές (padding) που αφαιρούνται στο τέλος. Αν και έχει χρονική πολυπλοκότητα $O(n \log^2 n)$, η ντετερμινιστική σειρά των συγκρίσεων του τον κάνει εξαιρετικά αποδοτικό σε παράλληλα συστήματα υλικού. Ειδικότερα, σε επεξεργαστές γραφικών (GPU), το σταθερό και προκαθορισμένο μοτίβο συγκρίσεων επιτρέπει σε εκατοντάδες νήματα να εκτελούν ταυτόχρονα πράξεις σύγκρισης και ανταλλαγής σε διαφορετικά στοιχεία,

χωρίς απόκλιση διακλαδώσεων (branch divergence), αξιοποιώντας πλήρως το μοντέλο SIMD εκτέλεσης που χαρακτηρίζει αυτές τις αρχιτεκτονικές (Satish, 2009).

2.4.2 Pairwise Sorting Network

Το Pairwise Sorting Network δημοσιεύθηκε από τον Ian Parberry το 1992 και ανήκει στην ίδια κατηγορία δικτύων ταξινόμησης με τον Bitonic Sort (Parberry, 1992). Μοιράζεται την ίδια χρονική πολυπλοκότητα $O(n \log^2 n)$, απαιτώντας $n(\log n)(\log n - 1)/4 + n - 1$ συγκριτές και βάθος $(\log n)(\log n + 1)/2$. Η λειτουργία του βασίζεται στο zero-one principle και ταξινομεί διαδοχικά ζεύγη bits, τα οποία στη συνέχεια οργανώνει σε λεξικογραφική σειρά μέσω αναδρομικής ταξινόμησης περιττών και άρτιων bits ξεχωριστά.

Η βασική δομική διαφορά από τον Bitonic Sort έγκειται στη σειρά των πράξεων. Ενώ ο Bitonic Sort δημιουργεί εναλλάξ αύξουσες και φθίνουσες υποακολουθίες και τις συγχωνεύει σταδιακά, το Pairwise εκτελεί όλες τις διαιρέσεις πρώτα και στη συνέχεια όλες τις συγχωνεύσεις. Όπως και ο Bitonic Sort, το προκαθορισμένο και ντετερμινιστικό μοτίβο συγκρίσεων του το καθιστά εξαιρετικά κατάλληλο για παράλληλα συστήματα, FPGA και GPU (Parberry, 1992).

2.5 Υβριδικοί & Σύνθετοι

Οι αλγόριθμοι της κατηγορίας αυτής δεν ανήκουν σε μία μόνο αλγοριθμική οικογένεια. Συνδυάζουν τεχνικές από δύο ή περισσότερους αλγορίθμους, με στόχο να αξιοποιούν τα πλεονεκτήματα του καθενός σε εκείνα τα τμήματα του προβλήματος όπου αποδίδουν βέλτιστα. Το αποτέλεσμα είναι αλγόριθμοι που επιτυγχάνουν θεωρητικά βέλτιστη ασυμπτωτική πολυπλοκότητα $O(n \log n)$, ενώ ταυτόχρονα αντιμετωπίζουν ρεαλιστικές συνθήκες δεδομένων όπως μερικώς ταξινομημένες ακολουθίες ή δεδομένα με επαναλαμβανόμενες τιμές. Η κατηγορία περιλαμβάνει τέσσερις αλγορίθμους οι οποίοι είναι οι Tim Sort, Intro Sort, Block Sort και Merge-Insertion Sort.

Αλγόριθμος	Best	Random	Worst	Μνήμη	Σταθερότητα
Tim Sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Ναι
Intro Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	Όχι
Block Sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Ναι
Merge-Insertion	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Όχι

2.5.1 Tim Sort

Ο Tim Sort σχεδιάστηκε από τον Tim Peters το 2002 και εντάχθηκε στη γλώσσα Python από την έκδοση 2.3, αντικαθιστώντας τον προηγούμενο αλγόριθμο

ταξινόμησής της. Η σχεδίαση βασίστηκε στην παρατήρηση ότι τα πραγματικά δεδομένα σπάνια είναι τελείως τυχαία και ότι συνήθως περιέχουν ήδη ταξινομημένα τμήματα που μπορούν να αξιοποιηθούν. Λόγω της υψηλής πρακτικής του απόδοσης, ο αλγόριθμος υιοθετήθηκε ως η βασική μέθοδος ταξινόμησης σε περιβάλλοντα όπως η Java SE 7 (για αντικείμενα) και το Android (Peters, 2002).

Πρόκειται για υβριδικό αλγόριθμο που συνδυάζει Merge Sort και Binary Insertion Sort. Σαρώνει τον πίνακα εισόδου και εντοπίζει μονοτονικά τμήματα (natural runs), ενώ φθίνοντα runs αντιστρέφει αμέσως. Αν ένα run είναι μικρότερο από ένα κατώφλι (minrun, συνήθως 32-64), επεκτείνεται με Binary Insertion Sort.

Στη συνέχεια, τα runs συγχωνεύονται μέσω μιας στοίβας που διατηρεί δύο αμετάβλητες ισορροπίας ($|Z| > |Y| + |X|$ και $|Y| > |X|$), εξασφαλίζοντας ισορροπημένες και αποδοτικές συγχωνεύσεις. Κατά τη συγχώνευση χρησιμοποιείται η τεχνική *galloning*, όταν ένα run υπερτερεί συνεχόμενα, αντικαθίστανται οι στοιχείο-προς-στοιχείο συγκρίσεις με εκθετική αναζήτηση, μειώνοντας δραστικά τον αριθμό των συγκρίσεων (Auger, 2018).

Η χρονική πολυπλοκότητα είναι $O(n)$ στη βέλτιστη περίπτωση και $O(n \log n)$ στη μέση και χειρότερη. Απαιτεί $O(n)$ βοηθητικό χώρο και είναι σταθερός (stable) αλγόριθμος. Είναι από τους πιο adaptive αλγορίθμους της βιβλιογραφίας, καθώς εκμεταλλεύεται πλήρως την υπάρχουσα τάξη της εισόδου.

Στην παρούσα εργασία, εκτός από τον Tim Sort (Built-in), έχω κάνει και μια ακόμα απλοποιημένη παραλλαγή (Simplified Tim Sort) η οποία χρησιμοποιεί σταθερό $\text{minrun} = 32$ χωρίς ανίχνευση φυσικών runs και παραλείπει την τεχνική *galloning*, αποδίδοντας ορθά αποτελέσματα με μειωμένη ωστόσο προσαρμοστικότητα σε σχέση με τον πλήρη Tim Sort.

2.5.2 Intro Sort

Ο Intro Sort σχεδιάστηκε από τον David Musser το 1997 με στόχο να παρέχει τόσο γρήγορη μέση απόδοση όσο και εγγυημένη βέλτιστη χειρότερη περίπτωση, κάτι που κανένας από τους τρεις αλγορίθμους που συνδυάζει δεν μπορεί να επιτύχει μόνος του. Αναπτύχθηκε αρχικά για τις ανάγκες της C++ Standard Library και αποτελεί σήμερα τον αλγόριθμο που κρύβεται πίσω από τη `std::sort` της C++ (Musser, 1997).

Ο αλγόριθμος ξεκινά με Quick Sort, χρησιμοποιώντας median-of-three επιλογή pivot και παράλληλα παρακολουθεί το βάθος αναδρομής. Αν αυτό υπερβεί το όριο $\lfloor 2 \log_2 n \rfloor$, μεταβαίνει αυτόματα σε Heap Sort για το τρέχον υποπρόβλημα, εξαλείφοντας πρακτικά τη χειρότερη περίπτωση $O(n^2)$ του Quick Sort. Επιπλέον, όταν το μέγεθος ενός υποπίνακα πέσει κάτω από ένα μικρό κατώφλι (συνήθως 16 στοιχεία), ο αλγόριθμος εναλλάσσεται σε Insertion Sort, ο οποίος αποδίδει βέλτιστα σε μικρές και σχεδόν ταξινομημένες ακολουθίες (Musser, 1997).

Η χρονική πολυπλοκότητα είναι $O(n \log n)$ τόσο στη μέση όσο και στη χειρότερη περίπτωση, με χώρο $O(\log n)$ για τη στοίβα αναδρομής. Είναι in-place αλλά μη

σταθερός (not stable) αλγόριθμος. Στην πράξη, η απόδοσή του προσεγγίζει εκείνη του Quick Sort στις τυπικές εισόδους, αποφεύγοντας παράλληλα την υποβάθμιση σε $O(n^2)$ που εμφανίζει ο Quick Sort σε ειδικές περιπτώσεις (π.χ. ταξινομημένος πίνακας με απλοϊκή επιλογή pivot).

2.5.3 Block Sort

Ο Block Sort (γνωστός και ως Block Merge Sort ή WikiSort) βασίζεται στη θεωρητική εργασία των Pok-Son Kim και Arne Kutzner (2008), η οποία περιέγραψε έναν πρακτικό αλγόριθμο για σταθερή in-place συγχώνευση βασισμένη σε λόγο μεγεθών (ratio-based merging). Ο αλγόριθμος σχεδιάστηκε για να επιλύσει έναν γνωστό περιορισμό των παραδοσιακών in-place προσεγγίσεων. Ειδικότερα, ο Merge Sort είναι σταθερός αλλά απαιτεί βοηθητικό χώρο $O(n)$, και οι in-place παραλλαγές είτε στερούνταν πρακτικής αξίας, είτε περιελάμβαναν υψηλές σταθερές που αύξαναν τον χρόνο εκτέλεσης (Kim & Kutzner, 2008).

Ο αλγόριθμος ξεκινά με Insertion Sort σε μικρές ομάδες στοιχείων. Στη συνέχεια εφαρμόζει bottom-up Merge Sort, αλλά αντί για εξωτερικούς βοηθητικούς πίνακες χρησιμοποιεί εσωτερικούς buffers μεγέθους $\sqrt{n}$ που εξάγει από τον ίδιο τον πίνακα. Τα A και B τμήματα χωρίζονται σε blocks μεγέθους $\sqrt{n}$, τα blocks ταξινομούνται, συγχωνεύονται με block swaps, και τελικά οι buffers επαναδιανέμονται στις σωστές θέσεις τους μέσω Insertion Sort.

Η πολυπλοκότητα είναι $O(n \log n)$ στη μέση και χειρότερη περίπτωση και $O(n)$ στη βέλτιστη. Ο Block Sort είναι σταθερός (stable) και, κρίσιμα, in-place με $O(1)$ βοηθητικό χώρο, αυτό σημαίνει ότι είναι ο μόνος αλγόριθμος στην κατηγορία αυτή που συνδυάζει και τις δύο ιδιότητες. Η υλοποίηση της παρούσας εργασίας αποτελεί απλοποιημένη προσέγγιση που χωρίζει τον πίνακα σε blocks μεγέθους $\sqrt{n}$, τα ταξινομεί με Insertion Sort και τα συγχωνεύει με κλασικό bottom-up merge, παραλείποντας τον πλήρη μηχανισμό εσωτερικών buffers και block swaps του Block Sort.

2.5.4 Merge-Insertion Sort

Ο αλγόριθμος δημοσιεύτηκε το 1959 από τους Ford και Johnson (1959), εξετάζοντας το θεωρητικό πρόβλημα του ελάχιστου απαιτούμενου αριθμού συγκρίσεων για την πλήρη διάταξη n στοιχείων (Ford & Johnson, 1959). Αρχικά ονομάστηκε «tournament sort» και μετονομάστηκε σε Merge-Insertion Sort από τον Donald Knuth, ο οποίος τον ανέλυσε εκτενώς στο The Art of Computer Programming (Knuth, 1998). Ανεξάρτητα ανακαλύφθηκε και από τους Trybula και Czen Ping την ίδια χρονιά.

Ο αλγόριθμος λειτουργεί σε τρία βήματα, χωρίζει τα στοιχεία σε ζεύγη και ταξινομεί κάθε ζεύγος ($n/2$ συγκρίσεις), ταξινομεί αναδρομικά τα μεγαλύτερα στοιχεία κάθε ζεύγους, και στη συνέχεια εισάγει τα υπόλοιπα στοιχεία με δυαδική αναζήτηση. Στη θεωρητική μορφή του αλγορίθμου, η σειρά εισαγωγής ακολουθεί τους αριθμούς Jacobsthal ώστε κάθε εισαγωγή να εκμεταλλεύεται το μέγιστο δυνατό μέγεθος της

ήδη ταξινομημένης ακολουθίας. Η υλοποίηση της παρούσας εργασίας χρησιμοποιεί απλή σειριακή σειρά εισαγωγής, απλοποιώντας τον αλγόριθμο εις βάρος του θεωρητικά βέλτιστου αριθμού συγκρίσεων.

Το θεωρητικό του ενδιαφέρον έγκειται στο ότι ελαχιστοποιεί τον αριθμό συγκρίσεων στη χειρότερη περίπτωση. Πιο συγκεκριμένα, για $n \leq 22$ στοιχεία παράγει το θεωρητικά βέλτιστο αριθμό συγκρίσεων, ενώ για $n \leq 46$ παραμένει ο αλγόριθμος με τις λιγότερες γνωστές συγκρίσεις. Για 20 χρόνια κατείχε τον τίτλο αυτό για όλα τα μεγέθη εισόδου, μέχρι το 1979 που ο Glenn Manacher δημοσίευσε βελτίωσή του για μεγάλα n . Η ασυμπτωτική πολυπλοκότητα χρόνου είναι $O(n \log n)$, ωστόσο η αυξημένη πολυπλοκότητα της υλοποίησης και τα υψηλά σταθερά κόστη (constant factors) τον καθιστούν πρακτικά λιγότερο αποδοτικό σε σύγκριση με τους υπόλοιπους αλγορίθμους της κατηγορίας.

2.6 Αλγόριθμοι Χωρίς Σύγκριση

Όλοι οι αλγόριθμοι που εξετάστηκαν ως εδώ ανήκουν στην κατηγορία των comparison-based που εξάγουν πληροφορία για τη σχετική τάξη των στοιχείων αποκλειστικά μέσω συγκρίσεων ζευγών. Η θεωρία decision tree αποδεικνύει ότι κανένας τέτοιος αλγόριθμος δεν μπορεί να τερματίσει σε λιγότερες από $\Omega(n \log n)$ συγκρίσεις στη χειρότερη περίπτωση, αφού πρέπει να διακρίνει μεταξύ $n!$ πιθανών διατάξεων εισόδου (Cormen, 2009, §8.1).

Οι non-comparison αλγόριθμοι παρακάμπτουν αυτό το κατώφλι με διαφορετικό τρόπο. Αντί να συγκρίνουν στοιχεία μεταξύ τους, εκμεταλλεύονται γνωστές ιδιότητες των δεδομένων, όπως για παράδειγμα ότι τα κλειδιά είναι ακέραιοι περιορισμένου εύρους ή ακολουθούν γνωστή κατανομή. Έτσι επιτυγχάνουν γραμμικό χρόνο $O(n)$ ή $O(n+k)$, έναντι όμως αυστηρών προϋποθέσεων για τη φύση της εισόδου.

Αλγόριθμος	Best	Random	Worst	Μνήμη	Σταθερότητα
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Ναι
Radix Sort	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$	$O(n + k)$	Ναι
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Ναι
Pigeonhole Sort	$O(n + N)$	$O(n + N)$	$O(n + N)$	$O(n + N)$	Ναι
Flash Sort	$O(n)$	$O(n)$	$O(n^2)$	$O(m)$	Όχι

2.6.1 Counting Sort

Ο Counting Sort αναπτύχθηκε από τον Harold H. Seward το 1954 στο MIT, ως μέρος της διδακτορικής του διατριβής, και αποτελεί έναν από τους πρώτους αλγορίθμους που εξάγει πληροφορία από τη φύση των κλειδιών αντί να τα συγκρίνει, παρακάμπτοντας έτσι το θεωρητικό κάτω φράγμα του $O(n \log n)$ για πρώτη φορά (Seward, 1954).

Ο αλγόριθμος λειτουργεί σε τρία βήματα, μετράει πόσες φορές εμφανίζεται κάθε τιμή (counting array), υπολογίζει τα prefix sums για να καθορίσει τη θέση κάθε στοιχείου στον τελικό πίνακα, και στη συνέχεια κατασκευάζει απευθείας τον ταξινομημένο πίνακα. Η χρονική πολυπλοκότητα είναι $O(n+k)$, όπου k το εύρος τιμών, και ο αλγόριθμος είναι σταθερός (Cormen, 2009), ιδιότητα που στην παρούσα υλοποίηση εξασφαλίζεται με την αντίστροφη διάσχιση του πίνακα κατά τη φάση τοποθέτησης.

Το βασικό μειονέκτημά του αφορά ακριβώς αυτό το k , όπου αν το εύρος τιμών είναι δυσανάλογα μεγάλο σε σχέση με το n , η κατανάλωση μνήμης $O(k)$ γίνεται απαγορευτική. Γι' αυτό στην πράξη εφαρμόζεται μόνο όταν $k = O(n)$, δηλαδή όταν τα δεδομένα είναι ακέραιοι με πυκνή και περιορισμένη κατανομή (Cormen, 2009). Επιπλέον, η υλοποίηση επεκτάθηκε για την υποστήριξη αρνητικών ακεραίων, χρησιμοποιώντας την ελάχιστη τιμή της εισόδου (`min_val`) ως μετατόπιση (`offset`) στους δείκτες του πίνακα συχνοτήτων.

2.6.2 Radix Sort

Η βασική αρχή του Radix Sort ανάγεται στο 1887, βασισμένη στην επεξεργασία δεδομένων απογραφής μέσω μηχανών διάτρητων καρτών από τον Herman Hollerith (Knuth, 1998). Η πρώτη υλοποίηση ως αλγόριθμος υπολογιστή έγινε από τον Seward το 1954 (Seward, 1954).

Αντί να συγκρίνει ολόκληρους αριθμούς, ο Radix Sort τους ταξινομεί ψηφίο-ψηφίο, από το λιγότερο σημαντικό (LSD) προς το πιο σημαντικό (MSD). Σε κάθε βήμα εφαρμόζει έναν σταθερό βοηθητικό αλγόριθμο (συνήθως τον Counting Sort) οπότε η σταθερότητα του αποτελέσματος εξαρτάται άμεσα από αυτόν. Η συνολική χρονική πολυπλοκότητα είναι $O(d(n+k))$, όπου d ο αριθμός ψηφίων και k η βάση (Cormen, 2009).

Για ακέραιους με σταθερό αριθμό ψηφίων, αυτό ισοδυναμεί πρακτικά με $O(n)$. Ο αλγόριθμος δεν είναι in-place γιατί απαιτεί βοηθητική μνήμη $O(n+k)$, αλλά χρησιμοποιείται ευρέως σε βάσεις δεδομένων, GPU sorting και δίκτυα, ακριβώς επειδή κλιμακώνεται καλά σε μεγάλα σύνολα ακεραίων (Cormen, 2009).

2.6.3 Bucket Sort

Ο Bucket Sort (ή Bin Sort) γενικεύει τη λογική του Pigeonhole Sort, αντί για μία θέση ανά τιμή, ομαδοποιεί τα στοιχεία σε k κάδους (buckets), καθένας αντιστοιχεί σε ένα υποδιάστημα τιμών. Κάθε κάδος ταξινομείται στη συνέχεια ανεξάρτητα (συνήθως με Insertion Sort) και τα αποτελέσματα συνενώνονται (Isaac & Singleton, 1956).

Η απόδοσή του εξαρτάται κρίσιμα από την κατανομή των δεδομένων. Υπό ομοιόμορφη κατανομή και με $k = O(n)$ κάδους, η μέση πολυπλοκότητα πέφτει σε $O(n)$. Αν όμως τα δεδομένα συσσωρευτούν σε έναν κάδο, ο αλγόριθμος εκπίπτει σε $O(n^2)$, άρα η χειρότερη περίπτωση ορίζεται από την κατανομή και όχι από τον αριθμό στοιχείων (Cormen, 2009, §8.4).

Δεν είναι in-place λόγω της βοηθητικής μνήμης $O(n+k)$, αλλά είναι stable εφόσον ο αλγόριθμος ταξινόμησης των κάδων είναι stable. Έχει ευρεία εφαρμογή σε floating-point δεδομένα και σε προβλήματα όπου η κατανομή είναι εκ των προτέρων γνωστή.

2.6.4 Pigeonhole Sort

Ο Pigeonhole Sort πήρε το όνομά του από τη μαθηματική αρχή του περιστερεώνα (Pigeonhole Principle). Κάθε πιθανή τιμή αντιστοιχεί σε μια ξεχωριστή τρύπα, και κάθε στοιχείο τοποθετείται απευθείας στη θέση που του αναλογεί. Είναι η πιο άμεση υλοποίηση της ιδέας της distribution-based ταξινόμησης.

Εννοιολογικά μοιάζει πολύ με τον Counting Sort, αλλά διαφέρει σε ένα σημείο, ο Counting Sort μετακινεί κάθε στοιχείο μία φορά (υπολογίζει θέσεις και χτίζει τον πίνακα εξόδου), ενώ ο Pigeonhole Sort μετακινεί κάθε στοιχείο δύο φορές, μία στον κάδο και μία στη θέση εξόδου. Η χρονική πολυπλοκότητα είναι $O(n+N)$, όπου N το εύρος τιμών (Knuth, 1998).

Η προϋπόθεση εφαρμογής του είναι σαφής, καθώς το n και το N πρέπει να είναι της ίδιας τάξης μεγέθους, αλλιώς η κατανάλωση μνήμης γίνεται μη αποδεκτή. Στην πράξη ταιριάζει καλά για ακέραια δεδομένα με πυκνή κατανομή, όπως βαθμοί, αξιολογήσεις ή κωδικοί περιορισμένου εύρους.

2.6.5 Flash Sort

Ο Flash Sort δημοσιεύτηκε το 1998 από τον Γερμανό φυσικό Karl-Dietrich Neubert στο Dr. Dobbs's Journal (Neubert, 1998). Ο κύριος στόχος ήταν να γίνει το μεγαλύτερο μέρος της ταξινόμησης μέσω in-place μετακινήσεων, χωρίς δαπανηρές αντιγραφές.

Ο αλγόριθμος δουλεύει σε τρία στάδια. Πρώτα εκτελεί κατηγοριοποίηση (classification), χωρίζει το εύρος τιμών σε $m \approx 0.45n$ κλάσεις και αντιστοιχεί κάθε στοιχείο στην κλάση του. Στη συνέχεια κάνει επί τόπου μετάθεση (in-situ permutation), μετακινώντας κάθε στοιχείο κοντά στη σωστή θέση μέσω κυκλικών εναλλαγών. Τέλος, απλή ταξινόμηση παρεμβολής (straight insertion sort) τακτοποιεί τα στοιχεία εντός κάθε κλάσης. Για ομοιόμορφα κατανεμημένα δεδομένα, ο χρόνος είναι $O(n)$ και η βοηθητική μνήμη μόλις $O(m)$, κάτι που τον διακρίνει ουσιαστικά από τον Bucket Sort.

Ο βασικός του περιορισμός είναι η ευαισθησία στην κατανομή, καθώς σε μη ομοιόμορφα δεδομένα η απόδοσή του υποβαθμίζεται σημαντικά, φτάνοντας $O(n^2)$ στη χειρότερη περίπτωση (Neubert, 1998). Επίσης δεν είναι stable. Παρά τις εντυπωσιακές επιδόσεις σε ειδικές συνθήκες, η χρήση του στην πράξη παραμένει περιορισμένη ακριβώς λόγω αυτής της εξάρτησης από την κατανομή.

ΚΕΦΑΛΑΙΟ 3 Μεθοδολογία

3.1 Γενική Περιγραφή

Το παρόν κεφάλαιο περιγράφει αναλυτικά τη μεθοδολογία που ακολουθήθηκε για τη διεξαγωγή των εμπειρικών μετρήσεων της εργασίας. Παρουσιάζεται το υλικό και λογισμικό περιβάλλον εκτέλεσης, οι αλγόριθμοι που εξετάστηκαν, ο σχεδιασμός των δεδομένων εισόδου, καθώς και τα εργαλεία και τεχνικές που χρησιμοποιήθηκαν για τη μέτρηση χρόνου εκτέλεσης και κατανάλωσης μνήμης. Ιδιαίτερη έμφαση δίνεται στα μέτρα αξιοπιστίας που υιοθετήθηκαν ώστε τα αποτελέσματα να είναι επαναλήψιμα και ακριβή.

3.2 Περιβάλλον Εκτέλεσης

Το σύνολο των πειραμάτων εκτελέστηκε σε σταθερό υπολογιστικό σύστημα με τα ακόλουθα χαρακτηριστικά:

Επεξεργαστής: Intel Core i5-10400 (6 πυρήνες / 12 νήματα, 2.90-4.30 GHz)

Μνήμη RAM: 32 GB DDR4-3200MHz

Λειτουργικό Σύστημα: Windows 11 με χρήση Subsystem for Linux 2 (WSL2)

Γλώσσα Προγραμματισμού: Python 3.13

Το περιβάλλον εκτέλεσης επιλέχθηκε να είναι WSL2 (Windows Subsystem for Linux 2), καθώς ο μηχανισμός διαχείρισης χρονικού ορίου που χρησιμοποιήθηκε βασίζεται στο σήμα SIGALRM του POSIX, το οποίο δεν υποστηρίζεται από τον εγγενή διερμηνευτή Python σε περιβάλλον Windows.

3.3 Υλοποίηση Αλγορίθμων

3.3.1 Αμιγείς Υλοποιήσεις σε Python

Το σύνολο των αλγορίθμων υλοποιήθηκε από μηδενική βάση στη γλώσσα προγραμματισμού Python 3, χωρίς τη χρήση εξωτερικών βιβλιοθηκών για την ίδια τη λογική ταξινόμησης. Η επιλογή αυτή εξασφαλίζει ότι οι χρόνοι εκτέλεσης αντικατοπτρίζουν πιστά τη θεωρητική πολυπλοκότητα κάθε αλγορίθμου, χωρίς να επηρεάζονται από βελτιστοποιήσεις χαμηλού επιπέδου. Συνολικά υλοποιήθηκαν 30 αλγόριθμοι σε καθαρή Python, οι οποίοι κατηγοριοποιούνται ως εξής:

- Αλγόριθμοι Σύγκρισης (Comparison-Based): Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, Shell Sort, Comb Sort, Tim Sort (Simplified), Intro Sort, Tree Sort, Smooth Sort, Strand Sort, Patience Sort, Merge-Insertion Sort (Ford-Johnson), Gnome Sort, Tournament Sort, Cocktail Shaker Sort, Cycle Sort, Pancake Sort, Cartesian Tree Sort, Block Sort, Pairwise Sorting Network, Bitonic Sort, Stooge Sort.

- Αλγόριθμοι Χωρίς Σύγκριση (Non-Comparison-Based): Counting Sort, Radix Sort, Bucket Sort, Pigeonhole Sort, Flash Sort.

Ιδιαίτερη προσοχή δόθηκε σε αλγορίθμους που εσωτερικά απαιτούν δομές δεδομένων όπως σωρός (heap) ή ουρά προτεραιότητας (priority queue). Για αυτές τις περιπτώσεις (Tournament Sort, Patience Sort, Cartesian Tree Sort) υλοποιήθηκαν βοηθητικές συναρτήσεις γραμμένες εξ ολοκλήρου σε Python για min-heap (py_heappush, py_heappop) καθώς και δυαδική αναζήτηση (py_bisect_left, py_bisect_right), ώστε να αποφευχθεί η έμμεση χρήση κώδικα C.

Σημειώνεται ότι ο Stooge Sort υλοποιήθηκε πλήρως, αλλά εξαιρέθηκε από τα πειράματα λόγω της εξαιρετικά αργής πολυπλοκότητας του $O(n^{2.71})$, η οποία προκαλεί timeout ακόμα και για μικρά N .

3.3.2 Βελτιστοποιημένες Υλοποιήσεις (Alt)

Για επιλεγμένο σύνολο αλγορίθμων δημιουργήθηκε παράλληλα μία βελτιστοποιημένη έκδοση (επισημαινόμενη ως «Alt» στο σύνολο δεδομένων), η οποία εκμεταλλεύεται τις ενσωματωμένες βιβλιοθήκες της Python, heapq (για heap operations), bisect (για δυαδική αναζήτηση) και την ενσωματωμένη sorted() / list.sort(). Αυτές οι εκδόσεις αφορούν τους αλγορίθμους με την ένδειξη “Alt”, Block Sort (Alt), Bucket Sort (Alt), Tournament Sort (Alt), Cartesian Tree Sort (Alt), Patience Sort (Alt), Merge-Insertion Sort (Alt) και Tim Sort (Built-in) (Python Software Foundation, 2024).

Η σύγκριση μεταξύ των βασικών υλοποιήσεων (αμιγώς σε Python) και των βελτιστοποιημένων (Alt) εκδόσεων στοχεύει στην ποσοτική τεκμηρίωση του overhead που εισάγει η ερμηνευτική φύση της Python σε σχέση με εγγενείς C-υλοποιήσεις, ένα φαινόμενο που εξηγεί σημαντικό μέρος των αποκλίσεων μεταξύ θεωρητικής και εμπειρικής πολυπλοκότητας. (Beazley, 2009)

3.4 Σχεδιασμός Δεδομένων Εισόδου

3.4.1 Μεγέθη Πίνακα

Κάθε αλγόριθμος αξιολογήθηκε για τέσσερα διαφορετικά μεγέθη εισόδου και συγκεκριμένα για 1.000, 10.000, 100.000, 1.000.000 στοιχεία.

Η επιλογή των μεγεθών αυτών παρέχει κάλυψη τριών τάξεων μεγέθους, επιτρέποντας την παρατήρηση της ασυμπτωτικής συμπεριφοράς κάθε αλγορίθμου. Η κλιμάκωση κατά παράγοντα 10 ευνοεί την οπτική ανάδειξη των θεωρητικών πολυπλοκοτήτων $O(n \log n)$ και $O(n^2)$ σε λογαριθμικά διαγράμματα.

Για την αξιολόγηση και τη συγκριτική ανάλυση των αλγορίθμων ταξινόμησης, κρίθηκε απαραίτητη η δημιουργία συνόλων δεδομένων διαφορετικών μεγεθών. Προς τον σκοπό αυτό, αναπτύχθηκε ένα αυτοματοποιημένο σενάριο εκτέλεσης (script) στη

γλώσσα προγραμματισμού Python, το οποίο παράγει αρχεία μορφής CSV (Comma-Separated Values) περιέχοντας τυχαίους ακέραιους αριθμούς.

Η παραγωγή των τυχαίων τιμών πραγματοποιήθηκε μέσω της ενσωματωμένης βιβλιοθήκης random της Python, η οποία εξασφαλίζει την ομοιόμορφη κατανομή (uniform distribution) των παραγόμενων ακεραίων. (Python Software Foundation, 2024)

Ένα βασικό χαρακτηριστικό της πειραματικής διάταξης είναι ότι το μέγιστο όριο των παραγόμενων τιμών ορίστηκε να ταυτίζεται αυστηρά με το πλήθος των στοιχείων του εκάστοτε συνόλου ($k = N$). Για παράδειγμα, στο σύνολο των 10.000 στοιχείων, οι παραγόμενες τιμές κυμαίνονται στο διάστημα $[0, 10.000]$. Η επιλογή $k = N$ διατηρεί σταθερή την πυκνότητα σε όλα τα μεγέθη εισόδου. Αυτό ευνοεί αλγορίθμους που βασίζονται σε κατανομή (distribution-based), όπως ο Counting Sort με χωρική πολυπλοκότητα $O(N+k)$, και επιτρέπει τη συγκρισιμότητα μεταξύ μεγεθών (Cormen, 2009).

3.4.2 Κατηγορίες Εισόδου

Για κάθε μέγεθος N , ο πίνακας εισόδου κατασκευάστηκε σε τρεις διακριτές κατηγορίες, οι οποίες αντιστοιχούν στα κλασικά σενάρια ανάλυσης αλγορίθμων:

-Βέλτιστη Περίπτωση (Best Case): Ο πίνακας είναι ήδη ταξινομημένος σε αύξουσα σειρά.

-Τυχαία Περίπτωση (Random Case): Ο πίνακας περιέχει ακέραιους αριθμούς τυχαίας κατανομής. Αντιπροσωπεύει τη μέση περίπτωση (average case) και είναι το πιο ρεαλιστικό σενάριο χρήσης.

-Χειρότερη Περίπτωση (Worst Case): Ο πίνακας είναι ταξινομημένος σε φθίνουσα σειρά (αντίστροφα).

3.5 Εργαλεία Μέτρησης

3.5.1 Μέτρηση Χρόνου Εκτέλεσης

Για τη μέτρηση του χρόνου εκτέλεσης χρησιμοποιήθηκε η συνάρτηση `time.perf_counter()` της standard βιβλιοθήκης Python. Πρόκειται για το ρολόι υψηλότερης ανάλυσης που διαθέτει η Python, κατάλληλο για μετρήσεις σύντομης διάρκειας, καθώς δεν επηρεάζεται από αλλαγές ρολογιού συστήματος και προσφέρει υποδεύτερη ακρίβεια.

Ο χρόνος μετριέται αποκλειστικά γύρω από την κλήση της συνάρτησης ταξινόμησης, αφού προηγουμένως γίνει αντιγραφή (`copy()`) του αρχικού πίνακα εισόδου.

Η αντιγραφή εξασφαλίζει ότι κάθε εκτέλεση λαμβάνει τον ίδιο αδιαφοροποίητο πίνακα ως είσοδο, ανεξαρτήτως της ενδεχόμενης καταστροφικής τροποποίησης από προηγούμενη εκτέλεση. (Python Software Foundation, 2024)

3.5.2 Μέτρηση Κατανάλωσης Μνήμης

Η κατανάλωση μνήμης μετρήθηκε με τη βιβλιοθήκη tracemalloc, η οποία παρέχει μηχανισμό παρακολούθησης δεσμεύσεων μνήμης σε επίπεδο Python objects. Καταγράφεται η κορυφαία κατανάλωση (peak memory) κατά τη διάρκεια εκτέλεσης κάθε αλγορίθμου, δηλαδή η μέγιστη ποσότητα μνήμης που δεσμεύτηκε ταυτόχρονα σε οποιοδήποτε χρονικό σημείο της εκτέλεσης. Η τιμή αυτή αντικατοπτρίζει ακριβώς τον επιπλέον χώρο (auxiliary space) που απαιτεί ο αλγόριθμος πέρα από την εισαγωγική ακολουθία, και αναφέρεται σε kilobytes (KB).

Σημειώνεται ότι το tracemalloc εκκινείται ακριβώς πριν την κλήση ταξινόμησης και διακόπτεται αμέσως μετά, ώστε να αποφευχθεί ο συνυπολογισμός μνήμης που δεσμεύεται από το ίδιο το περιβάλλον εκτέλεσης. (Python Software Foundation, 2024)

3.6 Μέτρα Αξιοπιστίας Μετρήσεων

3.6.1 Διαχείριση Timeout

Η εκτέλεση αλγορίθμων για μεγάλα N ενδέχεται να διαρκέσει απρόβλεπτα μεγάλο χρονικό διάστημα. Για την αντιμετώπιση αυτού του ζητήματος υλοποιήθηκε μηχανισμός αναγκαστικής διακοπής (timeout) βάσει σήματος λειτουργικού συστήματος. Συγκεκριμένα, χρησιμοποιείται το σήμα SIGALRM του Linux μέσω της βιβλιοθήκης signal, το οποίο αποστέλλεται στη διεργασία μετά από προκαθορισμένο χρονικό όριο.

Ορίστηκε ανώτατο χρονικό όριο εκτέλεσης 180 δευτερολέπτων ανά αλγόριθμο ανά συνδυασμό (N , Input Case). Σε περίπτωση υπέρβασης του ορίου, η εκτέλεση διακόπτεται αμέσως και η μέτρηση καταγράφεται στο σύνολο δεδομένων ως «Time Limit», χωρίς να αποθηκεύεται τιμή χρόνου ή μνήμης.

3.6.2 Διαχείριση Ορίου Αναδρομής

Αρκετοί αλγόριθμοι (Merge Sort, Quick Sort, Tree Sort, Bitonic Sort, Cartesian Tree Sort κ.ά.) βασίζονται σε αναδρομική υλοποίηση. Η Python διαθέτει εξ ορισμού όριο βάθους αναδρομικής στοίβας (default recursion limit: 1.000 κλήσεις). Για τις ανάγκες του πειράματος, το όριο αυτό ανυψώθηκε στις 20.000 αναδρομικές κλήσεις μέσω της εντολής `sys.setrecursionlimit(20000)`.

Παρά τη ρύθμιση αυτή, ορισμένοι αλγόριθμοι, ιδίως ο Tree Sort σε ταξινομημένη είσοδο (Best/Worst Case), εξαντλούν και αυτό το ανυψωμένο όριο λόγω της δημιουργίας εκφυλισμένου δέντρου (degenerate BST) με βάθος $O(N)$. Τέτοιες εκτελέσεις καταγράφονται στο σύνολο δεδομένων ως «Recursion Limit».

3.6.3 Αντιμετώπιση Θερμικής Επιβράδυνσης

Κατά τη διεξαγωγή παρατεταμένων πειραμάτων, οι σύγχρονοι επεξεργαστές ενδέχεται να μειώσουν αυτόματα τη συχνότητα λειτουργίας τους (thermal throttling) λόγω υπερθέρμανσης, επηρεάζοντας την αξιοπιστία των μετρήσεων χρόνου. Για τον μετριασμό αυτού του φαινομένου, υλοποιήθηκε παύση 1 δευτερολέπτου (`time.sleep`) πριν από κάθε εκτέλεση αλγορίθμου, ώστε να επιτρέπεται η μερική ψύξη του επεξεργαστή. (Python Software Foundation, 2024)

3.6.4 Garbage Collector

Η Python διαθέτει αυτόματο μηχανισμό αποδέσμευσης μνήμης (Garbage Collector - GC), ο οποίος ενδέχεται να ενεργοποιηθεί κατά τη διάρκεια εκτέλεσης ενός αλγορίθμου, εισάγοντας μη ντετερμινιστική καθυστέρηση στις μετρήσεις χρόνου. Για την εξάλειψη αυτής της πηγής θορύβου, πριν από κάθε εκτέλεση καλείται ρητά η εντολή `gc.collect()`, η οποία αναγκάζει τον GC να εκτελεστεί και να ολοκληρώσει τυχόν εκκρεμείς αποδεσμεύσεις, διασφαλίζοντας ότι δεν θα παρεμβληθεί κατά τη μέτρηση. (Python Software Foundation, 2024)

3.7 Περιορισμοί της Μελέτης

Παρά τα μέτρα αξιοπιστίας που περιγράφηκαν, η παρούσα μεθοδολογία εμπεριέχει συγκεκριμένους περιορισμούς που πρέπει να ληφθούν υπόψη κατά την ερμηνεία των αποτελεσμάτων. Οι περιορισμοί αυτοί αφορούν θεμελιώδεις τεχνολογικούς παράγοντες της πλατφόρμας εκτέλεσης, μεθοδολογικές επιλογές σχετικά με τη συχνότητα των μετρήσεων, καθώς και υλικούς περιορισμούς του συστήματος.

3.7.1 Όριο Αναδρομής και Σταθερότητα Συστήματος

Όπως αναφέρθηκε στην ενότητα 3.6.2, το όριο αναδρομής ανυψώθηκε στις 20.000 κλήσεις μέσω της εντολής `sys.setrecursionlimit(20000)` για την υποστήριξη αλγορίθμων με βαθύ αναδρομικό δέντρο. Ωστόσο, το όριο αυτό δεν είναι απεριόριστο, τελικά περιορίζεται από το μέγεθος της στοίβας (stack size) που διαθέτει το λειτουργικό σύστημα για κάθε διεργασία.

Η Python εκτελεί έλεγχο αναδρομικού βάθους σε επίπεδο ερμηνευτή προκειμένου να αποτρέψει υπερχείλιση στοίβας (stack overflow) σε επίπεδο λειτουργικού συστήματος. Όμως, αν το `setrecursionlimit()` οριστεί σε υπερβολικά υψηλή τιμή, το πρόγραμμα μπορεί να ξεπεράσει το όριο μνήμης στοίβας που παρέχει το λειτουργικό σύστημα πριν προλάβει η Python να εκδώσει το σφάλμα `RecursionError`. Στην περίπτωση αυτή, το πρόγραμμα τερματίζει απότομα με `segmentation fault` (Unix) ή `MemoryError` (Windows), χωρίς δυνατότητα ελεγχόμενης διακοπής (Beazley, 2009).

Αυτός ο περιορισμός σημαίνει ότι ακόμη και με την αύξηση του ορίου αναδρομής, αλγόριθμοι που παράγουν πολύ βαθιές αναδρομικές κλήσεις, εξακολουθούν να τερματίζουν πρόωρα. Οι εκτελέσεις αυτές καταγράφονται ως `Recursion Limit` στο

σύνολο δεδομένων, αναγνωρίζοντας ότι η υλοποίηση δεν μπόρεσε να ολοκληρωθεί εντός των τεχνικών ορίων της πλατφόρμας.

3.7.2 Μονοεπαναληπτικό Πειραματικό Σχέδιο

Κάθε συνδυασμός (αλγόριθμος $\times$ N $\times$ κατηγορία εισόδου) εκτελέστηκε μία φορά. Η επιλογή αυτή έγινε λόγω του σημαντικού υπολογιστικού κόστους, καθώς με 36 αλγορίθμους, 4 μεγέθη, 3 κατηγορίες και ανώτατο χρονικό όριο 180 δευτερολέπτων ανά εκτέλεση, το συνολικό φάσμα πειραμάτων έφτανε τις 432 ατομικές εκτελέσεις, αρκετές εκ των οποίων πλησίαζαν το timeout.

Από στατιστική σκοπιά, η μονοεπαναληπτική μέτρηση δεν επιτρέπει τον υπολογισμό μέτρων διασποράς (τυπική απόκλιση, διακύμανση) ούτε τη διεξαγωγή ελέγχων στατιστικής σημαντικότητας για τη σύγκριση των αλγορίθμων. Σύμφωνα με τη βιβλιογραφία στο χώρο του benchmarking, ο ελάχιστος συνιστώμενος αριθμός επαναλήψεων είναι 3, ενώ για αξιόπιστα αποτελέσματα συνήθως χρησιμοποιούνται 10 έως 30 επαναλήψεις (Georges, 2007). Η απουσία επαναλήψεων αυξάνει την ευαισθησία των μετρήσεων σε μεταβλητότητα που προκαλείται από προσωρινές συνθήκες του συστήματος (background processes, context switches, cache effects).

Παρόλα αυτά, τα μέτρα αξιοπιστίας που περιγράφηκαν στην ενότητα 3.6 (garbage collection, timeout, παύση μεταξύ εκτελέσεων) συμβάλλουν στη μείωση του θορύβου μεμονωμένης μέτρησης, επιτρέποντας την ποιοτική σύγκριση της τάξης μεγέθους των επιδόσεων μεταξύ των αλγορίθμων.

3.7.3 Στατική Παύση και Απουσία Δυναμικής Θερμικής Διαχείρισης

Για την αντιμετώπιση της θερμικής επιβράδυνσης (thermal throttling) υλοποιήθηκε στατική παύση 1 δευτερολέπτου με την `time.sleep(1)`, πριν από κάθε εκτέλεση αλγορίθμου. Σκοπός της παύσης αυτής ήταν για δύο βασικούς λόγους, αφενός να επιτρέψει τη μερική ψύξη του επεξεργαστή, αφετέρου να παρέχει στο λειτουργικό σύστημα ένα περιθώριο εκτέλεσης background processes (context switches, I/O operations) που θα μπορούσαν να επηρεάσουν τις μετρήσεις.

Ωστόσο, η προσέγγιση αυτή δεν περιλαμβάνει δυναμικό έλεγχο θερμοκρασίας, καθώς δεν υπάρχει αναμονή μέχρι η CPU να ψυχθεί κάτω από συγκεκριμένο κατώφλι (π.χ. 60°C) πριν ξεκινήσει η επόμενη μέτρηση. Αυτό σημαίνει ότι μετά από μία σειρά πολλαπλών εκτελέσεων, ο επεξεργαστής ενδέχεται να μην έχει επανέλθει στη βασική του θερμοκρασία idle πριν ξεκινήσει η επόμενη, με πιθανό αποτέλεσμα να αλλοιωθούν τα στατιστικά.

Επιπλέον, η θερμοκρασία καταγράφεται μέσω custom κώδικα που λειτουργεί μόνο σε περιβάλλον Linux (π.χ. ανάγνωση `/sys/class/thermal/thermal_zone*/temp`). Στο χρησιμοποιούμενο περιβάλλον WSL2 σε Windows 11, ο κώδικας αυτός δεν μπόρεσε να επιστρέψει αξιόπιστες μετρήσεις θερμοκρασίας, οπότε εκτελέστηκε χωρίς σφάλμα αλλά χωρίς καταγραφή θερμικών δεδομένων. Συνεπώς, δεν είναι δυνατή η ποσοτική επαλήθευση της επίδρασης της θερμοκρασίας στους μετρούμενους χρόνους.

Στη βιβλιογραφία του hardware benchmarking, προτείνεται είτε επαρκής περίοδος idle μεταξύ πειραμάτων (συχνά 10-30 λεπτά για πλήρη ψύξη), είτε συνεχής παρακολούθηση και κατώφλι θερμοκρασίας πριν την έναρξη κάθε test (Haj-Yihia, 2016). Η απουσία τέτοιας διαδικασίας αποτελεί περιορισμό που θα μπορούσε να μετριαστεί σε μελλοντική έρευνα.

ΚΕΦΑΛΑΙΟ 4 Αποτελέσματα και Συγκριτική Ανάλυση

4.1 Επισκόπηση Αποτελεσμάτων

Το παρόν κεφάλαιο παρουσιάζει και αναλύει τα εμπειρικά αποτελέσματα που συλλέχθηκαν κατά τη διεξαγωγή των πειραμάτων που περιγράφονται στο Κεφάλαιο 3. Συνολικά εξετάστηκαν 36 αλγόριθμοι-εκδόσεις σε τέσσερα μεγέθη εισόδου (1.000, 10.000, 100.000, 1.000.000 στοιχεία) και τρεις κατηγορίες εισόδου (Best, Random, Worst Case), παράγοντας 432 ατομικές μετρήσεις χρόνου εκτέλεσης και κατανάλωσης μνήμης.

Η δομή του κεφαλαίου ακολουθεί ιεραρχία από το γενικό στο ειδικό. Ξεκινάμε με μια επισκόπηση αποτυχιών εκτέλεσης, συνεχίζουμε με αναλυτική παρουσίαση ανά κατηγορία αλγορίθμου και ολοκληρώνουμε με συνολική συγκριτική ανάλυση που αναδεικνύει τις διαφορές σε χρόνο και μνήμη μεταξύ κατηγοριών.

4.1.1 Αποτυχίες Εκτέλεσης - Time Limit & Recursion Limit

Από τις 432 συνολικές εκτελέσεις, 46 δεν ολοκληρώθηκαν με επιτυχία. Οι αποτυχίες χωρίζονται σε δύο τύπους, υπέρβαση χρονικού ορίου (Time Limit) και υπέρβαση ορίου αναδρομής (Recursion Limit). Και οι δύο κατηγορίες έχουν ήδη αναλυθεί μεθοδολογικά στις Ενότητες 3.6.1 και 3.6.2 αντίστοιχα.

Αλγόριθμος	Τύπος Αποτυχίας	Μέγεθος Εισόδου (N)	Σενάρια Εισόδου
Block Sort	Time Limit	1.000.000	Random, Worst
Bubble Sort	Time Limit	100.000, 1.000.000	Random, Worst
Bucket Sort	Time Limit	1.000.000	Random, Worst
Cocktail Shaker Sort	Time Limit	100.000, 1.000.000	Random, Worst
Cycle Sort	Time Limit	100.000, 1.000.000	Best, Random, Worst
Gnome Sort	Time Limit	100.000, 1.000.000	Random, Worst
Insertion Sort	Time Limit	100.000, 1.000.000	Random, Worst
Pairwise Sorting Network	Time Limit	1.000.000	Random
Pancake Sort	Time Limit	100.000, 1.000.000	Best, Random, Worst
Selection Sort	Time Limit	100.000, 1.000.000	Best, Random, Worst
Strand Sort	Time Limit	100.000	Worst
Strand Sort	Time Limit	1.000.000	Random, Worst
Tree Sort	Recursion Limit	100.000, 1.000.000	Best, Worst

4.1.2 Ερμηνεία των αποτυχιών Time Limit

Η πλειονότητα των TL αποτυχιών αφορά αλγόριθμους $O(n^2)$ που δεν είναι adaptive (Selection Sort, Cycle Sort, Pancake Sort). Ο Selection Sort, λόγω του ότι εκτελεί πάντα ακριβώς $O(n^2)$ συγκρίσεις ανεξαρτήτως εισόδου, απέτυχε σε όλες τις εκτελέσεις για $N \geq 100.000$. Αντίθετα, οι adaptive αλγόριθμοι Bubble Sort, Insertion Sort, Gnome Sort και Cocktail Shaker Sort απέτυχαν μόνο στα Random και Worst case. Στο Best Case έτρεξαν κανονικά, ακόμα και για $N=1.000.000$, επαληθεύοντας εμπειρικά την ιδιότητά τους $O(n)$ για ήδη ταξινομημένη είσοδο (Cormen, 2009).

Ο Strand Sort απέτυχε στο Worst Case (αντίστροφα ταξινομημένος πίνακας) για $N \geq 100.000$, όπου κάθε υπολίστα (strand) αποτελείται από ένα στοιχείο και ο αλγόριθμος εκπίπτει σε $O(n^2)$. Επιπλέον, απέτυχε και στο Random Case για $N=1.000.000$. Αυτό οφείλεται στο γεγονός ότι, με τυχαία είσοδο, το μέσο μήκος κάθε υπολίστας είναι πολύ μικρό, με αποτέλεσμα ο αλγόριθμος να "κόβει" την αρχική δομή σε υπερβολικά πολλά ανεξάρτητα κομμάτια. Η διαδικασία συγχώνευσης (merging) όλων αυτών των μικρών λιστών δημιουργεί τεράστιο overhead διαχείρισης μνήμης στην Python, οδηγώντας στην υπέρβαση του χρονικού ορίου. Ο Block Sort απέτυχε στα non-Best cases για $N=1M$, γιατί παρά τη θεωρητική $O(n \log n)$ πολυπλοκότητα, η εκτέλεση αποκλειστικά μέσω του διερμηνέα της Python εισάγει σημαντικά σταθερά κόστη από τις λεπτομερείς block-based λειτουργίες, με αποτέλεσμα να αγγίζει το όριο. Η βελτιστοποιημένη έκδοση Block Sort (Alt) ωστόσο ολοκλήρωσε κανονικά όλες τις εκτελέσεις, αναδεικνύοντας τη συνολική επίδραση του αμιγούς κώδικα υλοποίησης σε αντιδιαστολή με τις δομικές βελτιστοποιήσεις που προσφέρουν οι εγγενείς (built-in) συναρτήσεις της γλώσσας.

Ο Bucket Sort παρουσίασε αποτυχία χρονικού ορίου (Time Limit) στις περιπτώσεις Random και Worst Case για $N=1.000.000$. Παρότι ο αλγόριθμος διαθέτει θεωρητική μέση πολυπλοκότητα $O(n+k)$, η αμιγώς σε Python υλοποίησή του επιφέρει τεράστιο διαχειριστικό κόστος (overhead). Η διαδικασία κατανομής ενός εκατομμυρίου στοιχείων σε πλήθος επιμέρους λιστών (buckets), η δυναμική προσθήκη στοιχείων σε αυτές και η τελική συνένωσή τους, προκαλούν εξαιρετικά μεγάλο αριθμό λειτουργιών σε επίπεδο διερμηνέα, οδηγώντας στην υπέρβαση του ορίου των 180 δευτερολέπτων. Αντίθετα, στο Best Case για το ίδιο μέγεθος, ο αλγόριθμος ολοκληρώθηκε επιτυχώς (3.7s), καθώς η ομοιόμορφη ή ήδη ταξινομημένη κατανομή ελαχιστοποιεί τις συγκρούσεις και τη διαδικασία ταξινόμησης εντός των buckets.

Ο Pairwise Sorting Network απέτυχε σε μία μόνο εκτέλεση ($N=1.000.000$, Random), ενώ τα Best και Worst ολοκληρώθηκαν οριακά εντός του ορίου (~142s και ~163s αντίστοιχα). Η αποτυχία αυτή παρουσιάζει ιδιαίτερο θεωρητικό ενδιαφέρον, καθώς τα δίκτυα ταξινόμησης (sorting networks) είναι "data-independent" και εκτελούν ακριβώς τον ίδιο αριθμό συγκρίσεων ανεξαρτήτως της αρχικής διάταξης της εισόδου. Επομένως, το Time Limit στην τυχαία είσοδο δεν οφείλεται σε αστάθεια της αλγοριθμικής πολυπλοκότητας, αλλά στην αρχιτεκτονική του συστήματος. Η τυχαία διάταξη προκαλεί εξαιρετικά ασυνεχή μοτίβα προσπέλασης μνήμης (cache misses) και διαρκείς εναλλαγές τιμών (swaps), γεγονός που επιβαρύνει δραματικά το overhead διαχείρισης μνήμης (garbage collection) της μηχανής της Python, καθυστερώντας την εκτέλεση πέραν του αποδεκτού ορίου.

4.1.3 Ερμηνεία των αποτυχιών Recursion Limit

Ο Tree Sort αποτελεί τη μοναδική περίπτωση Recursion Limit στο σύνολο δεδομένων. Όπως εξηγήθηκε στην Ενότητα 3.6.2, για ήδη ταξινομημένη (Best Case) ή αντίστροφα ταξινομημένη (Worst Case) είσοδο, το Δυαδικό Δέντρο Αναζήτησης εκφυλίζεται σε γραμμική αλυσίδα βάθους $O(n)$ (Cormen, 2009). Για $N \geq 100.000$ αυτό υπερβαίνει ακόμα και το ανυψωμένο όριο των 20.000 κλήσεων. Χαρακτηριστικά, για τυχαία είσοδο (Random Case) ο Tree Sort εκτελέστηκε κανονικά ακόμα και για $N=1.000.000$ (6.57s), καθώς ένα BST με τυχαία στοιχεία έχει αναμενόμενο βάθος $O(\log n)$.

4.1.4 Γενική Εικόνα του Συνόλου Δεδομένων

Το πειραματικό σύνολο δεδομένων αποτελείται από 29 αλγορίθμους που έχουν αναπτυχθεί εξ ολοκλήρου σε Python, επιλεγμένους ώστε να καλύπτουν πέντε διακριτές κατηγορίες πολυπλοκότητας, δηλαδή αλγορίθμους $O(n^2)$, αλγορίθμους $O(n \log n)$, υβριδικούς, δίκτυα ταξινόμησης και αλγορίθμους χωρίς σύγκριση. Επιπλέον, 7 βελτιστοποιημένες εκδόσεις (Alt) που αξιοποιούν βοηθητικές βιβλιοθήκες, στις οποίες υπάρχει μέσα και ο ενσωματωμένος Tim Sort (Built-in), εντάχθηκαν στο σύνολο δεδομένων και αναλύονται ξεχωριστά στην Ενότητα 4.7.

Κάθε αλγόριθμος δοκιμάστηκε σε 4 μεγέθη εισόδου ($N = 1.000, 10.000, 100.000, 1.000.000$) και 3 τύπους εισόδου (Best, Random, Worst Case), παράγοντας 348 ατομικές μετρήσεις για τους 29 επίσημους αλγορίθμους (συνολικά 432 μετρήσεις για το πλήρες σετ των 36 εκδόσεων). Από αυτές, 46 κατέληξαν σε αποτυχία (Time Limit ή Recursion Limit), συμπεριλαμβάνοντας τους αλγορίθμους που αναλύθηκαν στις προηγούμενες ενότητες.

4.2 Αλγόριθμοι Τετραγωνικής Πολυπλοκότητας $O(n^2)$

Η κατηγορία αυτή περιλαμβάνει επτά αλγορίθμους που χαρακτηρίζονται από τετραγωνική χρονική πολυπλοκότητα στη μέση και χειρότερη περίπτωση. Χωρίζονται σε δύο υποομάδες, τις τρεις κλασικές βασικές υλοποιήσεις (Bubble Sort, Selection Sort, Insertion Sort) και τέσσερις παραλλαγές ή εξειδικευμένους αλγορίθμους (Gnome Sort, Cocktail Shaker Sort, Cycle Sort, Pancake Sort). Οι πρώτοι αποτελούν σημεία αναφοράς τόσο στη βιβλιογραφία όσο και στην παρούσα ανάλυση, καθώς η θεωρητική τους συμπεριφορά είναι πλήρως τεκμηριωμένη και ευρέως μελετημένη (Cormen, 2009).

4.2.1 Βασικές Υλοποιήσεις - Bubble Sort, Selection Sort, Insertion Sort

Οι τρεις κλασικοί αλγόριθμοι τετραγωνικής πολυπλοκότητας επιβεβαιώνουν εμπειρικά αυτό που η θεωρία προβλέπει, δηλαδή η απόδοσή τους υποβαθμίζεται απότομα καθώς το N αυξάνεται. Στα $N=1.000$, και οι τρεις ολοκληρώνουν σε κλάσματα δευτερολέπτου, αλλά ήδη στα $N=10.000$ τα Random και Worst σενάρια απαιτούν δεκάδες δευτερόλεπτα (Cormen, 2009).

Το κρίσιμο ερώτημα της ομάδας δεν είναι αν είναι αργοί, αλλά ποιοι εκμεταλλεύονται την τάξη της εισόδου. Ο Insertion Sort και ο Bubble Sort διαθέτουν μηχανισμό πρώιμου τερματισμού (early exit), καθώς τερματίζουν μετά από μία μόνο γραμμική διέλευση σε ήδη ταξινομημένο πίνακα. Αυτό αντικατοπτρίζεται άμεσα στα δεδομένα, καθώς ο Insertion Sort ολοκληρώνει σε 0.003s για $N=1.000$ και σε 1.28s για $N=1.000.000$ στο Best Case, ενώ ο Bubble Sort επιτυγχάνει 0.003s και 0.83s αντίστοιχα. Επιπλέον, ο Insertion Sort υπερτερεί σαφώς του Bubble Sort στα μη-ταξινομημένα σενάρια, με 24.6s έναντι 67.3s στο Random για $N=10.000$. Η διαφορά δεν οφείλεται στην πολυπλοκότητα (ίδια, $O(n^2)$), αλλά στη σταθερά, δεδομένου ότι ο Bubble Sort εκτελεί συστηματικά περισσότερες εναλλαγές ανά πέρασμα (Knuth, 1998).

Ο Selection Sort βρίσκεται στον αντίποδα ως προς την προσαρμοστικότητα. Δεν διαθέτει κανέναν μηχανισμό πρώιμου τερματισμού και εκτελεί πάντα τον ίδιο αριθμό συγκρίσεων ανεξαρτήτως εισόδου (Cormen, 2009). Αυτό αντικατοπτρίζεται στους χρόνους για $N=1.000$ όπου παρατηρείται το παράδοξο φαινόμενο η καλύτερη περίπτωση με 0.219s να είναι ελαφρώς πιο αργή από την τυχαία και τη χειρότερη που απαιτούν 0.189s. Εφόσον οι συγκρίσεις παραμένουν σταθερές η μικρή αυτή απόκλιση οφείλεται συνήθως στο λειτουργικό κόστος της πρόβλεψης διακλαδώσεων του επεξεργαστή. Η πλήρης απουσία προσαρμοστικότητας τον καθιστά τον πιο αδύναμο της ομάδας σε επίπεδο κλιμάκωσης εμφανίζοντας Time Limit ήδη από $N=100.000$ σε όλα τα σενάρια χωρίς εξαίρεση.

Από πλευράς μνήμης, και οι τρεις αλγόριθμοι είναι in-place με αμελητέα κατανάλωση. Ο Insertion Sort χρησιμοποιεί μόλις 0.17 KB, ο Bubble Sort 0.24 KB (Best) έως 0.30 KB (Random/Worst), και ο Selection Sort 0.30 KB. Αυτές οι τιμές παραμένουν ουσιαστικά σταθερές ανεξαρτήτως N , επιβεβαιώνοντας πλήρως την $O(1)$ χωρική πολυπλοκότητα.

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Bubble Sort	1.000	0.003	0.477	0.545	0.3
Bubble Sort	10.000	0.029	67.286	89.858	0.3
Bubble Sort	100.000	0.084	Time Limit	Time Limit	0.24
Bubble Sort	1.000.000	0.828	Time Limit	Time Limit	0.24
Selection Sort	1.000	0.219	0.189	0.189	0.3
Selection Sort	10.000	19.976	20.156	20.264	0.3
Selection Sort	100.000	Time Limit	Time Limit	Time Limit	-
Selection Sort	1.000.000	Time Limit	Time Limit	Time Limit	-
Insertion Sort	1.000	0.003	0.228	0.279	0.17
Insertion Sort	10.000	0.046	24.62	45.241	0.17
Insertion Sort	100.000	0.164	Time Limit	Time Limit	0.17
Insertion Sort	1.000.000	1.282	Time Limit	Time Limit	0.17

4.2.2 Παραλλαγές & εξειδικευμένοι - Gnome, Cocktail Shaker, Cycle, Pancake

Οι τέσσερις αυτοί αλγόριθμοι δεν βελτιώνουν τη θεωρητική πολυπλοκότητα $O(n^2)$, αλλά αποτελούν ενδιαφέρουσες παραλλαγές της ίδιας ιδέας. Τα εμπειρικά αποτελέσματα δείχνουν ότι η ομάδα αυτή υστερεί συγκριτικά με τις βασικές υλοποιήσεις της προηγούμενης ενότητας με εξαίρεση τον Pancake Sort ο οποίος στο τυχαίο σενάριο υπερτερεί σημαντικά του Bubble Sort. Επιπλέον παρατηρείται ορατή διαφοροποίηση στο πόσο γρήγορα αντιδρά η κάθε υλοποίηση στην τάξη της εισόδου.

Ο Gnome Sort και ο Cocktail Shaker Sort ακολουθούν λογική παρόμοια με τον Bubble Sort και επωφελούνται αποτελεσματικά από ταξινομημένα δεδομένα. Στο Best Case για $N=1.000$, ο Gnome ολοκληρώνει σε 0.004s και ο Cocktail σε 0.003s, με τις επιδόσεις να κλιμακώνονται γραμμικά έως $N=1.000.000$ (0.87s και 0.83s αντίστοιχα). Αντίθετα, ο Cycle Sort και ο Pancake Sort αδυνατούν να εκμεταλλευτούν την τάξη, καθώς ο Cycle Sort καταγράφει 0.231s στο Best Case για $N=1.000$, ενώ ο Pancake Sort, αν και για $N=1.000$ εμφανίζει παρόμοιους χρόνους (0.157s-0.201s). Στα μεγαλύτερα δείγματα ($N=10.000$) παρουσιάζει αισθητή επιβάρυνση στο Random σενάριο (29.62s έναντι 18.60s του Best Case). Το γεγονός αυτό υποδηλώνει ότι ο αλγόριθμος δεν παρουσιάζει πλήρη αναισθησία στη διάταξη της εισόδου, καθώς η τυχαία σειρά φαίνεται να προκαλεί αυξημένο κόστος στην πρόβλεψη διακλαδώσεων (branch prediction) και στην τοπικότητα των δεδομένων (cache locality) κατά τις συνεχείς αντιστροφές.

Στα Random και Worst σενάρια, η εικόνα επιδεινώνεται σημαντικά για όλους. Στα $N=10.000$, ο Gnome Sort φτάνει τα 140s στο Worst Case (ο πιο αργός αλγόριθμος ολόκληρης της κατηγορίας), ο Cocktail Shaker 122s, και ο Cycle Sort 129.7s (Random). Αξιοσημείωτη εξαίρεση αποτελεί ο Cycle Sort στον οποίο ο χρόνος της χειρότερης περίπτωσης με 67.7s είναι σημαντικά μικρότερος από την τυχαία διάταξη που φτάνει τα 129.7s για $N=10.000$. Το γεγονός αυτό αντανακλά ότι η απόδοση εξαρτάται πρωτίστως από τον αριθμό κυκλωμάτων στη μετάθεση και όχι απλώς από τη γενική διάταξη των στοιχείων. Όλοι εμφανίζουν Time Limit από $N=100.000$ για μη-ταξινομημένα σενάρια, ενώ ο Cycle Sort και ο Pancake Sort αποτυγχάνουν ακόμα και στο Best Case από αυτό το μέγεθος.

Ιδιαίτερο ενδιαφέρον παρουσιάζει η χωρική πολυπλοκότητα του Pancake Sort ο οποίος διαφοροποιείται από την υπόλοιπη ομάδα. Ενώ οι άλλοι αλγόριθμοι λειτουργούν in place με σταθερή κατανάλωση ο Pancake Sort καταγράφει σημαντική αύξηση μνήμης καθώς μεγαλώνει η είσοδος. Η παρατηρούμενη γραμμική αύξηση δεν αποτελεί εγγενές δομικό χαρακτηριστικό του αλγορίθμου ο οποίος θεωρητικά είναι in place αλλά συνιστά ιδιαιτερότητα του κώδικα στην Python. Συγκεκριμένα η χρήση κομματιών λίστας για την εκτέλεση των αντιστροφών προκαλεί τη δημιουργία προσωρινών αντιγράφων στη μνήμη. Συνεπώς, η πειραματική επαλήθευση αναδεικνύει πώς οι περιορισμοί του περιβάλλοντος εκτέλεσης μπορούν να αλλοιώσουν τα θεωρητικά χαρακτηριστικά ενός αλγορίθμου.

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
------------	---	------	--------	-------	-----------------

Gnome Sort	1.000	0.004	0.616	0.875	0.11
Gnome Sort	10.000	0.037	72.11	140.081	0.11
Gnome Sort	100.000	0.107	Time Limit	Time Limit	0.11
Gnome Sort	1.000.000	0.867	Time Limit	Time Limit	0.11
Cocktail Shaker Sort	1.000	0.003	0.663	1.081	0.32
Cocktail Shaker Sort	10.000	0.039	74.874	122.196	0.32
Cocktail Shaker Sort	100.000	0.095	Time Limit	Time Limit	0.2
Cocktail Shaker Sort	1.000.000	0.832	Time Limit	Time Limit	0.2
Cycle Sort	1.000	0.231	0.97	0.642	0.3
Cycle Sort	10.000	21.478	129.668	67.685	0.3
Cycle Sort	100.000	Time Limit	Time Limit	Time Limit	-
Cycle Sort	1.000.000	Time Limit	Time Limit	Time Limit	-
Pancake Sort	1.000	0.18	0.201	0.157	8.02
Pancake Sort	10.000	18.6	29.616	18.666	78.33
Pancake Sort	100.000	Time Limit	Time Limit	Time Limit	-
Pancake Sort	1.000.000	Time Limit	Time Limit	Time Limit	-

4.3 Αλγόριθμοι $O(n \log n)$

4.3.1 Βασικοί - Merge Sort, Quick Sort, Heap Sort

Η μετάβαση από το $O(n^2)$ στο $O(n \log n)$ είναι από τις πιο δραματικές βελτιώσεις στην Επιστήμη Υπολογιστών, και τα εμπειρικά δεδομένα την επιβεβαιώνουν αδιαμφισβήτητα, καθώς και οι τρεις αλγόριθμοι αυτής της υποενοότητας ολοκληρώνουν επιτυχώς σε όλα τα μεγέθη εισόδου, συμπεριλαμβανομένου του $N=1.000.000$. Αυτό τους καθιστά την πρώτη ομάδα της ανάλυσής μας που δεν εμφανίζει κανέναν Time Limit (Cormen, 2009).

Ως προς την προσαρμοστικότητα, τα τρία αυτά σχήματα διαφέρουν σημαντικά μεταξύ τους. Ο Heap Sort παρουσιάζει την πιο σταθερή συμπεριφορά, καθώς εκτελεί πάντα $O(n \log n)$ συγκρίσεις ανεξαρτήτως εισόδου, και τα δεδομένα το αποδεικνύουν με ακρίβεια. Για ένα εκατομμύριο στοιχεία ο χρόνος κυμαίνεται μεταξύ 22.8 δευτερολέπτων στη χειρότερη περίπτωση και 25.3 δευτερολέπτων στην καλύτερη περίπτωση. Η παράδοση αυτή υπεροχή της αντίστροφα ταξινομημένης εισόδου εξηγείται από τη διαδικασία κατασκευής του σωρού, καθώς η ήδη ταξινομημένη διάταξη συχνά προκαλεί μεγαλύτερο αριθμό λειτουργιών ολίσθησης προς τα κάτω στην προσπάθεια του αλγορίθμου να διατηρήσει την ιδιότητα της δομής. Ο Quick

Sort εμφανίζει μεγαλύτερη διακύμανση, με 18.1s στο Best έναντι 25.3s στο Random για $N=1.000.000$. Η υλοποίησή του επιλέγει pivot με στρατηγική που αποφεύγει το θεωρητικό $O(n^2)$ χειρότερης περίπτωσης, γι' αυτό και ο Worst (19.1s) παραμένει ανταγωνιστικός. Ο Merge Sort παρουσιάζει σχεδόν αναλλοίωτο χρόνο εκτέλεσης σημειώνοντας 33.8 δευτερόλεπτα στην καλύτερη περίπτωση, 33.9 στην τυχαία, και 31.6 στη χειρότερη για ένα εκατομμύριο στοιχεία. Επιβεβαιώνοντας ότι η επίδοση του πρακτικά δεν εξαρτάται από τη διάταξη της εισόδου με τη μικρή αυτή διακύμανση και την ελαφρώς ταχύτερη ολοκλήρωση της χειρότερης περίπτωσης να αποδίδονται κατά κύριο λόγο στη συμπεριφορά της κρυφής μνήμης και στο μοτίβο των προσπελάσεων του συστήματος.

Η μεγαλύτερη διαφοροποίηση εντοπίζεται στη μνήμη. Ο Heap Sort είναι in-place με μόλις 0.96 KB στο $N=1.000.000$, μια από τις χαμηλότερες τιμές ολοκλήρης της μελέτης. Ο Quick Sort καταναλώνει από 1.22 KB στην καλύτερη περίπτωση έως 3.16 KB στην τυχαία διάταξη για ένα εκατομμύριο στοιχεία με τη μικρή αυτή διακύμανση να οφείλεται στο εκάστοτε μέγεθος της στοίβας των αναδρομικών κλήσεων. Ο Merge Sort αντίθετα απαιτεί 15.627 KB ποσό που αντιστοιχεί σε σχεδόν οκτώ χιλιάδες φορές μεγαλύτερη κατανάλωση από την αντίστοιχη του Quick στη χειρότερη περίπτωση λόγω της βοηθητικής δομής που χρησιμοποιεί κατά τη συγχώνευση. Αυτό αντιπροσωπεύει έναν κλασικό συμβιβασμό μεταξύ χρόνου και μνήμης, όπου ο Merge Sort προσφέρει εγγυημένα σταθερό χρόνο, αλλά με κόστος $O(n)$ χώρου (Cormen, 2009).

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Merge Sort	1.000	0.005	0.026	0.008	16.77
Merge Sort	10.000	0.189	0.189	0.177	157.99
Merge Sort	100.000	2.467	2.535	2.341	1564.67
Merge Sort	1.000.000	33.761	33.878	31.553	15627.6
Quick Sort	1.000	0.036	0.031	0.008	1.22
Quick Sort	10.000	0.144	0.198	0.162	2.22
Quick Sort	100.000	1.583	2.174	1.682	2.72
Quick Sort	1.000.000	18.062	25.321	19.135	3.16
Heap Sort	1.000	0.034	0.029	0.005	0.34
Heap Sort	10.000	0.17	0.143	0.115	0.59
Heap Sort	100.000	1.957	1.952	1.662	0.78
Heap Sort	1.000.000	25.292	25.155	22.805	0.96

4.3.2 Προσαρμοστικοί και Gap-Based Αλγόριθμοι - Shell, Comb, Smooth, Strand Sort

Οι αλγόριθμοι αυτής της υποενότητας δεν εντάσσονται σε μία ομοιογενή κατηγορία πολυπλοκότητας. Ο Shell Sort και ο Comb Sort επιτυγχάνουν πολυπλοκότητα περίπου $O(n \log^2 n)$ με κατάλληλη ακολουθία χασμάτων (Knuth, 1998), ενώ ο Smooth Sort πλησιάζει θεωρητικά το $O(n \log n)$. Ο Strand Sort αποκλίνει εντελώς από τους

υπόλοιπους, καθώς φτάνει το $O(n)$ στο Best Case αλλά υποβαθμίζεται σε $O(n^2)$ στο Worst. Αυτή η ανομοιογένεια φαίνεται ξεκάθαρα στα δεδομένα.

Ως προς την προσαρμοστικότητα, ο Smooth Sort είναι ο ξεκάθαρος πρωταθλητής της ομάδας. Στο Best Case για $N=1.000.000$, ολοκληρώνει σε μόλις 5.6s, ταχύτερα και από τον Heap Sort (25.3s), χάρη στον μηχανισμό που ανιχνεύει υπάρχουσες runs στην είσοδο. Ο Strand Sort επιδεικνύει ακόμα εντονότερη προσαρμοστικότητα στα μικρά μεγέθη σημειώνοντας 0.003 δευτερόλεπτα για χίλια στοιχεία στην καλύτερη περίπτωση, όντας ο ταχύτερος από κάθε άλλον αλγόριθμο στο συγκεκριμένο σενάριο. Ωστόσο, η αρχική αυτή αποδοτικότητα μειώνεται σημαντικά στα μεγάλα μεγέθη, καθώς για ένα εκατομμύριο στοιχεία απαιτεί πάνω από εκατό δευτερόλεπτα ακόμη και στην ήδη ταξινομημένη είσοδο, γεγονός που αποδεικνύει τον εξαιρετικά περιορισμένο ορίζοντα κλιμάκωσης του. Ο Shell Sort εκμεταλλεύεται επίσης την τάξη αποτελεσματικά, ολοκληρώνοντας σε 14.9s για $N=1.000.000$ (Best). Ο Comb Sort, αντίθετα, δείχνει ελάχιστη προσαρμοστικότητα, με τους χρόνους Best και Worst να μην απέχουν σημαντικά.

Στα Random και Worst σενάρια, η εικόνα αντιστρέφεται για αρκετούς από αυτούς. Ο Shell Sort εμφανίζει την πιο δραματική υποβάθμιση, φτάνοντας τα 102.2s στο Random για $N=1.000.000$, ενώ στο Worst επιστρέφει στα 29.4s. Αυτή η ανωμαλία οφείλεται στη φύση των χάσμάτων (gaps) που χρησιμοποιεί η υλοποίηση, όπου η τυχαία διάταξη δημιουργεί μοτίβα που παρεμποδίζουν αποτελεσματική μείωση του χάσματος, ενώ η αντίστροφη διάταξη ευνοεί πιο κανονική μείωση (Sedgewick, 1996). Ο Strand Sort κατέρρευσε πλήρως, εμφανίζοντας Time Limit στο Worst Case για $N=100.000$ και στο Random Case για $N=1.000.000$, γιατί στη χειρότερη περίπτωση (αντίστροφα ταξινομημένα δεδομένα) κάθε πέρασμα εξάγει μόνο ένα στοιχείο, υποβαθμίζοντας τον αλγόριθμο σε $O(n^2)$.

Στη μνήμη, η ομάδα χωρίζεται καθαρά σε δύο κατηγορίες. Ο Shell Sort (0.20-0.26 KB) και ο Comb Sort (0.29 KB) παραμένουν ουσιαστικά in-place. Ο Smooth Sort αυξάνεται ελαφρά από τα 1.14 KB έως τα 1.95 KB εξαιτίας των μεταδεδομένων λογαριθμικής πολυπλοκότητας που απαιτούνται για τη διαχείριση του σωρού Λεονάρντο και τις αναδρομικές κλήσεις διατηρώντας ωστόσο τον επί τόπου χαρακτήρα του καθώς δεν δημιουργεί γραμμικές δομές αποθήκευσης στο παρασκήνιο. Ο Strand Sort εμφανίζει γραμμική αύξηση μνήμης ανάλογη του μεγέθους N η οποία στην καλύτερη περίπτωση ξεκινά από τα 16.7 KB για χίλια στοιχεία και φτάνει στα 16.063 KB για ένα εκατομμύριο στοιχεία. Παράλληλα, αξίζει να σημειωθεί πως στη χειρότερη περίπτωση για χίλια στοιχεία η κατανάλωση σχεδόν διπλασιάζεται αγγίζοντας τα 31.7 KB, γεγονός που αντανakλά τη δυναμική φύση της βοηθητικής λίστας που δημιουργείται κατά τη διαδικασία.

Αλγόριθμος	N	Best	Random	Worst Case	Μέγ. Μνήμη (KB)
Shell Sort	1.000	0.015	0.047	0.01	0.23
Shell Sort	10.000	0.126	0.363	0.237	0.26
Shell Sort	100.000	1.252	6.559	2.713	0.26
Shell Sort	1.000.000	14.866	102.167	29.384	0.26

Comb Sort	1.000	0.035	0.021	0.054	0.29
Comb Sort	10.000	0.315	0.443	0.335	0.29
Comb Sort	100.000	3.613	5.282	4.107	0.29
Comb Sort	1.000.000	46.407	65.823	49.163	0.29
Smooth Sort	1.000	0.024	0.053	0.056	1.3
Smooth Sort	10.000	0.067	0.338	0.265	1.58
Smooth Sort	100.000	0.598	3.884	3.007	1.8
Smooth Sort	1.000.000	5.615	44.245	33.646	1.95
Strand Sort	1.000	0.003	0.062	0.308	31.73
Strand Sort	10.000	0.017	1.459	36.705	312.98
Strand Sort	100.000	0.981	50.369	Time Limit	3123.97
Strand Sort	1.000.000	102.634	Time Limit	Time Limit	16063.45

4.3.3 Αλγόριθμοι βασισμένοι σε δέντρα - Tree, Cartesian Tree, Tournament, Patience Sort

Οι τέσσερις αυτοί αλγόριθμοι αξιοποιούν δομές δέντρου για να επιτύχουν $O(n \log n)$ θεωρητικό χρόνο, αλλά η εμπειρική τους συμπεριφορά είναι η πλέον ανομοιογενής από κάθε ομάδα της ανάλυσης. Το ενδιαφέρον δεν έγκειται στο ποιοι είναι απλά γρήγοροι ή αργοί, αλλά στο πώς η δομή δεδομένων που κρύβεται πίσω από κάθε αλγόριθμο ορίζει με μαθηματική ακρίβεια ποια είσοδος τον ευνοεί και ποια τον καταρρίπτει.

Ο Tournament Sort είναι ο πιο προβλέψιμος. Χρόνοι Best/Random/Worst αποκλίνουν ελάχιστα σε κάθε N , όπου για $N=1.000.000$, 20.2s, 22.5s και 27.4s αντίστοιχα. Η συμπεριφορά αυτή θυμίζει τον Heap Sort, αλλά με σημαντικά υψηλότερη κατανάλωση μνήμης, η οποία αγγίζει τα 16.063 KB για $N=1.000.000$, έναντι 0.96 KB του Heap. Ο Cartesian Tree Sort κινείται σε παρόμοια κατεύθυνση ολοκληρώνοντας σε 3.5 δευτερόλεπτα στην καλύτερη περίπτωση και 4.0 δευτερόλεπτα στη χειρότερη για ένα εκατομμύριο στοιχεία ενώ στην τυχαία είσοδο ο χρόνος ξεφεύγει στα 28.0 δευτερόλεπτα. Η τυχαία είσοδος δημιουργεί πιο ανισόρροπα Cartesian Trees, αυξάνοντας το βάθος διέλευσης.

Ο Tree Sort αποτελεί τον μεγαλύτερο outlier ολόκληρου του κεφαλαίου και απαιτεί ξεχωριστή ανάλυση. Εμφανίζει Recursion Limit σε δύο από τα τέσσερα μεγέθη για την καλύτερη και τη χειρότερη περίπτωση (συγκεκριμένα στα 100.000 και στο 1.000.000 στοιχεία), ενώ στην τυχαία διάταξη εκτελείται κανονικά. Η αιτία είναι δομική, καθώς σε ταξινομημένη ή αντίστροφα ταξινομημένη είσοδο, κάθε νέο στοιχείο εισάγεται πάντα στον ίδιο κλάδο (δεξιά ή αριστερά), μετατρέποντας το Binary Search Tree σε μια γραμμική αλυσίδα N κόμβων. Η αναδρομική διέλευση αυτής της εκφυλισμένης δομής απαιτεί βάθος $O(n)$, που υπερβαίνει το προεπιλεγμένο όριο αναδρομής της Python (Cormen, 2009). Στο Random Case, το δέντρο παραμένει

ισορροπημένο κατά πιθανότητα, διατηρώντας βάθος $O(\log n)$ και επιτρέποντας την επιτυχή εκτέλεση σε 6.6s για $N=1.000.000$.

Ο Patience Sort παρουσιάζει την πιο παράδοξη συμπεριφορά ολόκληρης της ανάλυσης. Ο Worst Case (αντίστροφα ταξινομημένη είσοδος) είναι ο ταχύτερος, σημειώνοντας 4.0s για $N=1.000.000$. Ο Best Case (ήδη ταξινομημένη είσοδος) είναι ο πιο αργός, με 45.6s για $N=1.000.000$. Αυτό αντιστρέφει κάθε παραδοσιακή λογική και έχει άμεση θεωρητική εξήγηση, δεδομένου ότι στον Patience Sort, κάθε στοιχείο τοποθετείται στην πρώτη στοίβα (pile) της οποίας η κορυφή είναι μεγαλύτερη ή ίση με αυτό. Σε φθίνουσα διάταξη, κάθε στοιχείο είναι μικρότερο από την κορυφή της πρώτης στοίβας και τοποθετείται πάντα εκεί. Δημιουργείται έτσι μία μόνο στοίβα και η τελική συγχώνευση είναι τετριμμένη. Σε αύξουσα διάταξη, κάθε στοιχείο είναι μεγαλύτερο από κάθε τρέχουσα κορυφή, επομένως δημιουργεί νέα στοίβα. Με N στοίβες, η τελική φάση συγχώνευσης απαιτεί $O(n \log n)$ εργασία σε πλήρη έκταση, ενώ η κατανάλωση μνήμης εκτοξεύεται στα 126.301 KB (Chandramouli & Goldstein, 2014).

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Tree Sort	1.000	0.069	0.013	0.125	102.91
Tree Sort	10.000	5.234	0.061	8.065	1054.76
Tree Sort	100.000	RL	0.474	RL	8597.51
Tree Sort	1.000.000	RL	6.569	RL	85941.41
Cartesian Tree	1.000	0.014	0.019	0.017	121.64
Cartesian Tree	10.000	0.059	0.157	0.074	1168.01
Cartesian Tree	100.000	0.366	1.956	0.384	11648.79
Cartesian Tree	1.000.000	3.545	27.999	3.964	115961.61
Tournament Sort	1.000	0.005	0.027	0.02	16.54
Tournament Sort	10.000	0.141	0.146	0.14	161.38
Tournament Sort	100.000	1.544	1.778	2.077	1563.54
Tournament Sort	1.000.000	20.21	22.521	27.377	16063.29
Patience Sort	1.000	0.038	0.012	0.023	122.02
Patience Sort	10.000	0.272	0.062	0.076	1259.88
Patience Sort	100.000	3.573	1.329	0.429	12713.47
Patience Sort	1.000.000	45.583	20.721	4.029	126301.38

4.4 Υβριδικοί Αλγόριθμοι

Οι υβριδικοί αλγόριθμοι δεν βασίζονται σε μία μόνο θεωρητική τεχνική, αλλά συνδυάζουν δύο ή περισσότερες στρατηγικές για να αντισταθμίσουν τις επιμέρους αδυναμίες τους. Η κατηγορία περιλαμβάνει τέσσερις αλγορίθμους, Tim Sort, Intro Sort, Block Sort και Merge-Insertion Sort, οι οποίοι μοιράζονται θεωρητική πολυπλοκότητα $O(n \log n)$ αλλά εμπειρικά αποκλίνουν σημαντικά μεταξύ τους, τόσο σε χρόνο εκτέλεσης όσο και σε κατανάλωση μνήμης.

4.4.1 Σύγχρονες προσεγγίσεις - Simplified Tim Sort, Intro Sort

Ο Simplified Tim Sort και ο Intro Sort αποτελούν τους πιο αξιόπιστους αλγορίθμους της κατηγορίας, καθώς ολοκληρώνουν επιτυχώς και τα δώδεκα σενάρια εκτέλεσης ($4 \text{ μεγέθη} \times 3 \text{ τύποι εισόδου}$) χωρίς καμία αποτυχία. Αυτό τους τοποθετεί ανάμεσα στους πλέον scalable αλγορίθμους της μελέτης, μαζί με τους βασικούς $O(n \log n)$ αλγορίθμους που εξετάστηκαν στην Ενότητα 4.3.1.

Ως προς την προσαρμοστικότητα, και οι δύο εκμεταλλεύονται ταξινομημένα δεδομένα, αλλά με διαφορετικό μηχανισμό. Ο Simplified Tim Sort επεξεργάζεται την είσοδο ανά σταθερά τμήματα ($\text{minrun}=32$) τα οποία ταξινομεί με Insertion Sort και στη συνέχεια τα συγχωνεύει, επιτυγχάνοντας 20.26s για $N=1.000.000$ στο Best Case (Auger, 2018). Ο Intro Sort ξεκινά με Quick Sort, το οποίο ωφελείται επίσης από μερικώς ταξινομημένα δεδομένα, και ολοκληρώνει το Best Case σε 17.73s για $N=1.000.000$, ελαφρώς γρηγορότερα από τον Simplified Tim Sort (Musser, 1997). Η διαφορά στο Best Case παραμένει μικρή, αλλά αντιστρέφεται σημαντικά στα δυσμενή σενάρια.

Στα Random και Worst σενάρια, ο Simplified Tim Sort εμφανίζει αξιοσημείωτη υποβάθμιση, με 34.50s (Random) και 50.74s (Worst) για $N=1.000.000$. Το Worst Case (αντίστροφα ταξινομημένα δεδομένα) αποδεικνύεται 2.5 φορές αργότερο από το Best, μια συμπεριφορά που δεν αναμένεται από έναν αλγόριθμο που θεωρητικά ανιχνεύει φθίνουσες ακολουθίες και τις αντιστρέφει σε $O(n)$. Η αιτία βρίσκεται στην αμιγή υλοποίηση σε Python, γιατί η ανίχνευση φθινουσών runs δεν εφαρμόζεται με την ίδια αποδοτικότητα που παρέχει η C-βασισμένη έκδοση του CPython, αφήνοντας το overhead της αντιστροφής να επηρεάσει ορατά την απόδοση. Ο Intro Sort, παρά τη θεωρητική εγγύηση $O(n \log n)$ χειρότερης περίπτωσης μέσω του Heap Sort fallback, φτάνει τα 44.65s στο Worst Case για $N=1.000.000$ (Musser, 1997). Η απόκλιση από τη θεωρία οφείλεται στο interpreter overhead της Python, δεδομένου ότι η εναλλαγή μεταξύ Quick Sort και Heap Sort εισάγει κόστη που σε υλοποίηση C είναι αμελητέα.

Η μεγαλύτερη πρακτική διαφορά μεταξύ των δύο εντοπίζεται στη μνήμη. Ο Intro Sort καταναλώνει 1.88 KB (Best) έως 5.41 KB (Worst) για $N=1.000.000$, τιμές που αντανακλούν αποκλειστικά το βάθος αναδρομής και επιβεβαιώνουν $O(\log n)$ χωρική πολυπλοκότητα. Ο Simplified Tim Sort χρειάζεται 7.813 KB ($\approx 7.6 \text{ MB}$) για $N=1.000.000$, κλιμακούμενος γραμμικά με το N (8.36 KB για $N=1.000$), λόγω της βοηθητικής δομής αποθήκευσης των runs κατά τη συγχώνευση (Auger, 2018). Αυτή η αντίθεση αποτελεί παράδειγμα κλασικού συμβιβασμού χρόνου-μνήμης, όπου ο Simplified Tim Sort θυσιάζει γραμμικό χώρο για να εγγυηθεί σταθερότητα (stability) και βέλτιστη προσαρμοστικότητα.

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Tim Sort (Simplified)	1.000	0.023	0.048	0.044	8.36
Tim Sort (Simplified)	10.000	0.135	0.299	0.42	78.67
Tim Sort (Simplified)	100.000	1.583	3.255	4.701	781.79
Tim Sort (Simplified)	1.000.000	20.261	34.497	50.74	7813.04
Intro Sort	1.000	0.031	0.024	0.014	1.45
Intro Sort	10.000	0.122	0.16	0.29	2.32
Intro Sort	100.000	1.494	2.137	3.539	3.31
Intro Sort	1.000.000	17.726	25.049	44.649	5.41

4.4.2 Λοιποί - Block Sort, Merge-Insertion Sort

Ο Block Sort και ο Merge-Insertion Sort αποκαλύπτουν μια ουσιαστική αλήθεια της πειραματικής ανάλυσης, το γεγονός ότι η θεωρητική $O(n \log n)$ πολυπλοκότητα δεν αρκεί από μόνη της για να εγγυηθεί πρακτική αποδοτικότητα για έναν αλγόριθμο που έχει αναπτυχθεί εξ ολοκλήρου στη γλώσσα Python. Ο Block Sort αποτυγχάνει με Time Limit για $N=1.000.000$ στα Random και Worst σενάρια, ενώ ο Merge-Insertion Sort ολοκληρώνει όλες τις εκτελέσεις αλλά με χρόνους που τον τοποθετούν ανάμεσα στους πιο αργούς αλγορίθμους ολόκληρης της μελέτης για μεγάλα N .

Ως προς την προσαρμοστικότητα, ο Block Sort είναι ουσιαστικά non-adaptive. Οι χρόνοι Best, Random και Worst παραμένουν σχεδόν πανομοιότυποι για κάθε μέγεθος εισόδου με τιμές 0.037s / 0.038s / 0.032s για $N=1.000$ και 0.238s / 0.263s / 0.234s για $N=10.000$. Αυτή η συμπεριφορά αντανάκλα ότι η λογική του, βασισμένη σε block swaps και local merges in-place, δεν περιέχει μηχανισμό ανίχνευσης υπάρχουσας τάξης (Huang & Langston, 1988). Ο Merge-Insertion Sort παρουσιάζει παράδοξη εικόνα, ιδίως στα μικρά N , όπου για $N=1.000$ ο Random (0.0096s) είναι ταχύτερος από τον Best (0.023s), και για $N=10.000$ ο Worst (0.144s) είναι ο πιο γρήγορος. Αυτές οι αντιστροφές δεν αντανάκλουν αληθινή προσαρμοστικότητα, αλλά αποτελούν τυχαίες διακυμάνσεις που εισάγει η Python στη διαχείριση λιστών κατά τη φάση σύγκρισης. Ο Ford-Johnson αλγόριθμος είναι εγγενώς non-adaptive, δηλαδή εκτελεί πάντα τον ίδιο αριθμό συγκρίσεων ανεξαρτήτως διάταξης (Ford & Johnson, 1959).

Στα Random και Worst σενάρια, το πρόβλημα του Block Sort αναδύεται στο $N=1.000.000$, δεδομένου ότι αν και ολοκληρώθηκε το Best Case σε 41.90s, οι Random και Worst εκτελέσεις κατέληξαν σε Time Limit. Η αιτία είναι το υπερβολικά υψηλό σταθερό κόστος των block-based λειτουργιών σε επίπεδο διερμηνείας της Python, καθώς κάθε block rotation, τοπική ταξινόμηση και in-place merge μεταφράζεται σε δεκάδες bytecode εντολές, πολλαπλασιάζοντας το πρακτικό κόστος

πολύ πάνω από τη θεωρητική πολυπλοκότητα. Ο Merge-Insertion Sort ολοκληρώνει όλα τα σενάρια, αλλά για $N=1.000.000$ καταγράφει 59.03s (Best), 109.38s (Random) και 103.54s (Worst), χρόνους που υπερβαίνουν ακόμα και αρκετούς $O(n^2)$ αλγορίθμους για μικρότερα N . Η κλιμάκωση από $N=100.000$ σε $N=1.000.000$ ($10\times$ αύξηση εισόδου) προκαλεί αύξηση χρόνου κατά $28\times$ έως $40\times$, υπερβαίνοντας σαφώς την αναμενόμενη $O(n \log n)$ κλίμακα και αποδεικνύοντας ότι η συγκεκριμένη αυτόνομη υλοποίηση κουβαλά εξαιρετικά μεγάλες σταθερές.

Ο Merge-Insertion Sort αποτελεί ξεκάθαρο outlier της κατηγορίας ως προς τη μνήμη. Καταναλώνει 79.154 KB (≈ 77 MB) για $N=1.000.000$ σε κάθε περίπτωση (Best, Random και Worst), και η κλιμάκωση είναι αυστηρά γραμμική, με την κατανάλωση να αυξάνεται από 84.20 KB ($N=1.000$) σε 804.03 KB ($N=10.000$), 8.048,66 KB ($N=100.000$), και τελικά 79.154.05 KB ($N=1.000.000$), επιβεβαιώνοντας $O(n)$ χωρική πολυπλοκότητα. Η τεράστια αυτή κατανάλωση οφείλεται στη δομή του ίδιου του αλγορίθμου, ο Ford-Johnson επιτυγχάνει ελάχιστο αριθμό συγκρίσεων δημιουργώντας και διατηρώντας ενδιάμεσες λίστες σύγκρισης (insertion chains) σε κάθε βήμα, με αποτέλεσμα η χρήση μνήμης να κλιμακώνεται ακριβώς ανάλογα με το N (Ford & Johnson, 1959). Αυτό καθιστά τον αλγόριθμο απαγορευτικό για μεγάλα N σε περιβάλλοντα με περιορισμένη μνήμη, παρά τη θεωρητική του αξία ως αλγόριθμου με βέλτιστο αριθμό συγκρίσεων. Ο Block Sort καταναλώνει 9.633 KB για $N=1.000.000$, σημαντικά λιγότερο από τον Merge-Insertion Sort, αλλά και αυτός ακολουθεί γραμμική κλίμακα.

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Block Sort	1.000	0.037	0.038	0.032	16.05
Block Sort	10.000	0.238	0.263	0.234	82.52
Block Sort	100.000	3.485	4.603	4.478	1265,64
Block Sort	1.000.000	41.897	Time Limit	Time Limit	9633,33
Merge-Insertion Sort	1.000	0.023	0.01	0.014	84.2
Merge-Insertion Sort	10.000	0.187	0.165	0.144	804.03
Merge-Insertion Sort	100.000	2.1	2.688	2.448	8048,66
Merge-Insertion Sort	1.000.000	59.03	109.378	103.536	79154,05

4.5 Δίκτυα Ταξινόμησης

Τα δίκτυα ταξινόμησης (sorting networks) αντιπροσωπεύουν ένα θεμελιωδώς διαφορετικό υπολογιστικό παράδειγμα από όλους τους αλγορίθμους που εξετάστηκαν έως τώρα. Αντί να εξετάζουν τα δεδομένα και να λαμβάνουν αποφάσεις βάσει της τρέχουσας κατάστασής τους, εκτελούν μια προκαθορισμένη, στατική ακολουθία

compare-and-swap λειτουργιών που παραμένει αμετάβλητη ανεξαρτήτως εισόδου. Αυτή η αρχή επηρεάζει άμεσα κάθε πτυχή της εμπειρικής τους συμπεριφοράς.

4.5.1 Bitonic Sort, Pairwise Sorting Network

Ο Bitonic Sort και το Pairwise Sorting Network ολοκληρώνουν μαζί 23 από τα 24 συνολικά σενάρια εκτέλεσης. Ο Bitonic Sort κατακτά τέλει σκορ 12/12, ολοκληρώνοντας ακόμα και για $N=1.000.000$ και στις τρεις περιπτώσεις (141.43s / 143.61s / 142.38s). Το Pairwise Sorting Network αποτυγχάνει μόνο σε ένα σενάριο ($N=1.000.000$, Random), ενώ Best και Worst ολοκληρώνουν οριακά εντός χρονικού ορίου (142.70s και 162.70s αντίστοιχα). Χρονικά, και οι δύο βρίσκονται στο κάτω άκρο της ιεραρχίας επίδοσης. Για $N=1.000.000$ είναι 5.5 έως 7.1 φορές αργότεροι από τον Heap Sort (22-25s), πληρώνοντας το θεωρητικό κόστος της $O(n \log^2 n)$ πολυπλοκότητάς τους έναντι της $O(n \log n)$ των αλγορίθμων της Ενότητας 4.3 (Batcher, 1968).

Η πτυχή που διαφοροποιεί ριζικά τα δίκτυα ταξινόμησης από κάθε άλλη κατηγορία της μελέτης είναι η πλήρης απουσία προσαρμοστικότητας, η οποία επιβεβαιώνεται εμπειρικά με αδιαμφισβήτητη σαφήνεια. Ο Bitonic Sort παρουσιάζει το πιο επίπεδο χρονικό προφίλ ολοκλήρης της ανάλυσης, αφού για $N=1.000.000$, η μέγιστη διαφορά μεταξύ των τριών περιπτώσεων είναι μόλις 2.18 δευτερόλεπτα (141.43s / 143.61s / 142.38s), δηλαδή διακύμανση μικρότερη του 1.5%. Κανένας άλλος αλγόριθμος στη μελέτη δεν πλησιάζει αυτό το επίπεδο ομοιομορφίας. Αυτή δεν είναι αδυναμία, αλλά εγγενής ιδιότητα δεδομένου ότι η ακολουθία συγκρίσεων υπολογίζεται πριν δει ο αλγόριθμος ακόμα και το πρώτο στοιχείο, επομένως η τάξη της εισόδου είναι άνευ σημασίας (Batcher, 1968).

Το Pairwise Sorting Network εμφανίζει την ίδια βασική ομοιομορφία, αλλά με μια εμφανή ανωμαλία που απαιτεί ερμηνεία. Για $N=100.000$, το Random (18.78s) είναι 46% αργότερο από το Best (12.89s) και 27% αργότερο από το Worst (14.78s), ενώ ο Bitonic Sort στο ίδιο μέγεθος παρουσιάζει σχεδόν πλήρη ομοιομορφία (13.38s / 14.13s / 13.55s). Η διαφορά δεν οφείλεται σε επιπλέον αλγοριθμικές λειτουργίες, καθώς η ακολουθία συγκρίσεων είναι σταθερή, αλλά στα μοτίβα πρόσβασης μνήμης που δημιουργεί η τυχαία διάταξη στο Python memory model, όπου τα τυχαία δεδομένα προκαλούν μη-διαδοχικές προσβάσεις στη μνήμη, αυξάνοντας το overhead ανά σύγκριση (Parberry, 1992). Αυτή η ίδια αιτία εξηγεί και το μοναδικό Time Limit για $N=1.000.000$ Random, καθώς και τη διαφορά ανάμεσα σε Best (142.70s) και Worst (162.70s), επειδή η αντίστροφη διάταξη δημιουργεί ένα διαφορετικό, λιγότερο ευνοϊκό μοτίβο πρόσβασης από τη φθίνουσα ακολουθία, παρόλο που οι συγκρίσεις είναι αριθμητικά ίδιες.

Ως προς τη μνήμη, τα δύο αυτά δίκτυα εμφανίζουν ένα χαρακτηριστικό εξαιρετικά σπάνιο στη μελέτη, αυτό της σχεδόν πανομοιότυπης κατανάλωσης σε κάθε N (10.02 KB έναντι 10.16 KB για $N=1.000$, και 9.595 KB έναντι 9.596 KB για $N=1.000.000$). Αυτή η σύμπτωση αντανακλά ότι αμφότεροι αποθηκεύουν το ίδιο βασικό αντικείμενο, δηλαδή τον πίνακα εισόδου και τα ζεύγη comparators του δικτύου. Η

κλιμάκωση είναι γραμμική (από 10 KB για $N=1.000$ σε 9.596 KB για $N=1.000.000$, δηλαδή $\sim 960x$ αύξηση μνήμης για $1000x$ αύξηση του N), αντανakλώντας κυρίως το ίδιο το αποθηκευμένο array. Αξίζει να σημειωθεί ότι, παρά τη γραμμική μνήμη, τα ~ 9.6 MB για $N=1.000.000$ είναι υψηλότερα από όλους τους $O(n \log n)$ αλγορίθμους της Ενότητας 4.3.1, εκτός από τον Merge Sort (15.6 MB).

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Bitonic Sort	1.000	0.075	0.079	0.059	10.02
Bitonic Sort	10.000	1.212	1.249	1.222	178.12
Bitonic Sort	100.000	13.384	14.132	13.547	1267
Bitonic Sort	1.000.000	141.434	143.608	142.383	9595,78
Pairwise Sorting Network	1.000	0.058	0.059	0.073	10.16
Pairwise Sorting Network	10.000	1.133	1.457	1.26	178.57
Pairwise Sorting Network	100.000	12.893	18.781	14.778	1267,45
Pairwise Sorting Network	1.000.000	142.698	Time Limit	162.703	9596.23

4.6 Non-Comparison Αλγόριθμοι $O(n+k)$

Οι αλγόριθμοι αυτής της ενότητας δεν βασίζονται σε συγκρίσεις στοιχείων. Εκμεταλλεύονται αντ' αυτού τις αριθμητικές ιδιότητες των τιμών (ψηφία, εύρος, κατανομή) για να επιτύχουν θεωρητική πολυπλοκότητα $O(n+k)$, όπου k αντιπροσωπεύει το εύρος των τιμών ή τον αριθμό των κάδων (Cormen, 2009). Αυτή η αρχή τους επιτρέπει να ξεπεράσουν το θεωρητικό κατώτατο όριο $\Omega(n \log n)$ που περιορίζει αναγκαστικά κάθε comparison-based αλγόριθμο.

4.6.1 Αλγόριθμοι κατανομής - Counting, Radix, Pigeonhole

Η πρώτη και πιο χαρακτηριστική παρατήρηση που αναδύεται από τα δεδομένα αφορά την εξαιρετική σταθερότητα ολόκληρης της ομάδας ως προς την κατάσταση εισόδου. Για κάθε αλγόριθμο και κάθε μέγεθος N , οι χρόνοι Best, Random και Worst κινούνται μέσα σε ένα στενό εύρος, χωρίς ουσιαστική διαφορά. Αυτό είναι ακριβώς το αναμενόμενο θεωρητικό αποτέλεσμα, επειδή κανένας από τους τρεις δεν συγκρίνει στοιχεία μεταξύ τους, η σειρά με την οποία φτάνουν τα δεδομένα δεν αλλάζει τον τρόπο λειτουργίας τους (Cormen, 2009).

Η σύγκριση ταχύτητας μεταξύ των τριών αναδεικνύει ως σαφή νικήτη το Pigeonhole Sort, το οποίο ολοκληρώνει την ταξινόμηση 1.000.000 στοιχείων σε περίπου 2.32 δευτερόλεπτα, ανεξαρτήτως κατάστασης εισόδου. Ο Counting Sort ακολουθεί με $\sim 7-8$ δευτερόλεπτα, ενώ ο Radix Sort υστερεί σημαντικά ($\sim 29-30$ δευτερόλεπτα). Αξίζει να

σημειωθεί ότι αυτή η ιεράρχηση αφορά την καθαρή Python υλοποίηση και το συγκεκριμένο dataset, και όχι αναγκαστικά τον γενικό χαρακτήρα των αλγορίθμων.

Ιδιαίτερο ενδιαφέρον παρουσιάζει το Pigeonhole Sort, το οποίο λειτουργεί ως outlier της μελέτης. Το αποτέλεσμα ~ 2.32 δευτερολέπτων για $N=1.000.000$ δεν είναι απλώς καλό για έναν non-comparison αλγόριθμο, αλλά είναι ταχύτερο από αρκετούς $O(n \log n)$ comparison-based αλγορίθμους που εξετάστηκαν στην ενότητα 4.3 (για σύγκριση, ο Heap Sort χρειαζόταν ~ 25 δευτερόλεπτα, ο Merge Sort ~ 33 δευτερόλεπτα). Η εξήγηση βρίσκεται στη δομή του, γιατί το Pigeonhole Sort χρησιμοποιεί τις τιμές απευθείας ως δείκτες σε μια λίστα κάδων, πετυχαίνοντας τοποθέτηση $O(1)$ ανά στοιχείο σε ένα μόνο πέρασμα. Το αποτέλεσμα αυτό, βέβαια, ισχύει μόνο όταν το εύρος των τιμών (range) παραμένει συγκρίσιμο με το n . Σε περίπτωση που το εύρος εκτιναχθεί πολύ πάνω από το n , η κατανάλωση μνήμης γίνεται απαγορευτική.

Η καθυστέρηση της Radix Sort εξηγείται από τη δομή της, δεδομένου ότι πραγματοποιεί πολλαπλές διελεύσεις στο σύνολο των δεδομένων (ένα pass ανά ψηφίο), δημιουργώντας βοηθητικές δομές σε κάθε βήμα. Για n ακέραιους με d ψηφία, η πολυπλοκότητα είναι $O(d(n+k))$, που για μεγάλα n εκφυλίζεται πρακτικά σε $O(n \log n)$ (Cormen, 2009). Ο overhead των επαναλαμβανόμενων passes μεταφράζεται σε αισθητή καθυστέρηση στην Python, όπου ακόμη και η δημιουργία βοηθητικών λιστών έχει μη-αμελητέο κόστος.

Στο επίπεδο μνήμης, η εικόνα αντιστρέφεται μερικώς. Ο Counting Sort καταναλώνει ~ 53.4 MB για $N=1.000.000$ (υψηλότερη της ομάδας), ο Pigeonhole ~ 38.1 MB, και η Radix ~ 23.3 MB. Ο Counting Sort χρειάζεται έναν πίνακα μεγέθους k (το εύρος των τιμών) για να αποθηκεύσει τις συχνότητες, ανεξαρτήτως του πόσα στοιχεία υπάρχουν πραγματικά (Cormen, 2009). Το εύρος k αποτελεί έτσι τον κρίσιμο παράγοντα που καθορίζει αν κάθε αλγόριθμος της ομάδας είναι πρακτικά εφαρμόσιμος σε ένα δεδομένο πρόβλημα.

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Counting Sort	1.000	0.022	0.012	0.029	46.16
Counting Sort	10.000	0.079	0.103	0.079	538.63
Counting Sort	100.000	0.738	0.793	0.735	5460.93
Counting Sort	1.000.000	7.05	7.985	7.056	54679.86
Radix Sort	1.000	0.025	0.036	0.018	24.67
Radix Sort	10.000	0.241	0.233	0.25	239.83
Radix Sort	100.000	2.376	2.473	2.363	2345.11
Radix Sort	1.000.000	29.205	30.043	28.673	23876.11
Pigeonhole Sort	1.000	0.004	0.006	0.01	32.02
Pigeonhole Sort	10.000	0.05	0.048	0.054	383.37
Pigeonhole Sort	100.000	0.231	0.272	0.24	3898.45

Pigeonhole Sort	1.000.000	2.316	2.317	2.314	39054.59
-----------------	-----------	-------	-------	-------	----------

4.6.2 Αλγόριθμοι κάδων - Bucket, Flash Sort

Ο Bucket Sort και ο Flash Sort μοιράζονται την ίδια βασική λογική, διαμερίζουν τα στοιχεία σε κάδους βάσει της κατανομής τους και ταξινομούν κάθε κάδο ξεχωριστά. Η κρίσιμη διαφορά εντοπίζεται στον τρόπο ορισμού των κάδων και, κατ' επέκταση, στην ανοχή τους σε μη-ομοιόμορφες κατανομές.

Ο Bucket Sort παρουσιάζει μια αξιοσημείωτη ασυμμετρία που αποτελεί χαρακτηριστικό παράδειγμα αλγορίθμου με ισχυρή εξάρτηση από την κατανομή των δεδομένων. Στο Best case (ομοιόμορφη κατανομή), ολοκληρώνει ταξινόμηση 1.000.000 στοιχείων σε μόλις 3.72 δευτερόλεπτα, μια επίδοση που συγκρίνεται ευνοϊκά με την Counting Sort. Ωστόσο, στο Random και Worst case για το ίδιο μέγεθος, υπερβαίνει το χρονικό όριο (Time Limit). Η εξήγηση είναι άμεσα προβλέψιμη από τη θεωρία, δεδομένου ότι ο Bucket Sort επιτυγχάνει $O(n)$ μόνο υπό την υπόθεση ομοιόμορφης κατανομής. Όταν αυτή παραβιαστεί, οι κάδοι γεμίζουν ανισόμετρα, μετατρέποντας κάθε κάδο σε ένα μικρό $O(n^2)$ υποπρόβλημα (Cormen, 2009).

Ιδιαίτερο ενδιαφέρον παρουσιάζει η αντίστροφη ιεράρχηση Best/Random στον Flash Sort, καθώς τα δεδομένα αποκαλύπτουν ένα φαινόμενο που φαίνεται αντιφατικό, όπου το Random case (18.21 s για $N=1M$) αποδεικνύεται συνεπώς ταχύτερο από το Best case (23.28 s). Αυτή η αντιστροφή δεν αποτελεί σφάλμα μέτρησης, αλλά αντανακλά μια θεμελιώδη ιδιοτυπία του αλγορίθμου. Ο Flash Sort χρησιμοποιεί ομοιόμορφη κατανομή του εύρους τιμών για τον ορισμό των κάδων του (Neubert, 1998). Στη μέτρηση του "Best case" η είσοδος είναι ήδη ταξινομημένη, κάτι που οδηγεί σε ισχυρά unbalanced κάδους (τα στοιχεία συγκεντρώνονται στους τελευταίους κάδους), αυξάνοντας το κόστος της εσωτερικής insertion sort φάσης. Αντίθετα, μια τυχαία είσοδος πλησιάζει περισσότερο στην ομοιόμορφη κατανομή που αποτελεί το πραγματικό θεωρητικό Best case του Flash Sort.

Ο Flash Sort παραμένει ο πλέον αξιόπιστος της ομάδας σε όλα τα σενάρια, χωρίς καμία αστοχία. Η κατανάλωση μνήμης των δύο αλγορίθμων είναι παρόμοια, με τον Bucket Sort να απαιτεί ~15.96 MB και τον Flash Sort ~17.16 MB για $N=1.000.000$. Τα μεγέθη αυτά συγκρίνονται άμεσα με τη Merge Sort (~15.3 MB), επιβεβαιώνοντας ότι η κατανομή σε κάδους συνεπάγεται δέσμευση μνήμης ανάλογη της κλίμακας των δεδομένων.

Αλγόριθμος	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Bucket Sort	1.000	0.005	0.011	0.013	19.92
Bucket Sort	10.000	0.055	0.059	0.071	169.66
Bucket Sort	100.000	0.299	1.2	1.525	1729.63

Bucket Sort	1.000.000	3.724	Time Limit	Time Limit	16344.07
Flash Sort	1.000	0.044	0.049	0.045	14.15
Flash Sort	10.000	0.238	0.232	0.213	172.48
Flash Sort	100.000	2.379	1.795	2.064	1754.73
Flash Sort	1.000.000	23.276	18.207	19.809	17575.14

4.7 Συγκριτική Ανάλυση Αμιγούς Κώδικα Python και Βελτιστοποιημένων Υλοποιήσεων (Alt)

Εκτός από τους τριάντα βασικούς αλγορίθμους, η παρούσα εργασία περιλαμβάνει επτά βελτιστοποιημένες υλοποιήσεις (Alt), που αφορούν τους: Block Sort, Bucket Sort, Cartesian Tree Sort, Merge-Insertion Sort, Patience Sort, Tim Sort (Built-in) και Tournament Sort. Σκοπός τους δεν είναι η αλλαγή του αλγορίθμου, αλλά η αντικατάσταση επιλεγμένων υπορουτίνων που είχαν αναπτυχθεί εξ ολοκλήρου σε Python με ισοδύναμες C-επιταχυνόμενες συναρτήσεις από τις τυπικές βιβλιοθήκες (heapq, bisect, sorted()). Το θεωρητικό κόστος ανά πράξη παραμένει αμετάβλητο. Αυτό που αλλάζει είναι το σταθερό πολλαπλασιαστικό κόστος (constant factor), το οποίο στην Python είναι ιδιαίτερα υψηλό λόγω της υπολογιστικής επιβάρυνσης (overhead) του διεργασίας (Van Rossum & Drake, 2010).

Τα αποτελέσματα αποκαλύπτουν τρεις ευδιάκριτες κατηγορίες βελτίωσης. Η πρώτη αφορά αλγορίθμους όπου η βελτιστοποιημένη έκδοση (Alt) είναι χρονικά ισοδύναμη αλλά εξαλείφει τις αποτυχίες λόγω υπέρβασης χρονικού ορίου (Time Limit) (Block Sort, Bucket Sort, Merge-Insertion Sort). Η δεύτερη αφορά αλγορίθμους με αξιόλογη επιτάχυνση ανάλογα με το σενάριο εισόδου (Tournament Sort, Patience Sort, Cartesian Tree Sort). Η τρίτη κατηγορία αποτελείται από έναν μόνο αλγόριθμο, τον Tim Sort, του οποίου η βελτιστοποιημένη έκδοση (Alt) υπερτερεί κατά χιλιάδες φορές σε ταχύτητα εκτέλεσης.

4.7.1 Οριακή Επιτάχυνση και Εξάλειψη Υπερβάσεων Χρόνου

Στην πρώτη κατηγορία εντάσσονται τρεις αλγόριθμοι όπου η βελτιστοποιημένη έκδοση (Alt) δεν προσφέρει εντυπωσιακή επιτάχυνση, αλλά λύνει ένα πιο σημαντικό πρακτικό πρόβλημα, την αδυναμία ολοκλήρωσης εντός χρονικού ορίου.

Στο Block Sort (Alt), η αντικατάσταση της Insertion Sort που είχε αναπτυχθεί σε αμιγή κώδικα Python με την ενσωματωμένη sorted() δεν αποδίδει αξιόλογη επιτάχυνση, καθώς ο λόγος κυμαίνεται μόλις από 0.9x έως 1.36x για όλα τα μεγέθη εισόδου. Ωστόσο, η ίδια αλλαγή εξαλείφει τις δύο υπερβάσεις χρονικού ορίου (Time Limit) που εμφανίζονταν στο N=1.000.000 για τα σενάρια Random και Worst, με την εκτέλεση να ολοκληρώνεται πλέον σε 38.8 έως 42.2 δευτερόλεπτα. Η ερμηνεία είναι ότι η αμιγής Insertion Sort μέσα σε κάθε block αποτελούσε το πραγματικό σημείο συμφόρησης, ιδίως σε μη ταξινομημένες εισόδους, όπου ο αριθμός των εσωτερικών μετακινήσεων αυξάνεται σημαντικά.

Παρόμοιο μοτίβο παρατηρείται στο Bucket Sort (Alt). Η βελτίωση ταχύτητας είναι σχετικά περιορισμένη για μικρά μεγέθη εισόδου, με λόγο 0.5x έως 1.8x στα $N=1.000$ και $N=10.000$, αλλά γίνεται αισθητή στα 100.000 στοιχεία, όπου φτάνει τις 4x και 5.6x φορές για τα σενάρια Random και Worst αντίστοιχα. Το πιο σημαντικό αποτέλεσμα είναι ότι οι δύο υπερβάσεις χρονικού ορίου στο $N=1.000.000$ εξαλείφονται πλήρως, γεγονός που καθιστά τον αλγόριθμο πρακτικά χρησιμοποιήσιμο σε μεγάλα σύνολα δεδομένων.

Το Merge-Insertion Sort (Alt) αντικαθιστά μόνο τη δυαδική αναζήτηση με τη συνάρτηση `bisect.bisect_right` της αντίστοιχης βιβλιοθήκης. Επειδή η αναδρομική δομή του αλγορίθμου παραμένει εξ ολοκλήρου σε αμιγή κώδικα Python, η συνολική βελτίωση είναι οριακή, με λόγο 1.1x έως 1.7x. Το αποτέλεσμα αυτό αποτελεί χαρακτηριστικό παράδειγμα μιας γενικής αρχής, σύμφωνα με την οποία το κέρδος ταχύτητας από μια βελτιστοποιημένη βιβλιοθήκη περιορίζεται από το ποσοστό του συνολικού χρόνου εκτέλεσης που πράγματι δαπανάται σε αυτήν, όπως περιγράφει ο νόμος του Amdahl (Amdahl, 1967). Στην περίπτωση του Merge-Insertion Sort, το υπολογιστικό κόστος της αναδρομής σε αμιγή κώδικα Python κυριαρχεί χρονικά έναντι της δυαδικής αναζήτησης, με αποτέλεσμα η μερική αντικατάσταση σε C να μην μπορεί να μεταφραστεί σε αισθητή συνολική επιτάχυνση.

Αλγόριθμος	Έκδοση	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Block Sort	Αμιγής Python	1.000	0.037	0.038	0.032	16.05
	Βελτιστοποιημένη (Alt)	1.000	0.032	0.032	0.033	15.93
	Αμιγής Python	10.000	0.238	0.263	0.234	82.52
	Βελτιστοποιημένη (Alt)	10.000	0.264	0.22	0.236	81.68
	Αμιγής Python	100.000	3.485	4.603	4.478	1265.64
	Βελτιστοποιημένη (Alt)	100.000	3.369	3.641	3.278	1264.46
	Αμιγής Python	1.000.000	41.897	Time Limit	Time Limit	9633.33
	Βελτιστοποιημένη (Alt)	1.000.000	40.155	42.218	38.784	9625.46
Bucket Sort	Αμιγής Python	1.000	0.005	0.011	0.013	19.92
	Βελτιστοποιημένη (Alt)	1.000	0.01	0.006	0.01	20.16
	Αμιγής Python	10.000	0.055	0.059	0.071	169.66
	Βελτιστοποιημένη (Alt)	10.000	0.056	0.048	0.059	170.58
	Αμιγής Python	100.000	0.299	1.2	1.525	1729.63
	Βελτιστοποιημένη (Alt)	100.000	0.278	0.303	0.274	1732.16
	Αμιγής Python	1.000.000	3.724	Time Limit	Time Limit	16344.07

	Βελτιστοποιημένη (Alt)	1.000.000	2.812	2.879	2.801	16356.52
Merge-Insertion	Αμιγής Python	1.000	0.023	0.01	0.014	84.2
	Βελτιστοποιημένη (Alt)	1.000	0.02	0.003	0.014	84.19
	Αμιγής Python	10.000	0.187	0.165	0.144	804.03
	Βελτιστοποιημένη (Alt)	10.000	0.129	0.098	0.087	804.03
	Αμιγής Python	100.000	2.1	2.688	2.448	8048.66
	Βελτιστοποιημένη (Alt)	100.000	1.353	1.851	1.746	8048.66
	Αμιγής Python	1.000.000	59.03	109.378	103.536	79154.05
	Βελτιστοποιημένη (Alt)	1.000.000	50.407	99.438	92.583	79154.05

4.7.2 Σημαντική Επιτάχυνση μέσω Δομών Heap και Δυναδικής Αναζήτησης

Η δεύτερη κατηγορία περιλαμβάνει τρεις αλγορίθμους όπου η αντικατάσταση χειροκίνητων δομών με ισοδύναμες βιβλιοθήκες της Python αποφέρει αισθητή επιτάχυνση, η οποία όμως ποικίλλει σημαντικά ανάλογα με το σενάριο εισόδου.

Το Tournament Sort (Alt) αντικαθιστά την εξ ολοκλήρου χειροκίνητη υλοποίηση του min-heap (`py_heappush`, `py_heappop`) με τις συναρτήσεις `heapq.heappify()` και `heapq.heappop()`. Επειδή ο αλγόριθμος αποτελείται σχεδόν αποκλειστικά από πράξεις `heap`, το κέρδος είναι ουσιαστικά ολοκληρωτικό, καθώς στο $N=1.000.000$ ο χρόνος εκτέλεσης μειώνεται από 20 έως 27 δευτερόλεπτα σε μόλις 0.27 έως 1.19 δευτερόλεπτα, δηλαδή επιτάχυνση 19x έως 102x ανάλογα με το σενάριο. Αξίζει να σημειωθεί ότι το Worst case κερδίζει περισσότερο, με επιτάχυνση 102x, ακριβώς επειδή σε ήδη ταξινομημένες εισόδους η υλοποιημένη σε C δομή `heap` εκτελεί τις συγκρίσεις με ελάχιστο πρόσθετο κόστος.

Το Patience Sort (Alt) παρουσιάζει ανομοιόμορφα κέρδη που εξαρτώνται έντονα από το σενάριο εισόδου. Η βελτιστοποιημένη υλοποίηση αντικαθιστά τη `py_bisect_left` με την `bisect.bisect_left` σε C, και χρησιμοποιεί `heapq.merge()` αντί για το min-heap αμιγούς κώδικα Python κατά τη φάση συγχώνευσης. Στο Worst case (ήδη ταξινομημένη είσοδος) για $N=1.000.000$, η επιτάχυνση φτάνει τις 34x φορές, καθώς ο χρόνος εκτέλεσης μειώνεται από 4.03 σε 0.12 δευτερόλεπτα, αφού εκτελούνται ακριβώς 1.000.000 δυαδικές αναζητήσεις σε C. Στο Best case (αντίστροφα ταξινομημένη είσοδος), η επιτάχυνση είναι μικρότερη, μόλις 5.5x, και συνοδεύεται μάλιστα από αύξηση της κατανάλωσης μνήμης στα 202.924 KB, περίπου 198 MB, καθώς η `heapq.merge()` διαχειρίζεται ταυτόχρονα έναν σωρό πάνω από 1.000.000 μονοστοιχιακές στοίβες. Η τιμή αυτή ξεπερνά ακόμη και το αντίστοιχο μέγεθος της αρχικής αμιγούς υλοποίησης, που ήταν 126.301 KB, και αποτελεί τη μεγαλύτερη

καταγεγραμμένη κατανάλωση μνήμης σε ολόκληρο το σύνολο δεδομένων της εργασίας.

Για το Cartesian Tree Sort (Alt), η αντικατάσταση του χειροκίνητα υλοποιημένου σωρού με τη heap κατά τη διάσχιση του δέντρου ωφελεί αποκλειστικά το Random case, με επιτάχυνση 3.4x έως 5.7x ανάλογα με το μέγεθος εισόδου. Στα σενάρια Best και Worst case, η επιτάχυνση είναι ουσιαστικά μηδενική, μόλις 0.95x έως 1.1x. Η αιτία είναι ότι για ταξινομημένες εισόδους το πραγματικό σημείο συμφόρησης είναι η κατασκευή του ίδιου του Cartesian Tree, η οποία παραμένει βασισμένη σε αμιγή κώδικα Python, και όχι η διάσχιση μέσω heap που βελτιστοποιήθηκε.

Αλγόριθμος	Έκδοση	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Cartesian Tree	Αμιγής Python	1.000	0.014	0.019	0.017	121.64
	Βελτιστοποιημένη (Alt)	1.000	0.016	0.003	0.017	118.75
	Αμιγής Python	10.000	0.059	0.157	0.074	1168.01
	Βελτιστοποιημένη (Alt)	10.000	0.053	0.069	0.07	1165
	Αμιγής Python	100.000	0.366	1.956	0.384	11648.79
	Βελτιστοποιημένη (Alt)	100.000	0.351	0.484	0.389	11645.75
	Αμιγής Python	1.000.000	3.545	27.999	3.964	115961.61
	Βελτιστοποιημένη (Alt)	1.000.000	3.733	8.179	3.993	115958.56
Patience Sort	Αμιγής Python	1.000	0.038	0.012	0.023	122.02
	Βελτιστοποιημένη (Alt)	1.000	0.02	0.001	0.001	193.29
	Αμιγής Python	10.000	0.272	0.062	0.076	1259.88
	Βελτιστοποιημένη (Alt)	10.000	0.085	0.024	0.003	2021.96
	Αμιγής Python	100.000	3.573	1.329	0.429	12713.47
	Βελτιστοποιημένη (Alt)	100.000	0.736	0.159	0.026	20363.64
	Αμιγής Python	1.000.000	45.583	20.721	4.029	126301.38
	Βελτιστοποιημένη (Alt)	1.000.000	8.339	1.764	0.119	202924.21
Tournament Sort	Αμιγής Python	1.000	0.005	0.027	0.02	16.54
	Βελτιστοποιημένη (Alt)	1.000	0.001	0.001	0	16.48
	Αμιγής Python	10.000	0.141	0.146	0.14	161.38
	Βελτιστοποιημένη (Alt)	10.000	0.01	0.013	0.01	161.33

	Αμιγής Python	100.000	1.544	1.778	2.077	1563.54
	Βελτιστοποιημένη (Alt)	100.000	0.025	0.08	0.048	1563.48
	Αμιγής Python	1.000.000	20.21	22.521	27.377	16063.29
	Βελτιστοποιημένη (Alt)	1.000.000	0.332	1.19	0.268	16063.23

4.7.3 Tim Sort (Built-in) έναντι Simplified Tim Sort

Η τρίτη κατηγορία περιλαμβάνει μία μόνο περίπτωση, τον Tim Sort, όπου η διαφορά ανάμεσα στην αμιγή και τη βελτιστοποιημένη υλοποίηση δεν αφορά μια μεμονωμένη υπορουτίνα, όπως στις προηγούμενες ενότητες, αλλά ολόκληρο τον αλγόριθμο.

Η Built-in έκδοση αντιστοιχεί στην ενσωματωμένη συνάρτηση `sorted()` ή `list.sort()` της Python, η οποία υλοποιεί τον Timsort απευθείας σε C μέσα στον διερμηνέα CPython (Peters, 2002). Αυτό σημαίνει ότι δεν πρόκειται απλώς για επιτάχυνση μιας υπορουτίνας, αλλά για εκτέλεση ολόκληρου του αλγορίθμου εκτός Python bytecode. Αντίθετα, ο Simplified Tim Sort υλοποιεί χειροκίνητα την εύρεση των runs, τον υπολογισμό του minrun, τη διαχείριση της στοίβας συγχώνευσης και τις ίδιες τις πράξεις merge, ακολουθώντας πιστά τις προδιαγραφές του Peters (2002), αλλά φέρει το πλήρες υπολογιστικό βάρος του διερμηνέα Python.

Τα αποτελέσματα αναδεικνύουν με τον πιο ξεκάθαρο τρόπο σε όλη την εργασία το κόστος αυτού του overhead. Για $N=1.000.000$, στο Best case ο Tim Sort (Built-in) ολοκληρώνει την ταξινόμηση σε μόλις 0.008 δευτερόλεπτα, έναντι 20.26 δευτερολέπτων της Simplified έκδοσης, δηλαδή με επιτάχυνση περίπου 2.500 φορές. Στο Worst case η επιτάχυνση είναι της τάξης των 1.100 φορές, με τον χρόνο εκτέλεσης να πέφτει από 50.74 σε 0.046 δευτερόλεπτα. Στο Random case, η επιτάχυνση περιορίζεται σε περίπου 157 φορές, καθώς ο χρόνος μειώνεται από 34.50 σε 0.219 δευτερόλεπτα. Το γεγονός ότι το Random case κερδίζει αναλογικά λιγότερο δεν οφείλεται σε κάποια αδυναμία της βελτιστοποιημένης έκδοσης, αλλά στο ότι απαιτεί πράγματι πιο σύνθετες πράξεις συγχώνευσης, οι οποίες δεν μπορούν να παρακαμφθούν πλήρως ούτε από τον υλοποιημένο σε C Timsort.

Έκδοση	N	Best	Random	Worst	Μέγ. Μνήμη (KB)
Αμιγής Python (Simplified)	1.000	0.023	0.048	0.044	8.36
Built-in	1.000	0	0	0	3.9
Αμιγής Python (Simplified)	10.000	0.135	0.299	0.42	78.67
Built-in	10.000	0	0.003	0.001	39.09
Αμιγής Python (Simplified)	100.000	1.583	3.255	4.701	781.79
Built-in	100.000	0.001	0.041	0.01	390.49

Αμιγής Python (Simplified)	1.000.000	20.261	34.497	50.74	7813.04
Built-in	1.000.000	0.008	0.219	0.046	3906.04

4.7.4 Επίδραση της Βελτιστοποίησης στην Κατανάλωση Μνήμης

Σε αντίθεση με την ανάλυση χρόνου, η επίδραση των βελτιστοποιημένων υλοποιήσεων στην κατανάλωση μνήμης δεν ακολουθεί ενιαία κατεύθυνση, καθώς ορισμένοι αλγόριθμοι επωφελούνται, άλλοι επιβαρύνονται και άλλοι παραμένουν σχεδόν ανεπηρέαστοι. Αυτή η ασυμμετρία αναδεικνύει ότι η αντικατάσταση αμιγούς κώδικα Python με βιβλιοθήκες σε C δεν συνεπάγεται αυτόματα και μικρότερο μνημονικό αποτύπωμα, καθώς αυτό εξαρτάται από τον τρόπο εσωτερικής διαχείρισης δεδομένων της εκάστοτε συνάρτησης.

Η πιο χαρακτηριστική περίπτωση αύξησης μνήμης εντοπίζεται στο Patience Sort (Alt). Στο Best case (ήδη ταξινομημένη είσοδος), η χρήση της `heapq.merge()` οδηγεί σε κατανάλωση 202.924,21 KB, έναντι 126.301,38 KB της αμιγούς έκδοσης, σημειώνοντας αύξηση περίπου 60%. Η αιτία είναι ότι η `heapq.merge()` διατηρεί ταυτόχρονα ενεργό έναν σωρό πάνω από όλες τις μονοστοιχειακές στοίβες, κάτι που η χειροκίνητη υλοποίηση απέφευγε μέσω μιας απλούστερης διαχείρισης πινάκων. Το εύρημα αυτό είναι σημαντικό, γιατί αποδεικνύει ότι η επιδίωξη ταχύτητας μέσω μιας βιβλιοθήκης της C μπορεί να αντισταθμιστεί από δραματικά αυξημένη κατανάλωση μνήμης, ένας συμβιβασμός που δεν προκύπτει από τη θεωρητική πολυπλοκότητα $O(n)$ του αλγορίθμου.

Αντίθετα, στην πλειονότητα των εξεταζόμενων αλγορίθμων, η χρήση βελτιστοποιημένων βιβλιοθηκών ως υπορουτίνες δεν επέφερε καμία ουσιαστική μεταβολή. Ειδικότερα, για μέγεθος εισόδου 1.000.000 στοιχείων, η κατανάλωση μνήμης των Block Sort (Alt), Bucket Sort (Alt), Tournament Sort (Alt) και Cartesian Tree Sort (Alt) παρέμεινε πρακτικά ταυτόσημη, με απόκλιση μικρότερη του 0.1% σε σχέση με τις αντίστοιχες αμιγείς υλοποιήσεις. Ιδιαίτερο ενδιαφέρον παρουσιάζει το Merge-Insertion Sort, το οποίο απαιτεί γενικότερα τεράστια ποσά μνήμης. Η αντικατάσταση της χειροκίνητης δυαδικής αναζήτησης με την `bisect.bisect_right` δεν προσέφερε καμία απολύτως εξοικονόμηση, καθώς και οι δύο εκδόσεις κατέγραψαν ακριβώς 79.154,05 KB κατανάλωσης στο Best και Random case. Η παρατήρηση αυτή αποτελεί σαφή ένδειξη ότι το εξαιρετικά υψηλό μνημονικό κόστος του αλγορίθμου οφείλεται αποκλειστικά στη δομική διαχείριση των αλυσίδων εισαγωγής σε επίπεδο αντικειμένων της Python, και όχι στον τρόπο υλοποίησης της ίδιας της αναζήτησης.

Η μοναδική περίπτωση όπου καταγράφηκε δραματική μείωση της μνήμης ήταν ο Tim Sort. Καθώς η Built-in έκδοση (`sorted()`) εκτελεί το σύνολο του αλγορίθμου εκτός του Python interpreter, αποφεύγεται το overhead της δημιουργίας Python objects. Έτσι, ενώ η Simplified Python έκδοση καταναλώνει 7.813,04 KB για $N=1.000.000$, η ενσωματωμένη συνάρτηση απαιτεί μόλις 3.906,04 KB στο Random και Worst case, δηλαδή ακριβώς τη μισή μνήμη. Αξιοσημείωτο είναι το γεγονός ότι στο Best case, η

Built-in έκδοση δεσμεύει μόλις 0.05 KB βοηθητικού χώρου, καθώς αναγνωρίζει ακαριαία την ήδη ταξινομημένη ακολουθία χωρίς να δημιουργήσει επιπλέον εσωτερικές δομές.

Συνολικά, τα ευρήματα της ενότητας δείχνουν ότι η αξιολόγηση μιας βελτιστοποίησης δεν πρέπει να περιορίζεται στον χρόνο εκτέλεσης. Ένας αλγόριθμος μπορεί να γίνει σημαντικά ταχύτερος και ταυτόχρονα να καταστεί λιγότερο πρακτικός σε περιβάλλοντα με περιορισμένη διαθέσιμη μνήμη, όπως φάνηκε καθαρά στην περίπτωση του Patience Sort (Alt).

4.8 Συνολική Συγκριτική Αξιολόγηση

Οι ενότητες 4.2 έως 4.7 εξέτασαν κάθε κατηγορία αλγορίθμων ξεχωριστά, επικεντρωμένες στις εσωτερικές διαφορές κάθε ομάδας. Η παρούσα ενότητα έχει σκοπό να συνθέσει τα ευρήματα σε επίπεδο ολόκληρου του dataset, απαντώντας σε δύο θεμελιώδη ερωτήματα. Πρώτον, ποιο είναι το πραγματικό εύρος των αποκλίσεων στη μνήμη μεταξύ όλων των κατηγοριών. Δεύτερον, ποιος αλγόριθμος αποδεικνύεται η βέλτιστη επιλογή για κάθε πρακτικό σενάριο χρήσης.

4.8.1 Ανάλυση Μνήμης σε Όλες τις Κατηγορίες Αλγορίθμων

Η συγκριτική αξιολόγηση της κατανάλωσης μνήμης αναδεικνύει εξαιρετικά ενδιαφέροντα στοιχεία. Η πιο αξιοσημείωτη παρατήρηση αφορά το μεγάλο εύρος των αποκλίσεων μεταξύ των διαφορετικών αλγορίθμων. Για είσοδο ενός εκατομμυρίου στοιχείων, η ελάχιστη βοηθητική μνήμη καταγράφεται στον Gnome Sort. Ο συγκεκριμένος αλγόριθμος απαιτεί μόλις 0.11 KB. Αντίθετα, ο Patience Sort αγγίζει τα 126.301 KB στην καλύτερη περίπτωση (περίπου 123 MB). Η αναλογία ανάμεσα σε αυτές τις δύο ακραίες τιμές υπερβαίνει το ένα εκατομμύριο προς ένα. Συνεπώς, ένας αλγόριθμος μπορεί να καταναλώσει ένα εκατομμύριο φορές περισσότερη μνήμη από έναν άλλον για το ίδιο σύνολο δεδομένων. Αυτή η τεράστια απόκλιση δεν οφείλεται αποκλειστικά στη θεωρητική πολυπλοκότητα χώρου. Αποκαλύπτει παράλληλα το πραγματικό υπολογιστικό κόστος της κατασκευής και διαχείρισης σύνθετων δομών δεδομένων στην Python.

Με βάση τα πειραματικά δεδομένα, οι εξεταζόμενοι αλγόριθμοι διακρίνονται σε τρεις κατηγορίες ως προς τη συμπεριφορά τους στη μνήμη. Η πρώτη κατηγορία περιλαμβάνει προσεγγίσεις με σχεδόν μηδενική επιπλέον μνήμη. Η συνολική τους κατανάλωση δεν υπερβαίνει τα 6 KB για ένα εκατομμύριο στοιχεία. Από θεωρητική σκοπιά η ομάδα αυτή υποδιαιρείται σε δύο διακριτές ακαδημαϊκές κλάσεις. Στην πρώτη κλάση ανήκουν οι αλγόριθμοι αυστηρής επί τόπου εκτέλεσης με χωρική πολυπλοκότητα $O(1)$. Τέτοιοι είναι η πλειονότητα των αλγορίθμων τετραγωνικής πολυπλοκότητας (εξαιρουμένης της προκείμενης υλοποίησης του Pancake Sort). Σε αυτή την κλάση ανήκουν επίσης οι Heap Sort, Comb Sort, Shell Sort και Smooth Sort. Η κατανάλωση μνήμης τους παραμένει απόλυτα σταθερή. Η ανεξαρτησία από το μέγεθος της εισόδου αποδεικνύεται και από το αποτύπωμα του Gnome Sort (0.11

KB). Στη δεύτερη θεωρητική κλάση εντάσσονται οι αναδρομικοί αλγόριθμοι Quick Sort και Intro Sort. Μολονότι πρακτικά είναι εξαιρετικά αποδοτικοί, δεν εκτελούνται αμιγώς επί τόπου. Οι αλγόριθμοι αυτοί καταναλώνουν $O(\log n)$ βοηθητική μνήμη. Η μνήμη αυτή δεσμεύεται δυναμικά για τη συντήρηση της στοίβας των αναδρομικών τους κλήσεων (call stack). Έτσι, η κατανάλωση μνήμης παραμένει ουσιαστικά σταθερή και ανεξάρτητη από το μέγεθος της εισόδου. Το χαμηλό αποτέλεσμα του Gnome Sort (0.11 KB) δεν αποτελεί προϊόν εξειδικευμένης βελτιστοποίησης. Προκύπτει φυσικά από τη δομή του, καθώς ο αλγόριθμος δεν διατηρεί καμία βοηθητική δομή δεδομένων.

Η δεύτερη κατηγορία χαρακτηρίζεται από γραμμική πολυπλοκότητα χώρου. Η κατανάλωση μνήμης κυμαίνεται από 7 MB έως 55 MB για το μέγιστο μέγεθος εισόδου. Στην ομάδα αυτή εντάσσονται οι Merge Sort, Simplified Tim Sort, Strand Sort, Tournament Sort και Block Sort. Παράλληλα, στην ίδια κατηγορία ανήκουν τα δίκτυα ταξινόμησης Bitonic Sort και Pairwise Sorting Network. Την ομάδα συμπληρώνουν όλοι οι αλγόριθμοι χωρίς σύγκριση (Flash Sort, Bucket Sort, Radix Sort, Pigeonhole Sort και Counting Sort). Η γραμμική κλιμάκωση της μνήμης είναι απολύτως συμβατή με τις θεωρητικές εκτιμήσεις της βιβλιογραφίας. Παρουσιάζει ιδιαίτερο ενδιαφέρον η διαφορά ανάμεσα στον Merge Sort (15.6 MB) και στον Counting Sort (53.4 MB). Η απόκλιση αυτή δεν προέρχεται από διαφορετική κλάση πολυπλοκότητας. Οφείλεται αποκλειστικά στο μέγεθος των βοηθητικών δομών. Ο Counting Sort συντηρεί έναν πίνακα συχνοτήτων με μέγεθος ίσο προς το εύρος τιμών. Στο παρόν πείραμα, το εύρος τιμών ταυτίζεται με το μέγεθος εισόδου, προκαλώντας τη συγκεκριμένη αύξηση.

Η τρίτη κατηγορία περιλαμβάνει αλγορίθμους με ιδιαίτερα επιβαρυνόμενη συμπεριφορά. Αυτοί οι αλγόριθμοι ξεπερνούν σταθερά τα 75 MB σε κατανάλωση μνήμης για το μέγιστο μέγεθος εισόδου. Στην ομάδα αυτή συναντάμε τον Cartesian Tree Sort. Οι απαιτήσεις αυτού του αλγορίθμου κυμαίνονται από 102 έως 116 MB. Επίσης, περιλαμβάνεται ο Merge-Insertion Sort με κατανάλωση ακριβώς 79 MB. Ο κατάλογος συμπληρώνεται από τον Tree Sort στην τυχαία περίπτωση με περίπου 84 MB και τον Patience Sort στην καλύτερη περίπτωση με 123 MB. Οι τιμές αυτές υπερβαίνουν κατά πολύ την τυπική κατανάλωση άλλων αλγορίθμων γραμμικής πολυπλοκότητας χώρου. Η κοινή αιτία εντοπίζεται στην κατασκευή και συντήρηση σύνθετων αντικειμένων (objects) της Python. Τέτοιες δομές είναι οι κόμβοι των δέντρων, οι λίστες για τις στοίβες και οι αλυσίδες εισαγωγής. Κάθε αυτόνομο αντικείμενο επιφέρει σημαντική επιβάρυνση στο σύστημα διαχείρισης μνήμης του διεργαστή.

Η συμπεριφορά του Patience Sort παρουσιάζει το μεγαλύτερο ερευνητικό ενδιαφέρον μέσα σε αυτή την ομάδα. Ο αλγόριθμος εμφανίζει μια εντελώς αντίστροφη σχέση ανάμεσα στην καλύτερη και τη χειρότερη περίπτωση εισόδου. Όταν τα δεδομένα είναι ήδη ταξινομημένα (Best Case), κάθε νέο στοιχείο είναι μεγαλύτερο από τις υπάρχουσες κορυφές. Αυτό αναγκάζει τον αλγόριθμο να δημιουργήσει μια νέα στοίβα για κάθε στοιχείο. Η διαδικασία οδηγεί στη δημιουργία ενός εκατομμυρίου ξεχωριστών στοιβών, εκτοξεύοντας την κατανάλωση μνήμης στα 123 MB. Αντίθετα, στην αντίστροφη ταξινομημένη είσοδο (Worst Case), κάθε στοιχείο τοποθετείται στην

πρώτη στοίβα. Το γεγονός αυτό έχει ως αποτέλεσμα τη δημιουργία μίας και μοναδικής στοίβας. Έτσι, η συνολική χρήση μνήμης περιορίζεται δραματικά στα 24 MB. Αυτή η εντυπωσιακή διακύμανση αποτελεί ένα μοναδικό χαρακτηριστικό του αλγορίθμου στο σύνολο των πειραματικών δεδομένων.

Κατηγορία	Εύρος (N=1.000.000)	Αλγόριθμοι
Επί τόπου (In-place)	0.11 KB – 5.41 KB	Gnome Sort (0.11 KB), Insertion Sort (0.17 KB), Cocktail Shaker (0.20 KB), Bubble Sort (0.24 KB), Shell Sort (0.26 KB), Comb Sort (0.29 KB), Heap Sort (0.96 KB), Smooth Sort (1.95 KB), Quick Sort (3.16 KB), Intro Sort (5.41 KB)
Γραμμική - O(n)	7.6 MB – 53.4 MB	Tim Sort Simplified (7.6 MB), Bitonic Sort (9.4 MB), Pairwise Network (9.4 MB), Block Sort (9.4 MB), Merge Sort (15.3 MB), Tournament Sort (15.7 MB), Strand Sort (15.7 MB), Bucket Sort (16.0 MB), Flash Sort (17.2 MB), Radix Sort (23.3 MB), Pigeonhole Sort (38.1 MB), Counting Sort (53.4 MB)
Υψηλή Κατανάλωση	75 MB – 123 MB	Merge-Insertion Sort (77.3 MB), Tree Sort (83.9 MB), Cartesian Tree Sort (113.2 MB), Patience Sort (123.3 MB)

4.8.2 Βέλτιστες Αλγοριθμικές Επιλογές ανά Σενάριο Εκτέλεσης

Η αξιολόγηση της απόδοσης δείχνει ότι δεν υπάρχει μία καθολικά βέλτιστη επιλογή. Ο καταλληλότερος αλγόριθμος εξαρτάται από το πλήθος των στοιχείων, την αρχική κατανομή των δεδομένων και τους διαθέσιμους πόρους μνήμης. Ο παρακάτω πίνακας συγκεντρώνει τους ταχύτερους αλγορίθμους αμιγούς κώδικα ανά σενάριο για είσοδο ενός εκατομμυρίου στοιχείων. Η παρουσίαση διαχωρίζει τις μεθόδους με σύγκριση από τις μεθόδους χωρίς σύγκριση, ώστε η αντιπαραβολή να γίνεται αυστηρά εντός της ίδιας θεωρητικής κλάσης.

Σενάριο Εισόδου (N = 1.000.000)	Κατηγορία Αλγορίθμων	Ταχύτερος Αλγόριθμος	Χρόνος Εκτέλεσης (s)	Κατανάλωση Μνήμης (KB)
Best Case	Με Σύγκριση	Bubble Sort	0.82	0.24
	Χωρίς Σύγκριση	Pigeonhole Sort	2.32	39.054,59

Random Case	Με Σύγκριση	Tournament Sort	22.52	16.063,29
	Χωρίς Σύγκριση	Pigeonhole Sort	2.32	39.054,59
Worst Case	Με Σύγκριση	Quick Sort	19.14	1.97
	Χωρίς Σύγκριση	Pigeonhole Sort	2.32	39.054,59

Ο Pigeonhole Sort κυριαρχεί ολοκληρωτικά σε όλα τα σενάρια της κατηγορίας του. Καταγράφει σταθερό χρόνο περίπου 2.32 δευτερολέπτων, ανεξάρτητα από την αρχική διάταξη των δεδομένων. Η εξήγηση για αυτή την επίδοση πηγάζει από τη θεωρητική φύση της μεθόδου. Στο συγκεκριμένο πείραμα, το εύρος τιμών ταυτίζεται απόλυτα με το πλήθος των στοιχείων. Κατά συνέπεια, ο Pigeonhole Sort λειτουργεί σε γραμμικό χρόνο χωρίς να εκτελεί συγκρίσεις. Ωστόσο, αυτή η ταχύτητα συνεπάγεται αυξημένο χωρικό κόστος. Η συγκεκριμένη υλοποίηση απαιτεί 39.054,59 KB μνήμης για ένα εκατομμύριο στοιχεία. Το ποσό αυτό αποδεικνύεται περίπου 1.6 φορές μεγαλύτερο από τις απαιτήσεις του Radix Sort. Οι δύο αλγόριθμοι ανήκουν θεωρητικά στην ίδια κλάση χωρικής πολυπλοκότητας. Η σημαντική αυτή απόκλιση εξηγείται από την αρχιτεκτονική διαχείρισης αντικειμένων της Python. Ο Pigeonhole Sort δεσμεύει μονομιάς έναν εκτενή πίνακα με μέγεθος ίσο με το εύρος των τιμών. Αυτός ο πίνακας περιέχει χιλιάδες επιμέρους λίστες για τη φιλοξενία των στοιχείων. Αντίθετα, ο Radix Sort κατανέμει τα δεδομένα επαναληπτικά σε σταθερό αριθμό κάδων. Ο αλγόριθμος ανακυκλώνει τον χώρο σε κάθε ψηφιακό πέρασμα. Αυτή η στρατηγική μειώνει δραστικά το συνολικό αποτύπωμα των δεσμευμένων αναφορών μνήμης και έτσι, δικαιολογείται η μικρότερη κατανάλωση. Παράλληλα, είναι περίπου σαράντα χιλιάδες φορές μεγαλύτερη από την αντίστοιχη κατανάλωση του Heap Sort.

Η ανάλυση των αλγορίθμων σύγκρισης αποκαλύπτει τρία κρίσιμα ευρήματα που αναθεωρούν τις συμβατικές παραδοχές. Πρώτον, οι προσαρμοστικοί αλγόριθμοι τετραγωνικής πολυπλοκότητας κυριαρχούν απόλυτα στα ήδη ταξινομημένα δεδομένα μεγάλου όγκου. Ο Bubble Sort αναδεικνύεται ως η ταχύτερη επιλογή με χρόνο μόλις 0.82 δευτερόλεπτα. Η επίδοση αυτή επιβεβαιώνει την ικανότητά του να τερματίζει σε γραμμικό χρόνο μέσω ενός απλού περάσματος ελέγχου. Ωστόσο, υψηλή αυτή επίδοση είναι εξαιρετικά ασταθής στην πράξη (καθώς εξαρτάται από τη διάταξη των δεδομένων). Έστω και ελάχιστα αταξινόμητα στοιχεία υποβαθμίζουν δραματικά τον αλγόριθμο σε τετραγωνικό χρόνο. Αντίθετα, ο Smooth Sort καταγράφει μεγαλύτερο χρόνο με 5.62 δευτερόλεπτα. Παρόλα αυτά, υπερτερεί ξεκάθαρα σε πρακτική αξιοπιστία. Η απόδοσή του υποβαθμίζεται ομαλά σε λογαριθμικό επίπεδο, αν η διάταξη δεν είναι απολύτως τέλεια. Δεύτερον, ο Tournament Sort αναδεικνύεται ως ο ταχύτερος αλγόριθμος σύγκρισης για τυχαία δεδομένα (Random Case) με 22.52 δευτερόλεπτα. Με αυτή την επίδοση ξεπερνά οριακά τον Heap Sort (25.15 δευτερόλεπτα) και τον Quick Sort (25.32 δευτερόλεπτα). Τρίτον, στο πειραματικό σενάριο της αντίστροφα ταξινομημένης εισόδου (συμβατικά ονομασμένο Worst Case), ο Quick Sort τερματίζει ταχύτερα από τον Heap Sort. Ο αλγόριθμος καταγράφει χρόνο 19.14 δευτερόλεπτα. Το εύρημα αυτό ευθυγραμμίζεται απόλυτα με την κλασική αλγοριθμική θεωρία. Η εξήγηση βρίσκεται στις σχεδιαστικές επιλογές της υλοποίησης. Η επιλογή του κεντρικού στοιχείου ως άξονα αναφοράς μετατρέπει την αντίστροφη διάταξη σε ευνοϊκή ακολουθία. Η τεχνική αυτή εξασφαλίζει τέλειες

διχοτομήσεις. Με αυτόν τον τρόπο αποτρέπεται πλήρως η εκφυλισμένη διαμέριση. Μια τέτοια διαμέριση θα οδηγούσε στην πραγματική ασυμπτωτική χειρότερη περίπτωση του $O(n^2)$. Ο Quick Sort, αποφεύγοντας αυτό το μειονέκτημα, αναδεικνύει την εγγενή υπεροχή του έναντι του Heap Sort. Ο Heap Sort εγγυάται θεωρητικά τον ίδιο αριθμό συγκρίσεων, αλλά επωφελείται λιγότερο από την τοπικότητα της κρυφής μνήμης (cache locality). Αυτό συμβαίνει εξαιτίας των αλμάτων (μη γραμμικές προσπελάσεις) που προκαλεί η διαδικασία ολίσθησης προς τα κάτω (sift-down).

Ένα τελικό εύρημα αξίζει ιδιαίτερη προσοχή. Ο Heap Sort παραμένει η ιδανική επιλογή όταν υπάρχουν ταυτόχρονοι περιορισμοί ταχύτητας και μνήμης. Απαιτεί μόλις 0.96 KB για ένα εκατομμύριο στοιχεία. Παράλληλα, εγγυάται την αναμενόμενη πολυπλοκότητα σε κάθε πιθανό σενάριο. Ο συγκεκριμένος αλγόριθμος δεν παρουσίασε καμία αστοχία χρονικού ή αναδρομικού ορίου καθ' όλη τη διάρκεια του πειράματος. Συνδυάζει, επομένως, αξιοπιστία και αποδοτικότητα που κανένας άλλος αυτόνομος αλγόριθμος δεν προσφέρει αθροιστικά. Αντιθέτως, μέθοδοι όπως ο Counting Sort ή ο Pigeonhole Sort υπερτερούν σε καθαρή ταχύτητα, αλλά απαιτούν σημαντικά ποσά μνήμης. Επιπλέον, οι μέθοδοι χωρίς σύγκριση προϋποθέτουν εκ των προτέρων γνώση για το εύρος των τιμών. Στην πράξη, η τελική επιλογή αποτελεί πάντα ένα πρόβλημα βελτιστοποίησης υπό περιορισμούς. Τα εμπειρικά δεδομένα της παρούσας μελέτης παρέχουν το απαραίτητο ποσοτικό υπόβαθρο για την τεκμηρίωση αυτής της απόφασης.

ΚΕΦΑΛΑΙΟ 5 Συμπεράσματα

5.1 Σύνοψη Ερευνητικής Προσπάθειας

Η παρούσα εργασία είχε ως κεντρικό σκοπό τη συστηματική συγκριτική αξιολόγηση αλγορίθμων ταξινόμησης ακεραίων υλοποιημένων σε αμιγή κώδικα Python, με έμφαση τόσο στη χρονική απόδοση όσο και στην κατανάλωση μνήμης. Για τον σκοπό αυτό υλοποιήθηκαν 30 αλγόριθμοι σε καθαρή Python 3.13, οι οποίοι κατανέμονται σε έξι θεματικές οικογένειες, αλγόριθμοι $O(n^2)$, αλγόριθμοι $O(n \log n)$, δίκτυα ταξινόμησης, υβριδικοί αλγόριθμοι και αλγόριθμοι χωρίς σύγκριση. Παράλληλα, για 7 από τους αλγορίθμους σχεδιάστηκαν βελτιστοποιημένες υλοποιήσεις (Alt), οι οποίες αντικαθιστούν βασικές δομές δεδομένων της αρχικής χειροκίνητης υλοποίησης με βελτιστοποιημένες βιβλιοθήκες C επέκτασης (heapq, bisect), επιτρέποντας τη μέτρηση του overhead του Python interpreter.

Κάθε αλγόριθμος αξιολογήθηκε σε τέσσερα μεγέθη εισόδου (1.000, 10.000, 100.000 και 1.000.000 στοιχεία) και σε τρεις τύπους εισόδου (Best, Random και Worst Case), παράγοντας συνολικά 432 μετρήσεις για το σύνολο των αμιγών υλοποιήσεων. Οι μετρήσεις πραγματοποιήθηκαν σε σταθερό περιβάλλον (Intel Core i5-10400, 32 GB DDR4, WSL2), με χρήση των εργαλείων time.perf_counter για τον χρόνο εκτέλεσης και tracemalloc για την κατανάλωση μνήμης κορυφής. Από τις 432 μετρήσεις, 46 απέτυχαν λόγω υπέρβασης χρονικού ορίου (Time Limit) ή ορίου αναδρομής (Recursion Limit), ενώ τα δεδομένα που ολοκληρώθηκαν αναλύθηκαν συγκριτικά στο Κεφάλαιο 4.

5.2 Κύρια Ευρήματα

Η συγκριτική ανάλυση ανέδειξε μια σταθερή τάση στο περιβάλλον της γλώσσας Python. Η θεωρητική κατάταξη της πολυπλοκότητας των αλγορίθμων επιβεβαιώνεται ως προς την ασυμπτωτική της κλιμάκωση, ωστόσο οι απόλυτες επιδόσεις επηρεάζονται έντονα από τις σταθερές του περιβάλλοντος εκτέλεσης.

Χαρακτηριστικό παράδειγμα αποτελεί ο Heap Sort. Ο συγκεκριμένος αλγόριθμος αναδείχθηκε ως η πιο ισορροπημένη επιλογή λογαριθμικής πολυπλοκότητας στις εμπειρικές μετρήσεις. Η διάκριση αυτή επιτεύχθηκε παρά τις γνωστές αδυναμίες του ως προς την τοπικότητα των αναφορών (locality of reference). Για το μέγιστο μέγεθος εισόδου, ο Heap Sort κατανάλωσε μόλις 0.96 KB βοηθητικής μνήμης. Παράλληλα, παρουσίασε σταθερή χρονική απόδοση, ανεξάρτητα από τον τύπο της εισόδου. Συγκεκριμένα, η χειρότερη περίπτωση απαίτησε 22.8 δευτερόλεπτα. Η καλύτερη περίπτωση ολοκληρώθηκε σε 25.3 δευτερόλεπτα. Από την άλλη πλευρά βρέθηκαν αλγόριθμοι με σημαντικά θεωρητικά πλεονεκτήματα, όπως ο Block Sort και ο Merge-Insertion Sort. Ο Block Sort απέτυχε να ολοκληρώσει τις μετρήσεις για ένα εκατομμύριο στοιχεία στην τυχαία και στη χειρότερη περίπτωση λόγω υπέρβασης του χρονικού ορίου. Αντίθετα, ο Merge-Insertion Sort (Ford-Johnson) ολοκλήρωσε τις αντίστοιχες εκτελέσεις, εμφανίζοντας όμως τεράστιες χρονικές καθυστερήσεις. Ο

αλγόριθμος σημείωσε 109.38 δευτερόλεπτα στην τυχαία περίπτωση και 103.54 δευτερόλεπτα στη χειρότερη. Μολονότι εγγυάται θεωρητικά τον ελάχιστο δυνατό αριθμό συγκρίσεων, η απόδοσή του μειώθηκε σημαντικά από το εξαιρετικά υψηλό λειτουργικό κόστος της διαχείρισης των δομών. Η διαρκής κατασκευή ενδιάμεσων αλυσίδων εισαγωγής (insertion chains) και η αδιάκοπη εκτέλεση πράξεων εισαγωγής (list.insert) εντός των δυναμικών πινάκων της Python επέφεραν μαζικές γραμμικές μετατοπίσεις δεικτών στη μνήμη. Το αθροιστικό υπολογιστικό κόστος αυτών των μετατοπίσεων είναι αναλογικό του $O(n)$ ανά εισαγωγή. Αυτό το κόστος ξεπέρασε εντελώς το θεωρητικό κέρδος της μείωσης των συγκρίσεων, καθιστώντας τον αλγόριθμο μη βιώσιμο για μεγάλα σύνολα δεδομένων.

Η αντιπαραβολή του αμιγούς κώδικα με τις βελτιστοποιημένες παραλλαγές ανέδειξε τη σημασία της υπολογιστικής επιβάρυνσης (overhead) του διερμηνέα. Το φαινόμενο αυτό γίνεται έντονο όταν οι κρίσιμες λειτουργίες δεν υποστηρίζονται από βελτιστοποιημένες επεκτάσεις σε γλώσσα C. Η πιο εντυπωσιακή απόκλιση καταγράφηκε κατά τη σύγκριση της απλοποιημένης έκδοσης του Tim Sort με την ενσωματωμένη συνάρτηση της γλώσσας. Στην καλύτερη περίπτωση εισόδου, η επιτάχυνση άγγιξε το επίπεδο των 2.500 φορές. Το εύρημα αυτό φανερώνει το μεγάλο κόστος του διερμηνέα σε αλγορίθμους με σύνθετη λογική διαχείρισης λιστών. Ανάλογη συμπεριφορά παρουσίασε και η βελτιστοποιημένη έκδοση του Tournament Sort. Η συγκεκριμένη παραλλαγή πέτυχε επιτάχυνση από 19 έως 102 φορές σε σχέση με την αρχική αμιγή υλοποίηση. Για την επίδοση αυτή χρειάστηκε απλώς η αξιοποίηση της έτοιμης βιβλιοθήκης διαχείρισης σωρού (heapq). Επιπλέον, η χρήση της βιβλιοθήκης δυαδικής αναζήτησης (bisect) στον Patience Sort μείωσε αισθητά τον χρόνο της χειρότερης περίπτωσης για ένα εκατομμύριο στοιχεία. Η εκτέλεση περιορίστηκε από τα 4.03 δευτερόλεπτα στα 0.12 δευτερόλεπτα. Η αλλαγή αυτή οδήγησε σε σημαντική επιτάχυνση της τάξης των περίπου 34 φορές. Τα συγκεκριμένα ευρήματα αποδεικνύουν ότι για την πρακτική αξιοποίηση της θεωρητικής πολυπλοκότητας, ο τρόπος της υλοποίησης και η στρατηγική επιλογή βελτιστοποιημένων βιβλιοθηκών αποτελούν καθοριστικούς παράγοντες στο πλαίσιο της Python.

Ορισμένοι αλγόριθμοι παρουσίασαν μη αναμενόμενη συμπεριφορά στην κατανάλωση μνήμης. Ο Pancake Sort αποτελεί θεωρητικά έναν αλγόριθμο επί τόπου εκτέλεσης με σταθερή πολυπλοκότητα χώρου. Ωστόσο, για μέγεθος 10.000 στοιχείων κατέγραψε κατανάλωση 78.33 KB. Το αποτέλεσμα αυτό οφείλεται στα προσωρινά αντίγραφα λιστών που δημιουργεί αυτόματα η Python σε κάθε πράξη αναστροφής. Ο Patience Sort αποκάλυψε μια εντελώς αντίστροφη σχέση ανάμεσα στην καλύτερη και τη χειρότερη περίπτωση. Για ένα εκατομμύριο στοιχεία, η καλύτερη περίπτωση κατανάλωσε 123 MB. Η αυξημένη αυτή τιμή οφείλεται στην ανάγκη δημιουργίας ισάριθμων στοιβών. Αντίθετα, η χειρότερη περίπτωση περιορίστηκε στα 24 MB. Στο συγκεκριμένο σενάριο, όλα τα στοιχεία τοποθετήθηκαν σε μία μοναδική στοιβή. Παρόμοια συμπεριφορά εμφάνισε και ο Counting Sort. Η μέθοδος αυτή κατανάλωσε 53.4 MB μνήμης. Η τιμή αυτή προέκυψε επειδή το εύρος των τιμών ορίστηκε ίσο με το πλήθος των στοιχείων. Το γεγονός αυτό φανερώνει ότι η θεωρητική πολυπλοκότητα χώρου επηρεάζεται άμεσα από τον σχεδιασμό του πειράματος. Δεν καθορίζεται αποκλειστικά από τον ίδιο τον αλγόριθμο.

Ο Bitonic Sort και το Pairwise Sorting Network επιβεβαίωσαν πειραματικά τους περιορισμούς τους. Η θεωρητική πολυπλοκότητά τους καθιστά αυτές τις μεθόδους ακατάλληλες για σειριακή εκτέλεση. Για ένα εκατομμύριο στοιχεία, οι χρόνοι ολοκλήρωσης κυμάνθηκαν από 141 έως 163 δευτερόλεπτα. Οι επιδόσεις αυτές είναι περίπου έξι με επτά φορές βραδύτερες σε σύγκριση με τον Heap Sort. Το συγκεκριμένο εύρημα θεωρείται απολύτως αναμενόμενο. Η αρχιτεκτονική των δικτύων ταξινόμησης στοχεύει στη μείωση του βάθους παραλληλοποίησης. Δεν αποσκοπεί στον περιορισμό του συνολικού αριθμού των σειριακών πράξεων. Στην κατηγορία των μεθόδων χωρίς σύγκριση, ο Flash Sort παρουσίασε μια αξιοσημείωτη ιδιομορφία. Η θεωρητικά καλύτερη περίπτωση αποδείχθηκε βραδύτερη από την τυχαία περίπτωση για ένα εκατομμύριο στοιχεία. Η εκτέλεση σημείωσε 23.28 δευτερόλεπτα έναντι 18.21 δευτερολέπτων. Η ομοιόμορφη κατανομή μιας γραμμικά αυξανόμενης ακολουθίας διασφαλίζει τέλεια ισορροπημένη κατηγοριοποίηση (classification) στους κάδους. Επομένως, η καθυστέρηση αυτή δεν οφείλεται σε αλγοριθμική αστοχία. Αντίθετα, πηγάζει εξολοκλήρου από την εσωτερική υπολογιστική επιβάρυνση (instruction dispatch overhead) του διερμηνέα της Python. Κατά την εκτέλεση της φάσης των επί τόπου μεταθέσεων (in-situ permutation) και της επακόλουθης ταξινόμησης με εισαγωγή, ο εικονικός μηχανισμός της γλώσσας αναγκάζεται να αξιολογήσει σειριακά εκατομμύρια λογικές συνθήκες βρόχων. Η διαδικασία αυτή γίνεται απλώς για να επιβεβαιώσει ότι τα στοιχεία βρίσκονται ήδη στις τελικές τους θέσεις. Η Python αδυνατεί να αξιοποιήσει την πρόβλεψη διακλαδώσεων (branch prediction) σε επίπεδο υλικού με τον βέλτιστο τρόπο ενός μεταγλωττιστή. Ως αποτέλεσμα, οι αμέτρητοι κενοί έλεγχοι συσσωρεύουν σημαντικό χρόνο εκτέλεσης.

5.3 Πρακτικές Συστάσεις

Τα αποτελέσματα της παρούσας εργασίας επιτρέπουν τη διατύπωση συγκεκριμένων οδηγιών για την επιλογή του κατάλληλου αλγορίθμου ταξινόμησης στην Python. Η τελική απόφαση εξαρτάται άμεσα από τις ιδιαίτερες απαιτήσεις της εκάστοτε εφαρμογής. Η επιλογή αυτή δεν είναι καθολική. Κριτήρια όπως το μέγεθος των δεδομένων, οι περιορισμοί μνήμης, η φύση της εισόδου και η ανάγκη για σταθερότητα οδηγούν σε διαφορετικές αποφάσεις. Για τον λόγο αυτό, η δημιουργία ενός συγκεντρωτικού πίνακα κρίνεται απαραίτητη για τη γρήγορη λήψη αποφάσεων σε πρακτικό επίπεδο. Ο πίνακας αυτός λειτουργεί ως ένας οδηγός αναφοράς. Συνδυάζει τα εμπειρικά ευρήματα με τις πραγματικές ανάγκες ενός δημιουργού λογισμικού.

Συνθήκη Χρήσης (Σενάριο)	Προτεινόμενος Αλγόριθμος	Κύριος Λόγος Επιλογής
Ήδη ταξινομημένα δεδομένα (Best Case)	Smooth Sort ή Tim Sort (Built-in) ή Bubble Sort (early-exit)	Εκμετάλλευση της υπάρχουσας τάξης (προσαρμοστικότητα) με γραμμική πολυπλοκότητα χρόνου.

Αυστηρά περιορισμένη μνήμη και μεγάλο μέγεθος εισόδου	Heap Sort	Εγγυημένος χρόνος $O(n \log n)$ με επί τόπου (in-place) εκτέλεση και ελάχιστο χωρικό κόστος (0.96 KB).
Ανάγκη για σταθερότητα (Stability) και άγνωστη μορφή εισόδου	Merge Sort ή Tim Sort (Built-in)	Διατήρηση της σχετικής σειράς των ίσων στοιχείων του πίνακα με εγγυημένη χρονική απόδοση.
Ακέραιοι αριθμοί μικρού και εκ των προτέρων γνωστού εύρους τιμών	Pigeonhole Sort ή Counting Sort	Παράκαμψη του θεωρητικού φράγματος των συγκρίσεων $O(n \log n)$, επιτυγχάνοντας κορυφαία ταχύτητα.
Συχνή χρήση δομών σωρού ή λειτουργιών δυαδικής αναζήτησης	Βελτιστοποιημένες τύπου Alt (χρήση των <code>heapq</code> , <code>bisect</code>)	Δραστική μείωση του σταθερού υπολογιστικού κόστους και της επιβάρυνσης (overhead) που εισάγει ο διερμηνέας της Python.

Ένα επιπλέον κρίσιμο συμπέρασμα προκύπτει από τη σύγκριση του αμιγούς κώδικα με τις βελτιστοποιημένες παραλλαγές. Ορισμένες εφαρμογές απαιτούν επαναλαμβανόμενη χρήση σωρού ή δυαδικής αναζήτησης. Η χειροκίνητη ανάπτυξη αυτών των δομών επιφέρει τεράστια καθυστέρηση. Η αντικατάστασή τους με τις έτοιμες βιβλιοθήκες της γλώσσας (όπως η `heapq` και η `bisect`) προσφέρει επιτάχυνση δεκάδων φορές. Το κέρδος αυτό επιτυγχάνεται χωρίς καμία απολύτως αλλαγή στον γενικό αλγοριθμικό σχεδιασμό. Οι συγκεκριμένες βιβλιοθήκες αποτελούν βελτιστοποιημένες επεκτάσεις σε γλώσσα C. Συνεπώς, η βαθιά γνώση των ενσωματωμένων δυνατοτήτων της Python αναδεικνύεται σε έναν παράγοντα εξίσου σημαντικό με την επιλογή του αλγορίθμου.

Η έρευνα δείχνει επίσης ότι ορισμένοι αλγόριθμοι απαιτούν εναλλακτικές πλατφόρμες εκτέλεσης για την ανάδειξη των πλεονεκτημάτων τους. Τα δίκτυα ταξινόμησης, όπως ο Bitonic Sort, παρουσίασαν ιδιαίτερα χαμηλές επιδόσεις κατά τη σειριακή εκτέλεση στον επεξεργαστή. Ωστόσο, η στατική φύση των συγκρίσεών τους τα καθιστά ιδανικά για παράλληλη επεξεργασία. Πολλά σύγχρονα περιβάλλοντα απαιτούν μαζική επεξεργασία δεδομένων. Η μεταφορά της εκτέλεσης σε κάρτες γραφικών με χρήση εξειδικευμένων αρχιτεκτονικών προσφέρει καθοριστική βελτίωση της ταχύτητας σε αυτές τις περιπτώσεις. Η αξιοποίηση τέτοιων τεχνολογιών επιτρέπει την ταυτόχρονη εκτέλεση χιλιάδων συγκρίσεων. Με αυτόν τον τρόπο, μια μειονεκτική σειριακή δομή μετατρέπεται σε μια εξαιρετικά αποδοτική παράλληλη λύση.

5.4 Περιορισμοί και Προτάσεις Μελλοντικής Έρευνας

Οι μεθοδολογικοί και τεχνικοί περιορισμοί της παρούσας εργασίας αναλύθηκαν εκτενώς στο τρίτο κεφάλαιο. Στους περιορισμούς αυτούς περιλαμβάνονται η χρήση του περιβάλλοντος WSL2, οι απλοποιημένες εκδοχές ορισμένων αλγορίθμων και η αποκλειστική επιλογή ακεραίων αριθμών ομοιόμορφης κατανομής. Τα συγκεκριμένα στοιχεία δεν συνιστούν απλώς μεθοδολογικές επιφυλάξεις για την ερμηνεία των

αποτελεσμάτων. Αντίθετα, υποδεικνύουν σαφείς και συγκεκριμένες κατευθύνσεις για τη μελλοντική επέκταση της ερευνητικής προσπάθειας.

Η παρούσα εργασία επικεντρώθηκε στην ποσοτικοποίηση της επιβάρυνσης που προκαλεί ο διερμηνέας της Python. Η μελέτη αυτή υλοποιήθηκε μέσω της σύγκρισης των αμιγών υλοποιήσεων με τις βελτιστοποιημένες παραλλαγές (εκδόσεις Alt). Μια μελλοντική επέκταση της έρευνας σε ένα μεταγλωττισμένο περιβάλλον θα προσέφερε πολύτιμες διευκρινίσεις. Η προσέγγιση αυτή θα επέτρεπε τον ακριβή διαχωρισμό της παρατηρούμενης καθυστέρησης σε δύο διακριτούς παράγοντες. Ο πρώτος παράγοντας αφορά τις εγγενείς αδυναμίες του διερμηνέα. Ο δεύτερος σχετίζεται καθαρά με τον αλγοριθμικό σχεδιασμό των μεθόδων.

Μια άλλη σημαντική κατεύθυνση αφορά τη μετάβαση σε παράλληλες υλοποιήσεις, η οποία όμως απαιτεί εξαιρετική προσοχή στο περιβάλλον της Python. Η χρήση συμβατικών νημάτων (threading) καθίσταται πρακτικά απαγορευτική λόγω της ύπαρξης του Global Interpreter Lock (GIL). Το GIL αποτρέπει την ταυτόχρονη εκτέλεση κώδικα από πολλαπλά νήματα σε εργασίες υψηλής υπολογιστικής έντασης (CPU-bound). Αυτός ο περιορισμός μετατρέπει το πολυνηματικό μοντέλο σε μια διαδικασία που προσθέτει απλώς κόστος εναλλαγής πλαισίου. Το τελικό αποτέλεσμα είναι η επιβράδυνση της ταξινόμησης. Συνεπώς, η μελλοντική έρευνα οφείλει να επικεντρωθεί αποκλειστικά σε τεχνικές πολυεπεξεργασίας (multiprocessing). Παράλληλα, είναι επιβεβλημένη η χρήση προηγμένων δομών κοινόχρηστης μνήμης. Αυτή η πρακτική αποτρέπει το απαγορευτικό κόστος αντιγραφής δεδομένων κατά τη δια-διεργασιακή επικοινωνία (IPC overhead). Μόνο μέσω αυτής της αυστηρής προσέγγισης καθίσταται δυνατή η αξιολόγηση της συμπεριφοράς των αλγορίθμων σε σύγχρονες πολυπύρηνες αρχιτεκτονικές.

Η αξιοποίηση των δικτύων ταξινόμησης σε κάρτες γραφικών με χρήση της τεχνολογίας CUDA αποτελεί μια εξαιρετικά ενδιαφέρουσα πρόταση. Ο Bitonic Sort και το Pairwise Sorting Network κατέγραψαν τους υψηλότερους χρόνους εκτέλεσης (και συνεπώς τις χειρότερες επιδόσεις) κατά τη σειριακή εκτέλεση στον επεξεργαστή. Ωστόσο, το προκαθορισμένο και ντετερμινιστικό μοτίβο των συγκρίσεων τους τα καθιστά ιδανικά για μαζικά παράλληλη εκτέλεση σε αρχιτεκτονικές SIMD. Η υλοποίησή τους σε περιβάλλον GPU θα επέτρεπε την ανάδειξη των πραγματικών τους πλεονεκτημάτων. Τα πλεονεκτήματα αυτά δεν μπορούν να αξιοποιηθούν στο σειριακό μοντέλο εκτέλεσης.

Τέλος, η επέκταση των μετρήσεων σε διαφορετικούς τύπους δεδομένων θα συμπλήρωνε την εικόνα για την πρακτική αξία κάθε αλγορίθμου. Στους τύπους αυτούς περιλαμβάνονται οι αριθμοί κινητής υποδιαστολής (floating-point), οι συμβολοσειρές (strings) και διάφορα σύνθετα αντικείμενα (objects). Η ανάγκη αυτή γίνεται ιδιαίτερα εμφανής στους αλγορίθμους χωρίς σύγκριση. Η απόδοση αυτών των μεθόδων εξαρτάται και επηρεάζεται άμεσα από τα χαρακτηριστικά της κατανομής των δεδομένων. Παράλληλα, η μελέτη της εξωτερικής ταξινόμησης (external sorting) αποτελεί μια αναγκαία κατεύθυνση. Η διερεύνηση αυτή αφορά σύνολα δεδομένων που υπερβαίνουν τη διαθέσιμη μνήμη RAM, προσφέροντας λύσεις για την αντιμετώπιση προβλημάτων πραγματικά μεγάλης κλίμακας.

BIBΛΙΟΓΡΑΦΙΑ

- Aldous, D., & Diaconis, P. (1999). Longest increasing subsequences: From patience sorting to the Baik-Deift-Johansson theorem. *Bulletin of the American Mathematical Society*, 36(4), 413–432.
<https://doi.org/10.1090/S0273-0979-99-00796-X>
- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference on - AFIPS '67 (Spring)*, 483.
<https://doi.org/10.1145/1465482.1465560>
- Auger, N., Jugé, V., Nicaud, C., & Pivoteau, C. (2018). On the Worst-Case Complexity of TimSort [Application/pdf]. *LIPICs, Volume 112, ESA 2018*, 112, 4:1-4:13. <https://doi.org/10.4230/LIPICS.ESA.2018.4>
- Batcher, K. E. (1968). Sorting networks and their applications. *Proceedings of the April 30--May 2, 1968, Spring Joint Computer Conference on - AFIPS '68 (Spring)*, 307. <https://doi.org/10.1145/1468075.1468121>
- Beazley, D. M. (2009). *Python essential reference* (4th ed). Addison-Wesley.
- Chandramouli, B., & Goldstein, J. (2014). Patience is a virtue: Revisiting merge and sort on modern processors. *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, 731–742.
<https://doi.org/10.1145/2588555.2593662>

Cormen, T. H. (Επιμ.). (2009). *Introduction to algorithms* (3rd ed). MIT Press.

Cormen, T. H. (Επιμ.). (2001). *Introduction to algorithms—2001* (2nd ed). MIT Press.

Dijkstra, E. W. (1982). Smoothsort, an alternative for sorting in situ. *Science of Computer Programming*, 1(3), 223–233.

[https://doi.org/10.1016/0167-6423\(82\)90016-8](https://doi.org/10.1016/0167-6423(82)90016-8)

Dobosiewicz, W. (1980). An efficient variation of bubble sort. *Information Processing Letters*, 11(1), 5–6.

[https://doi.org/10.1016/0020-0190\(80\)90022-8](https://doi.org/10.1016/0020-0190(80)90022-8)

Floyd, R. W. (1964). Algorithm 245: Treesort. *Communications of the ACM*, 7(12), 701. <https://doi.org/10.1145/355588.365103>

Ford, L. R., & Johnson, S. M. (1959). A Tournament Problem. *The American Mathematical Monthly*, 66(5), 387–389.

<https://doi.org/10.1080/00029890.1959.11989306>

Gates, W. H., & Papadimitriou, C. H. (1979). Bounds for sorting by prefix reversal. *Discrete Mathematics*, 27(1), 47–57.

[https://doi.org/10.1016/0012-365X\(79\)90068-2](https://doi.org/10.1016/0012-365X(79)90068-2)

Georges, A., Buytaert, D., & Eeckhout, L. (2007). Statistically rigorous java performance evaluation. *SIGPLAN Not.*, 42(10), 57–76.

<https://doi.org/10.1145/1297105.1297033>

- Haddon, B. K. (1990). Cycle-Sort: A Linear Sorting Method. *The Computer Journal*, 33(4), 365–367. <https://doi.org/10.1093/comjnl/33.4.365>
- Haj-Yihia, J., Yasin, A., Asher, Y. B., & Mendelson, A. (2016). Fine-Grain Power Breakdown of Modern Out-of-Order Cores and Its Implications on Skylake-Based Systems. *ACM Transactions on Architecture and Code Optimization*, 13(4), 1–25. <https://doi.org/10.1145/3018112>
- Hamid Sarbazi-Azad. (2000). *Stupid Sort: A new sorting algorithm*. (599), 4.
- Hibbard, T. N. (1963). An empirical study of minimal storage sorting. *Communications of the ACM*, 6(5), 206–213. <https://doi.org/10.1145/366552.366557>
- Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1), 10–16. <https://doi.org/10.1093/comjnl/5.1.10>
- Huang, B.-C., & Langston, M. A. (1988). Practical in-place merging. *Communications of the ACM*, 31(3), 348–352. <https://doi.org/10.1145/42392.42403>
- Introduction to algorithms* (2nd. ed). (2001). MIT press.
- Isaac, E. J., & Singleton, R. C. (1956). Sorting by Address Calculation. *Journal of the ACM*, 3(3), 169–174. <https://doi.org/10.1145/320831.320834>
- Karl-Dietrich Neubert. (1998). *Dr. Dobb's Journal February 1998: The Flashsort1 Algorithm*. <https://www.neubert.net/Flapaper/9802n.htm>

- Kim, P.-S., & Kutzner, A. (2008). Ratio based stable in-place merging. *Proceedings of the 5th international conference on Theory and applications of models of computation, TAMC'08*, 246–257.
- Knuth, D. E. (1997). *The art of computer programming* (3rd ed). Addison-Wesley.
- Lacey, S., & Box, R. (1991). A fast, easy sort. *BYTE*, 16(4), 315-ff.
<https://doi.org/10.5555/117187.117218>
- Levcopoulos, C., & Petersson, O. (1989). Heapsort—Adapted for presorted files. Στο F. Dehne, J.-R. Sack, & N. Santoro (Επιμ.), *Algorithms and Data Structures* (τ. 382, σελ. 499–509). Springer Berlin Heidelberg.
https://doi.org/10.1007/3-540-51542-9_41
- Mallows, C. L. (1962). Patience Sorting. *SIAM Review*, 4(2), 148–149.
<https://doi.org/10.1137/1004036>
- Musser, D. R. (1997). Introspective Sorting and Selection Algorithms. *Software: Practice and Experience*, 27(8), 983–993.
[https://doi.org/10.1002/\(SICD\)1097-024X\(199708\)27:8%253C983::AID-SP E117%253E3.0.CO;2-%2523](https://doi.org/10.1002/(SICD)1097-024X(199708)27:8%253C983::AID-SP E117%253E3.0.CO;2-%2523)
- Parberry, I. (1992). THE PAIRWISE SORTING NETWORK. *Parallel Processing Letters*, 02(02n03), 205–211.
<https://doi.org/10.1142/S0129626492000337>

Peters, T. (2002). *Tim Sort*. GitHub.

<https://github.com/python/cpython/blob/main/Objects/listsort.txt>

Python Software Foundation. (2024a). *bisect—Array bisection algorithm*.

Python Documentation. <https://docs.python.org/3/library/bisect.html>

Python Software Foundation. (2024b). *heapq—Heap queue algorithm*. Python

Documentation. <https://docs.python.org/3/library/heapq.html>

Python Software Foundation. (2024c). *random—Generate pseudo-random numbers*. Python Documentation.

<https://docs.python.org/3/library/random.html>

Python Software Foundation. (2024d). *time—Time access and conversions*.

Python Documentation. <https://docs.python.org/3/library/time.html>

Python Software Foundation. (2024e). *tracemalloc—Trace memory allocations*. Python Documentation.

<https://docs.python.org/3/library/tracemalloc.html>

Rossum, G. van, & Drake, F. L. (2010). *The Python language reference*

(Release 3.0.1 [Repr.]). Python Software Foundation.

Satish, N., Harris, M., & Garland, M. (2009). Designing efficient sorting algorithms for manycore GPUs. *2009 IEEE International Symposium on Parallel & Distributed Processing*, 1–10.

<https://doi.org/10.1109/IPDPS.2009.5161005>

- Sedgewick, R. (1996). Analysis of Shellsort and Related Algorithms. *Proceedings of the Fourth Annual European Symposium on Algorithms, ESA '96*, 1–11.
- Seward, H. H. (1954). *Information sorting in the application of electronic digital computers to business operations*.
- Shell, D. L. (1959). A high-speed sorting procedure. *Communications of the ACM*, 2(7), 30–32. <https://doi.org/10.1145/368370.368387>
- Vuillemin, J. (1980). A unifying look at data structures. *Communications of the ACM*, 23(4), 229–239. <https://doi.org/10.1145/358841.358852>
- Williams, J. W. J. (1964). Algorithm 232: Heapsort. *Communications of the ACM*, 7(6), 347–348. <https://doi.org/10.1145/512274.3734138>